



Department of Computer Science

BSc (Hons) Computer Science

Academic Year 2023 - 2024

Utilising artificial intelligence to provide accessibility enhancements at railway stations for passengers with impaired vision

Saen Kazak

2113786

**A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science**

Brunel University London
Department of Computer Science
Uxbridge
Middlesex
UB8 3PH
United Kingdom
T: +44 1895 203397
F: +44 (0) 1895 25168

Abstract

This project attempts to leverage artificial intelligence-based techniques in order to enhance accessibility standards for visually impaired individuals navigating railway stations. This involved the design, implementation and testing of a system that utilises computer vision technologies to learn an instance segmentation model for platform edges and tracks at railway stations in order to conduct model inference on a smartphone camera feed via a web-based interface. Stereo audio channelling is used to feed this information back to the end user in an accessible format. Developing the system involved extensive research into the field of accessible design, frameworks for usability and acceptability, and significant levels of both participant-based and self-based testing conducted via a Hybrid Agile-Waterfall approach to comply with rigid and non-changing user requirements, with an iterative approach used for development cycles. The project places equal importance on hardware and software components, with rigorous evaluation into performance on different smartphone hardware conducted and extensive future work proposals including performance enhancements in both areas considered. The project outcome was a successful system that pertains to the original objectives and doubles as both a functional solution for the problems at stake and an insightful proof-of-concept into the use of deep learning technologies within this field.

Acknowledgements

I would like to thank my supervisor, Dr Stephen Swift, as well as second marker, Dr Zear Ibrahim, for assisting and aiding me in the creation of this project.

Additional thanks given to friends Rose, Billy, and Isolde for inspiring me to delve into this field of research, which has always been of interest to me.

I would also like to thank my roommates Charlie, Sophie, Riley, David, and Corky the rabbit for the motivation and support provided during the phases of the project.

Thanks also given to all friends and students who participated in the user testing.

I certify that the work presented in this dissertation is my own, except where explicitly referenced.

Signature 

Date 21/03/2024

Word Count (Main Document): 14500

Page Count (Main Document): 66

Table of Contents

Abstract	2
Acknowledgements	3
Table of Contents	4
List of Figures.....	7
List of Tables	9
1 Introduction	10
1.1 Project Synopsis.....	10
1.1.1 Problem Definition.....	10
1.1.2 Project Aim	11
1.1.3 Project Objectives.....	11
1.1.4 Dissertation Outline	12
2 Background.....	13
2.1 Project Domain	13
2.1.1 'Further Work' Sections: Can we Improve?.....	13
2.1.2 Acceptability: Designing for Success.....	14
2.1.3 Evaluative Measures: Adherence to Design Standards.....	15
2.1.4 Developing for Accessibility: Scaling Effect.....	16
2.1.5 Challenges in Accessible Development	17
2.1.6 Usability Scope.....	19
2.1.7 Self-Evaluations.....	20
2.1.8 Scaling for New Use Cases.....	20
2.2 Working with Images	21
2.2.1 Thresholding.....	21
2.2.2 Otsu's Thresholding.....	22
2.3 Working with Models	23
2.3.1 Computer Vision.....	23
2.3.2 Object Detection.....	24
2.3.3 Instance and Semantic Segmentation	25
3 Approach	27
3.1 Project Approach.....	27
3.2 Project Methodology.....	28
3.2.1 Choice of Methodology.....	28
3.2.2 Methodology Justification and Analysis.....	28

Utilising artificial intelligence to provide accessibility enhancements at railway stations for passengers with impaired vision	1.1
3.2.3 <i>Combining Waterfall and Agile Principles</i>	29
3.3 Project Ethics	29
4 Design	30
4.1 System Architecture	30
4.2 User Interface Design	31
4.2.1 <i>Designing Visual Aspects</i>	31
4.2.2 <i>Managing Audio Integration</i>	32
4.2.3 <i>Decision-Making and Optimisation</i>	33
4.3 Building Datasets	34
4.3.1 <i>Roboflow: Training Data</i>	34
4.3.2 <i>Roboflow: Annotations</i>	36
5 Implementation	37
5.1 Chapter Overview	37
5.1.1 <i>System Diagram and Discussion</i>	37
5.2 Model Training	38
5.2.1 <i>Training: Google Colab and Jupyter</i>	38
5.2.2 <i>Interpreting Model Results: Precision, Recall and F1-Score</i>	39
5.2.3 <i>Performance Evaluation Sample</i>	41
5.2.4 <i>Iterative Training</i>	41
5.3 Camera Streaming	42
5.3.1 <i>Web Cameras: DroidCam</i>	42
5.3.2 <i>OpenCV: Visual Displays</i>	42
5.4 Camera Integration	45
5.4.1 <i>WebRTC and AIORTC</i>	45
5.4.2 <i>Web UI Development: Streamlining Sample Code</i>	45
5.4.3 <i>Integrating WebRTC and Python</i>	46
5.4.4 <i>HTTPS: Protocol Certificate Errors</i>	49
5.5 GPU Inferencing	49
5.5.1 <i>CUDA and CuDNN: GPU Setup</i>	50
5.5.2 <i>Cross-Compatibility: Challenges</i>	50
5.5.3 <i>Second Prototype: Demonstration</i>	51
5.6 Audio Implementation	52
5.6.1 <i>Initial Research: Audio Frameworks</i>	52
5.6.2 <i>Working with Data Channels</i>	53
5.6.3 <i>Detections: Left and Right</i>	53

Utilising artificial intelligence to provide accessibility enhancements at railway stations for passengers with impaired vision	1.1
6 Testing.....	55
6.1 Participant Testing	55
6.1.1 Testing Formats.....	55
6.1.2 Setup and Limitations.....	57
6.1.3 Participant Testing Results.....	57
7 Evaluation	60
7.1 First Prototype: Case Study.....	60
7.1.1 Model Results.....	60
7.1.2 Problems Encountered.....	61
7.1.3 Next Steps.....	62
7.2 Second Prototype: Case Study	63
7.2.1 Comparative Model Testing.....	63
7.2.2 Problems Encountered.....	65
7.2.3 Next Steps.....	65
7.3 Final Prototype: Notes	66
7.3.1 Final Model Results.....	66
7.3.2 Future Considerations.....	67
7.4 Heuristic Evaluation.....	68
7.4.1 System Usability Scale.....	68
7.4.2 Evaluation of Results.....	69
7.4.3 Comment Feedback.....	70
8 Future Work	71
8.1 Customisation of Models	71
8.1.1 Tweaking Architecture and Re-Training	71
8.1.2 Colour-Based Convolution	71
8.2 Customisation of Filtering	72
8.3 Other Enhancements	72
8.3.1 Using Multiple Data Streams.....	72
8.3.2 Depth Perception for Enhancement of Results.....	73
8.3.3 Focus Reversal: Visual Information for Audially Impaired Passengers.....	73
9 Conclusion.....	74
10 References.....	75
APPENDIX A : REFLECTION.....	78
APPENDIX B : ETHICS CONFIRMATION	80
APPENDIX C OTHER MATERIAL	82
APPENDIX D : MADE IN BRUNEL	107

List of Figures

Figure 1 - Dissertation Outline.....	12
Figure 2 - The 'Curb Cut' Effect. Source: (Washington County, OR, n.d.).....	17
Figure 3 - Designing for Accessibility – From ‘Task 4 – Presentations’.....	18
Figure 4 – Use-Case Diagram: passing through accessible actors.....	18
Figure 5 - Thresholding pseudocode.....	21
Figure 6 - Binary thresholding with a threshold value of 127	21
Figure 7 - Binary and Otsu Comparison.....	22
Figure 8 – Types of computer vision modelling Source: NanoNets.....	23
Figure 9 - Object Detection on a football field. Source: Roboflow	24
Figure 10 - Object Detection Output Pseudocode.....	25
Figure 11 - Semantic Segmentation (left) vs Instance Segmentation (right) Source: Roboflow...25	
Figure 12 - Project Flow	27
Figure 13 - Modified Agile-Waterfall Hybrid Diagram.....	28
Figure 14 - 4-step process adapted from ‘Task 4 – Presentation’ (Kazak, 2024)	30
Figure 15 - Component Integrations	30
Figure 16 - High-level mock-up of the system design.....	31
Figure 17 - Per-frame activity diagram for audio system.....	32
Figure 18 - Activity diagram for frame inference	33
Figure 19 - Project dataset within Roboflow, split into train/test/validate categories	34
Figure 20 - Roboflow Deployment Page; shows iterative measures. Source: Roboflow	35
Figure 21 - Annotating within Roboflow.....	36
Figure 22 - Expanded System Diagram (adapted from ‘Task 4 – Presentation’)	37
Figure 23 - Colab-based training	39
Figure 24 - Sample F1-Confidence Curve	41
Figure 25 - Sample Precision-Recall Curve	41
Figure 26 - Python code for hooking into a DroidCam feed	42
Figure 27 - Instantiating the model in Python.....	43
Figure 28 - Model inference on a single frame	43
Figure 29 - Image & Mask Coordinate Manipulation with OpenCV.....	44
Figure 30 - Sample platform inference on a single frame. Source image: DC-Kits	44
Figure 31 - Default UI & Suggested Changes. Source: AIORTC.....	46
Figure 32 - VideoTransformTrack construction code	47

Utilising artificial intelligence to provide accessibility enhancements at railway stations for passengers with impaired vision	1.1
Figure 33 - VideoTransformTrack Instantiation Code	48
Figure 34 - DroidCam p2p Setup	48
Figure 35 - Console output on ICE peer connections	49
Figure 36 - GPU tool descriptions	50
Figure 37 - Dependency graph for GPU inference	50
Figure 38 - Second Prototype Demonstration	51
Figure 39 - Information pipelines	52
Figure 40 - Sending and receipt of data channel messages	53
Figure 41 - L/R Logic	54
Figure 42 - Box Testing Methods. (Adapted from www.ranorex.com)	55
Figure 43 - Testing Setup	57
Figure 44 - First Prototype Model Results	60
Figure 45 - First Prototype: Next Steps	62
Figure 46 - Precision-Confidence comparison between sets	63
Figure 47 - Recall-Confidence comparison between sets	64
Figure 48 - Second Prototype: Next Steps	65
Figure 49 - SUS Q1	69
Figure 50 - SUS Q3	69
Figure 51 - SUS Q8	70
Figure 52 - Band-based Otsu filtering	72
Figure 53 - Multiple Streams Proposal	72
Figure 54 - Depth Perception in AI. Source image: (K-Tech, 2021)	73
Figure 55 - Waterfall Model	85
Figure 56 - Agile Model	85
Figure 57 - YOLO Model Architecture. Source: YOLO	86
Figure 58 - Underfitting vs Overfitting (Amazon, 2024, p. 33)	92
Figure 8 - Adaptive Median works well in low light	95

List of Tables

Table 1 - Practical Acceptability definition, adapted from (Keates, 2007 p.3)	15
Table 2 - Limited scope definition based on non-functional constraints	20
Table 3 - Software choices and explanation	38
Table 4 - Model performance evaluation metrics	40
Table 5 - Participant testing procedure	56
Table 6 - Subsection of participant test results.....	59
Table 7 - Measurements & Descriptions.....	61
Table 8 - First Prototype: Problems Encountered.....	62
Table 9 - Second Prototype: Problems Encountered	65
Table 10 - SUS Survey Questionnaire.....	68
Table 11 - Project Objective Conclusions.....	74
Table 12 - Comment Feedback	91
Table 13 - Common Hyperparameters used in Machine Learning	93
Table 14 - Software Versions.....	94

1 Introduction

Within the context of rail travel, and more generally within society, individuals with partial vision or cognitive impairments often face significant challenges with independent navigation. UK railway stations are a specific area of concern due to the heightened safety profile of these environments; particularly as these environments often contain large amounts of infrastructure, people, obstacles, and hazards, many of which would be apparent to a fully sighted person, but less so for someone with partial sight.

1.1 Project Synopsis

Transport policies implemented over the last ten years have led to a significant push from railway operators to reduce staffing levels at stations; an effect exacerbated by the decline of passenger use following COVID. Research conducted by the Rail, Maritime and Transport Union (RMT) found that “just 11% of stations are staffed from first to last train, while a further 45% are only partially staffed, which often means for just a few hours during commuting peak and the rest are totally unstaffed” (RMT, 2021).

In addition to this, an investigation conducted by the DPTAC (Disabled Persons Transport Advisory Committee) found that Great British Railways has failed to deliver on two core recommendations provided for accessible safety:

- 1) Guaranteed staffing levels to provide unbooked ‘turn up and go’ assistance
 - 2) A £6 billion investment for full station accessibility by 2060
- (DPTAC, 2019)

The combined effect of many stations being left unstaffed during the day, a lack of step-free access, increased crowding (RMT, 2021) and a general decline in service quality since the pandemic (DPTAC, 2019) significantly diminishes safety standards, justifying the need for improvement.

1.1.1 Problem Definition

Rail travel is an essential part of many individuals’ lives. Approximately 397 million rail passenger journeys were recorded between July and September of 2023 (Office of Rail & Road, 2019). Despite the wide use of railways in Britain. However, evidence from 2017 looking at mobility impairments shows a disparity in the number of journeys made by disabled passengers in contrast to their non-disabled counterparts. Reports suggest that for every five passenger journeys made by the average non-disabled person, only two were made by their average

disabled counterparts (DPTAC, 2019). It can therefore be inferred from this data that disabled individuals are being disincentivised from railway travel due to the unresolved issues present in station and train environments. (Section 2.1.4 describes how accessibility enhancements can benefit everyone; not just the initially intended userbase.)

1.1.2 Project Aim

The aim of this project is to provide a proof-of-concept for the use of artificial intelligence-based systems within the context of railway accessibility, through the creation of a real-time camera-based deep learning inference system. The project will emphasise the use of image-based recognition to provide a software solution that can detect platform edges and tracks at railway stations, with extensive research conducted into image filtering and image augmentation techniques, designed with an access focus and self-evaluated to ensure compliance to accredited accessibility standards. The goal is to create a working system, complemented by extensive insight into future proposals that could be incorporated in order to develop a feasible system that has real-world potential to benefit lives.

1.1.3 Project Objectives

The project aim can be broken down into objectives that must be met. These objectives are as follows:

- 1) To design, develop and implement a system that assists visually impaired users with navigation of railway stations.
- 2) To expand upon shortfalls and successes in background research, analysing both design aspects and similar existing technologies.
- 3) To follow a software development methodology throughout the lifecycle of project.
- 4) To create a scalable proof-of-concept implementation that can be expanded upon through an extensive insight into future work and evaluation.
- 5) To evaluate feedback from testing stages and adjust the requirements, as necessary in order to refine the system to ensure elevated levels of precision and accuracy are complied with as far as practicably possible.
- 6) To design with a focus on continual improvement in attempts to support as broad a range of specialised requirements as possible, with a focus on users with partial sight.
- 7) To create UI prototypes for the system that are adapted to the user base, with a focus on simplicity and navigability.

1.1.4 Dissertation Outline

This document is divided into three main subsections and an appendix, with each subsection building on the research and insight covered previously. The appendix provides extra information. Figure 1 below shows the dissertation outline, with colour-coded sections to signify the various stages.

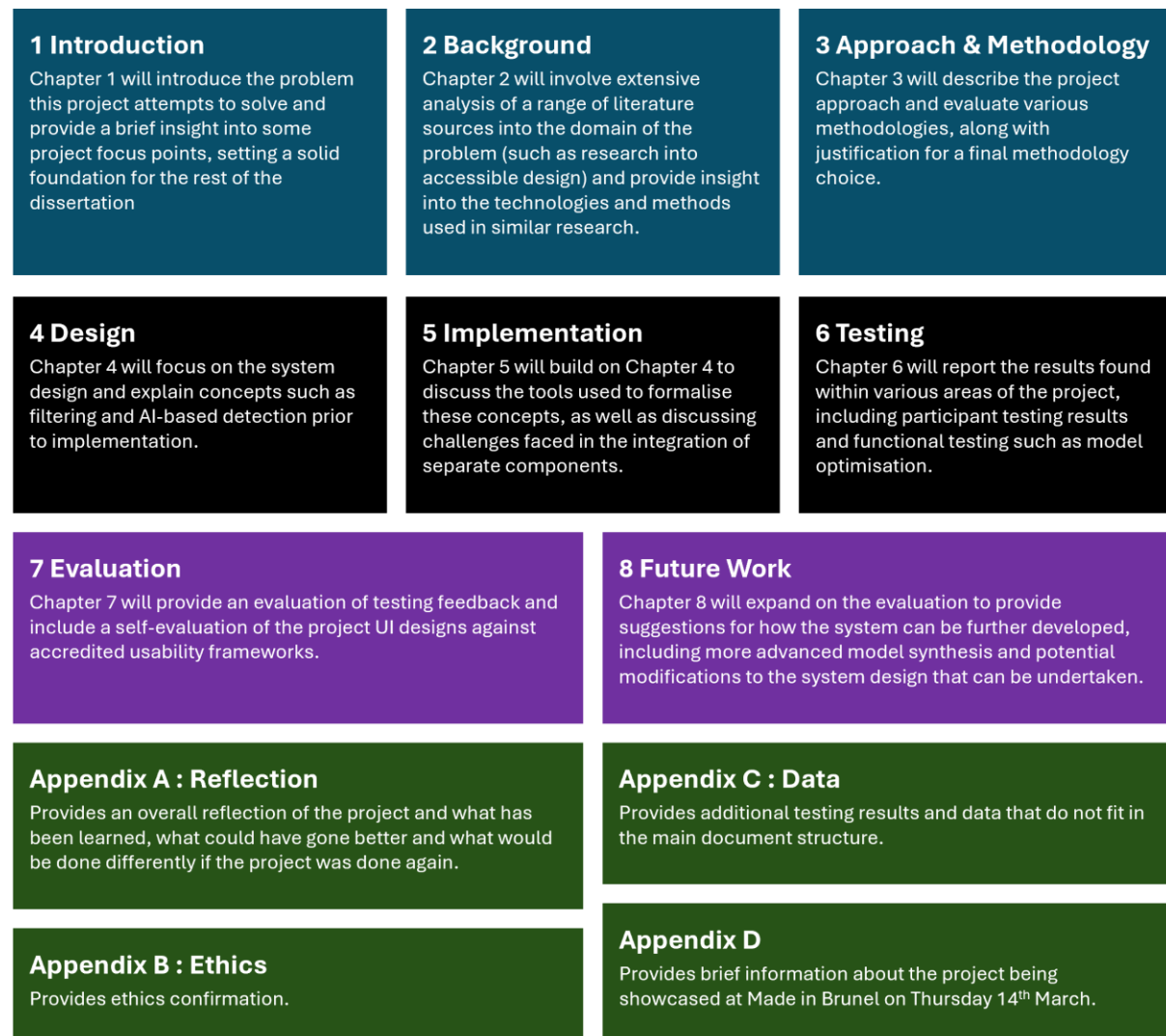


Figure 1 - Dissertation Outline

2 Background

This chapter provides an extensive analysis of a range of literature relevant to the domain of the problem (including research into accessible design). Similar studies for design are considered to provide a grounding in the field. Later sections of this chapter will provide technical information regarding the technologies and methods researched to fulfil the project objectives.

2.1 Project Domain

Although analysis of background literature is always important within research, it is vital in a project such as this one. It is important to exercise due care during design and development for access requirements, considering the impact on users. Examining background literature helps to contextualize and comprehend the scope of the problem and gain valuable insight into the limitations and challenges, questions asked, and answers received through previous findings.

2.1.1 ‘Further Work’ Sections: Can we Improve?

Existing literature within similar areas of interest will often contain a ‘further work’ section, where research teams discuss their findings, and provide a checkpoint that can be expanded upon. Careful reading of ‘further work’ sections can help foster innovative ideas for feature development – and patterns discovered across different avenues can help to find and address research gaps. This applies to both usability improvements such as UI design, and technical aspects such as process streamlining and object detection speeds.

With regard to the above question, the answer is almost always yes. Research papers of quality will clarify in detail where improvements can be made so they can be interpreted and understood by the reader.

The rest of this section will review relevant background literature. It is important to make use of a diverse range of literature sources, including books, frameworks, and research papers to provide a broad range of perspectives. Research across time periods can aid in this as well for the delivery. Since this project has an accessibility focus, sources from accredited institutions such as the National Institute of the Blind have also been examined, as well as contextualisation of the problem via research conducted on public service data, such as from TfL.

2.1.2 Acceptability: Designing for Success

Research into this area shows that functional software alone is not enough to incentivise an effective solution. In addition to meeting usability requirements, it is also crucial that systems are designed with an ‘acceptability’-context.

Keates discusses this in the book *Designing for Accessibility – A Business Guide to Countering Design Exclusion*, building on from prior usability research done by Nielsen discussing the concepts of Social Acceptability and Practical Acceptability. To quote Keates directly:

Social acceptability, as defined by Nielsen and used here, is achieved when the product meets the expectations and aspirations of the end-user. We can think of these as the ‘user wants.’

Social acceptability addresses issues such as:

- Does the product look nice?
- Do I trust this product?
- Does this product stigmatize me in any way?
- Do I want this product?

Obtaining and interpreting those requirements needs specialized approaches. As would be expected, it is not possible to provide a single definition of what a user wants from a product or service. For example, some people want a product that draws attention to itself, such as a memory aid or prompt to take medication, while others would prefer something more discreet, especially for a product to be used in public. (Keates, 2007)

General research conducted into the stigma around disability shows that 32% of respondents in a study agreed that “there is a lot of prejudice against disabled people in Britain”, while only 22% of non-disabled respondents agreed (Dixon, et al., 2018). In retrospect, it is undeniable that the third point regarding ‘stigmatization’ must be considered very carefully, especially as the project will not be evaluated by individuals with uncorrected visual impairments (see sub-section 2.1.6).

It is also important to accurately assess various factors of the project’s design, such as the need for individuals to hold up a mobile device with the camera active, or the requirement for headphones if there is audio implementation. Camera use in public could trigger stigmatic behaviours, making the challenges in navigation harder, rather than easier. In addition, some users with visual impairment may have other disabilities such as reduced motor function and struggle to hold a camera, for example those using mobility canes.

In any case, this project will provide a proof-of-concept for the use of an artificially trained model within the field of accessible navigation, and could help to inform future work, despite its true practical functionality and given the constraints on testing.

Keates goes on to discuss practical acceptability, focusing on functionality. Nielsen divides practical acceptability into four categories. Practical acceptability also links heavily to this project’s core objectives, particularly with regards to utility and usability.

Cost – Will the system be expensive to scale and implement?	
Compatibility – Will the system work for users on a range of different devices?	
Reliability – Will the system work without unexpectedly breaking?	
Usefulness – <i>subdivided into...</i>	
Usability – <i>defined as the provision of the necessary functionality by the product to perform the desired task</i>	Utility – <i>defined as including ease of learning, efficiency of use, ease to remembering, and low (user) error rates</i>

Table 1 - Practical Acceptability definition, adapted from (Keates, 2007 p.3)

For compatibility assurances, Nielsen’s requirements can be contextualised to the project use case by ensuring the camera inference pipeline is resistant to failure and supports a diverse range of hardware. This can be further applied to reliability; dips in framerate or resolution must be accounted for, especially if the system is using model inference on a video stream, which can fail with inadequate quality or connectivity. Failure detection is essential, and this must be relayed to a user. The system must also be easily navigable and require a minimal cognitive load to operate. Acute adherence to design principles such as Nielsen’s own frameworks and guidelines published by accredited institutions can help in ensuring this.

2.1.3 Evaluative Measures: Adherence to Design Standards

In addition to Nielsen’s frameworks, various studies have been conducted in order to construct design heuristics for specific problems. Analysis of a broad range of previous research design heuristics helps in determining conflicts or points of interest that our user base may have and provides an insightful comparison into Nielsen’s accepted standard.

One such study is an investigation into accessible mobile application design, conducted by Garaj, et al. in 2007. The study was a two-phase experiment attempting to produce a set of design heuristics tailored to mobile development. Making use of four participants with visual impairment, the time taken to perform specific smartphone tasks of various complexity across different applications was recorded. The results found that it was difficult in many cases for the participants to complete the tasks, and in some cases, they were unable to.

As stated in the report:

The participants reported a total of 63 accessibility problems, and 22 problems emerged after removing overlapping items (Table 5). The participants showed the highest success rates with Task 4 (KakaoTalk) and Task 5 (Facebook). Three of them successfully complete the given tasks. Overall, the successful completion rate was low (29.17%).

Specific problems with various applications were also recorded:

Task 6. VocalPost

- 1) Cannot start recording with Voiceover*
- 2) There is a useless non-clickable button*
- 3) When system sound volume is modified by a thirdparty app, it is difficult to hear the sound of Voiceover.*
- 4) Cannot move a focus to some button and text.*
- 5) Illogical focus sequence*
- 6) Cannot read a button name*

(Garaj, et al., 2007)

It is important to note that this study was conducted in 2007, when smartphone technology was in early stages. However, issues surrounding buttons use and focusing is still relevant today since these technologies are used extensively.

Analysis completed is useful when implementing and designing; a good example is problem (6). When designing the system, metrics such as these can be used to say e.g. the use of buttons may not be beneficial to our user base. Tools also exist to evaluate the accessibility of an application. The heuristics determined in the outcome of this study are detailed in Appendix C.2.

2.1.4 Developing for Accessibility: Scaling Effect

A report published by the ORR states:

Those who would benefit from audio information (65%) (...) were significantly less likely to be very satisfied than the overall sample (72%). This may indicate that these accommodations are not in place to ensure the needs of these groups are being met during the booking process. (Office of Rail & Road, 2023, p.31).

Research into accessible design in this area is beneficial due to the current under-representation of this focus group within existing solutions. Statistics from 2022-2023 show that 69% of individuals requiring visual impairment-based assistance are 'very satisfied' that the assistance currently available is relevant to their needs, compared to 76% of individuals without a reported disability or condition. (Office of Rail & Road, 2023, p.30). In addition to this,

many of those requiring audio information were conceivably less satisfied as a group in comparison to the overall sample.

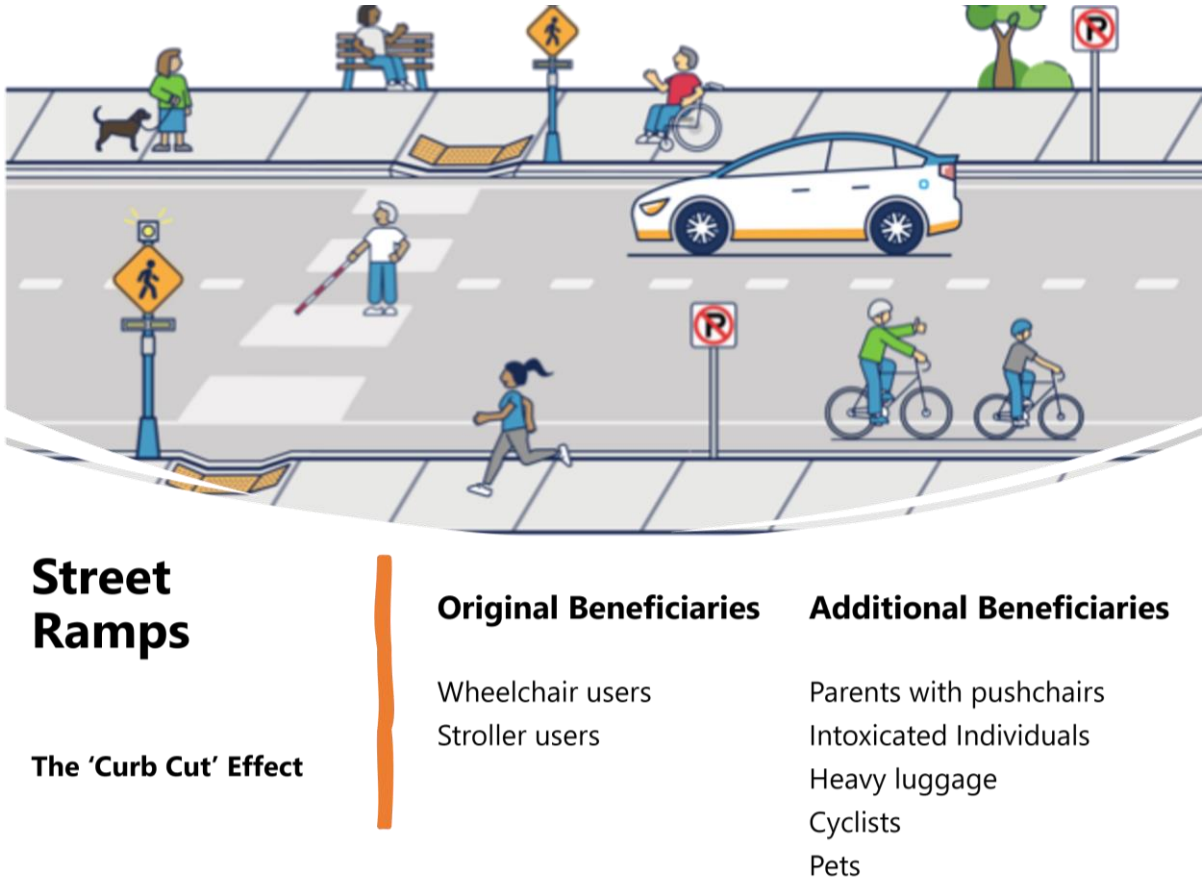


Figure 2 - The 'Curb Cut' Effect. Source: (Washington County, OR, n.d.)

Accessible design typically has a positive 'scaling effect.' Figure 2 shows the 'curb cut' effect, referring to how ramps installed on street pavements - an example of an access enhancement designed with a focus group in mind, have benefitted more users than initially intended, with new use cases arising from such changes. The scalability of accessible design increases the likelihood that an extensive range of users will benefit from a solution and further amplifies the benefits a proof-of-concept system such as this one could have.

2.1.5 Challenges in Accessible Development

One of the fundamental problems within accessible design is the trade-off between user scope and project complexity. In an ideal world, accessible design would meet its objectives for every use case. However, without unlimited resources, it is impossible to create a system that works perfectly for all. In an attempt to benefit as many people as possible within the project timeframe, refinement of the user scope must be undertaken.

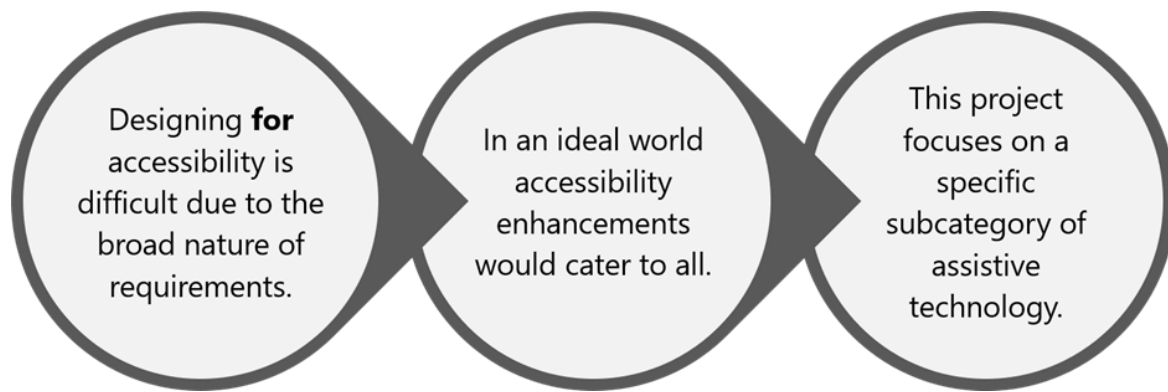


Figure 3 - Designing for Accessibility – From 'Task 4 – Presentations'

The nature of an accessibility-focused project is such that changes to the user requirements of the project are highly influential on the target user base. While in other areas of research, these requirements can be adjusted quite simply, for our focus group, should certain specifications not be met to an appropriate standard, the project will not be of benefit. Put simply, a user may not be able to actually use the system if certain components do not meet their specific needs. These requirements are a 'must-have' for the functionality of this project.

The use case diagram shown in Figure 4 is an example of how a component that is not responsible for accessibility (i.e. the detection model itself) must be 'wrapped' in a component that allows the output to be interpreted by our user. For this example, a voice-activated system is described. To provide accessible output information to our user, we must first pass it through the accessible actor (top right).

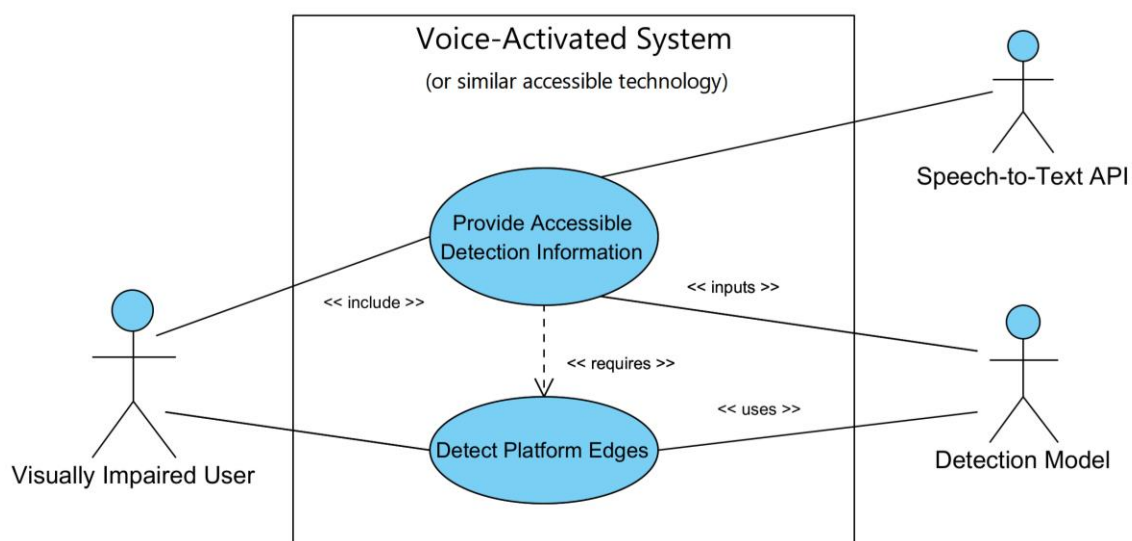


Figure 4 – Use-Case Diagram: passing through accessible actors

2.1.6 Usability Scope

Table 2 provides scope constraints made to ensure the system is compliant with ethical obligations and time limitations. It is important to note that these are not mitigations, and the project makes no attempt to try and circumvent these ‘hard limits’ (with the exception of the last limitation, which is less significant).

Limitation	Reason	Significance
The system is not intended to benefit individuals with full blindness . The focus will be on individuals with limited sight.	Attempting to design for individuals with full blindness significantly increases the complexity of the project. This would involve extensive research into integration with existing assistive technology and limit the visual aspects of the user interface (which are already quite limited). It was decided that this was not the best way to proceed when the project timeframe and constraints are considered. It is still possible that individuals with no vision can benefit from the application, just that the system is not designed with these specific considerations in mind.	4/5 – Moderately Significant
The testing stage of the system will not make use of participants with ‘uncorrected visual impairment’ i.e. the system will not be evaluated on visually impaired participants. Participant testing will also not take place on any live railways.	Ethical analysis conducted during this project has determined that there is a significant safety risk in testing with visually impaired participants, particularly within situations where hazards are present. (This is expanded on in Chapter 6 - Testing). Whilst this is a constraint on the value of testing feedback, the safety of participants is the most crucial factor. It is important to note that the research was not conducted by individuals with specific training in accessible safety.	5/5 – Very Significant
The system will not use a typical usability evaluation , with more emphasis placed instead on heuristic self-evaluation against usability frameworks. Qualitative data will, however, be collected from	As above, it has been deemed unnecessary within this project to provide a usability evaluation, since the feedback provided from an evaluation will not be of benefit unless performed with the aid of individuals directly	4/5 – Moderately Significant

Limitation	Reason	Significance
participants during the user testing.	impacted by the problem statement i.e. visually impaired persons.	
The system may have to be scaled down based on the computational demands of AI based predictive model inference.	In summary, model inference is GPU-intensive, and may perform poorly on CPU architecture, such as what is typically found in mobile phones. Mitigations and enhancements may be considered to circumvent this. (See Chapter 6).	2/5 – Mildly Significant

Table 2 - Limited scope definition based on non-functional constraints

2.1.7 Self-Evaluations

Section 2.1 discusses the importance of background research within design and development. A key component is accessible design frameworks presented by institutions to assist in the formation of projects such as this one. In lieu of a usability evaluation, an extensive self-evaluation against Nielsen's 10 Usability Heuristics will be used.

2.1.8 Scaling for New Use Cases

For this project to be successful, the best outcome is the creation of a useful tool with real capacity to improve the livelihoods of target users. Even if success is not achieved in this regard, the system will undoubtedly strive to provide a proof-of-concept for the use of the involved technologies within this field. Scalability is a point of interest for future research analysis, such as broadening the scope to accommodate a larger range of disabilities or enhancing the technologies with specialised hardware or integration with existing systems. Specifics on potential scaling capabilities are discussed in Chapter 7 - Evaluation.

2.2 Working with Images

2.2.1 Thresholding

Thresholding is a form of filtering used in digital image processing to reduce unwanted information from an image. This involves the separation of an image into different regions based on individual pixel values (known as 'segmentation').

One specific type of thresholding is binary thresholding, involving conversion of an image to greyscale before using a cut-off value to assign pixels into two segments; either black or white. The pseudocode for binary thresholding is shown in Figure 5.

```
# Assumptions
# threshold_value of 255 is a black pixel
# threshold_value of 000 is a white pixel
# Values in between are shades of grey

threshold_value = 128
# this can be changed to set B/W ratio

for each pixel in image:
    if pixel.value <= threshold_value:
        pixel.value = 255
    if pixel.value > threshold_value:
        pixel.value = 0
```

Figure 5 - Thresholding pseudocode



Figure 6 - Binary thresholding with a threshold value of 127

This allows for the creation of binary images; where each pixel is assigned either a black or white value based on the input and the threshold value.

2.2.2 Otsu's Thresholding

Otsu's thresholding is similar to binary thresholding, with one major difference. While binary thresholding takes a set value and applies the algorithm accordingly, Otsu's thresholding performs analysis on pixel intensity data to ensure important parts of the image are highlighted, giving us an image-dependent threshold. Within object detection algorithms, Otsu's thresholding may prove to be more useful than binary thresholding as it may highlight subtle differences e.g. a railway platform that binary thresholding would have eliminated.

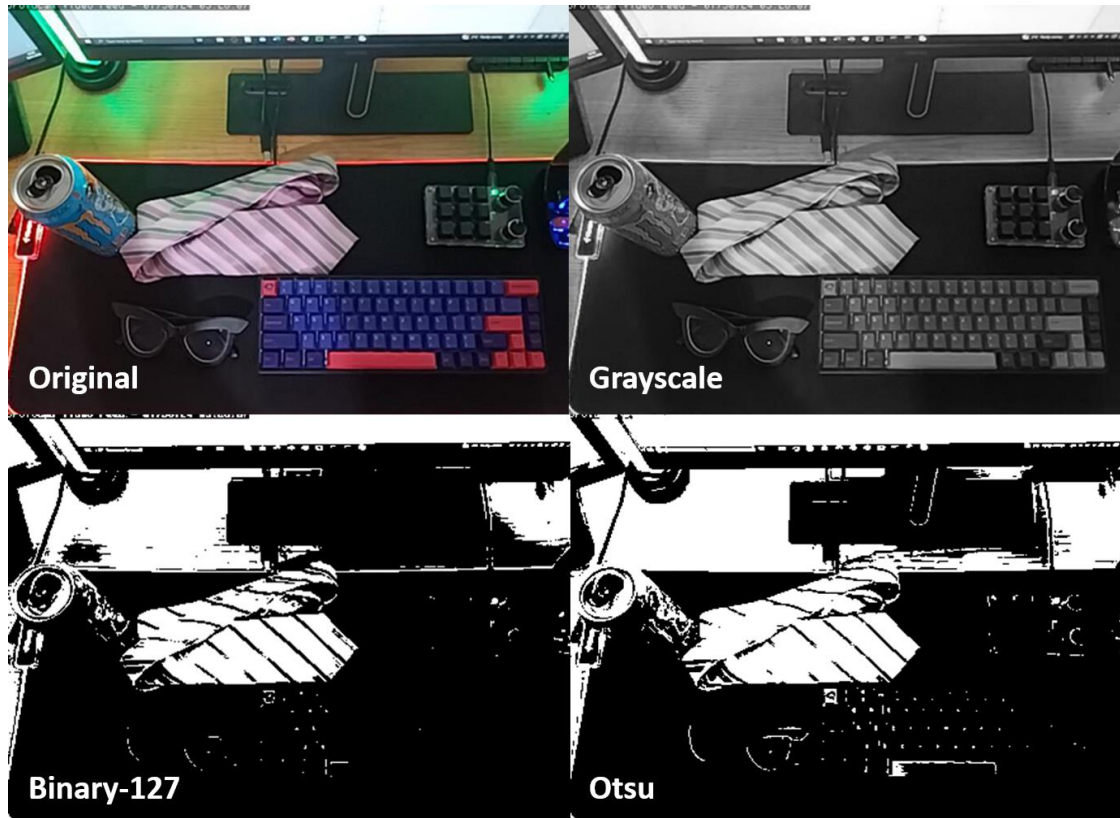


Figure 7 - Binary and Otsu Comparison

Figure 7 shows a comparison between binary and Otsu thresholding. Although the outputs are similar, subtle differences can be seen in the Otsu image – the finger skateboard on the desk remains visible, and some of the keyboard buttons are more prominent.

While applying Otsu to a camera feed is beneficial for model inference, it is more computationally expensive per-frame to analyse, which could draw CPU usage leading to less stable performance. This is an important consideration when both speed and precision are of importance.

Additional examples of filtering with adaptive media is documented in the Appendix at [add-ref].

2.3 Working with Models

Modelling is a computational term that helps to explain problems or events that occur in the real world through simulated representations. Within the context of artificial intelligence-based systems, predictive modelling is used to forecast unknown outputs when training is conducted on a set of known inputs. This has many benefits over human interpretation, such as greater efficiency to detect patterns and trends, especially when substantial amounts of data are analysed.

Many different types of models exist for different data including images, audio, and text-based models. Individual modelling frameworks often being highly specialised for specific data types. For the requirements of this project, a model that adequately detects and shows platform edges at train stations is needed.

Various modelling techniques and frameworks were researched and documented during the design phase, in order to decide which framework is best suited to the task at hand. The search was narrowed to the choice of either object detection or instance segmentation. Both techniques, including their pros and cons, are discussed in depth below, along with justifications for the final choice of model used.

2.3.1 Computer Vision

In order to understand modelling, it is first necessary to define computer vision. Computer vision is a field of computer science used heavily in artificial intelligence involving the interpretation of images or videos via computerised techniques in order to make sense of the real world. This is often done through analysis algorithms, with a primary example being the classification of objects into various categories. Figure 8 shows examples of outputs for some of these techniques.

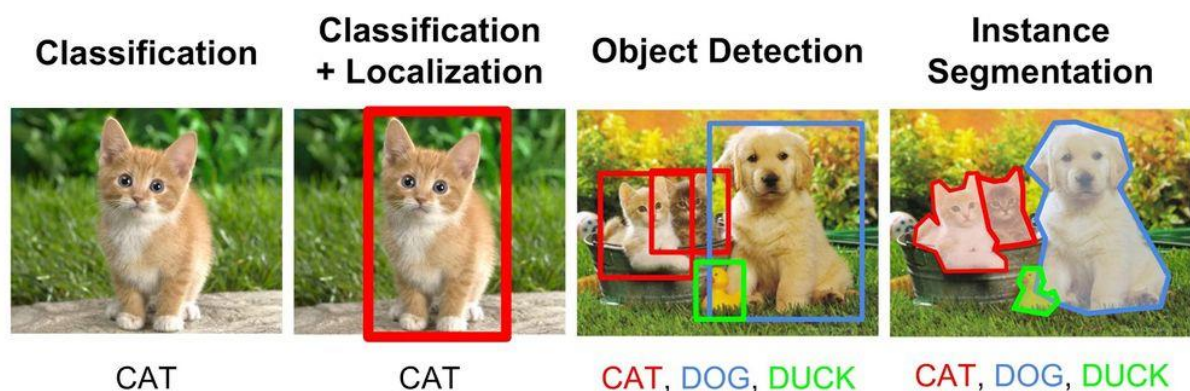


Figure 8 – Types of computer vision modelling Source: NanoNets

2.3.2 Object Detection

Detection is a fundamental computer vision technique for identifying and locating objects, with a focus on emphasizing distinct locations and specific instances within a larger image. It does not provide fine-grained information about object boundaries.

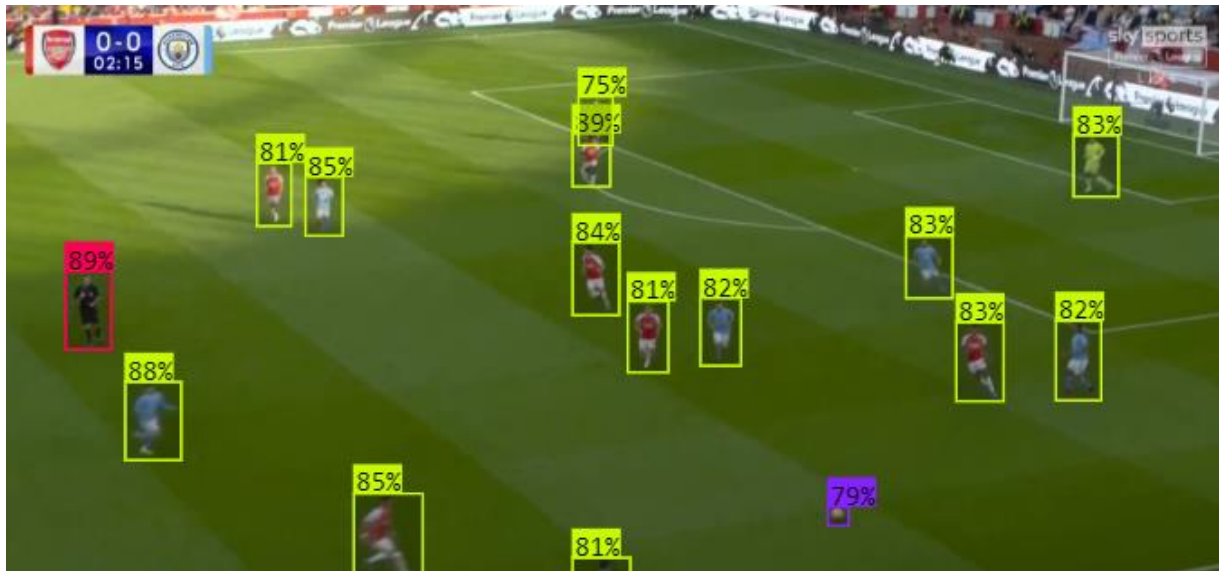


Figure 9 - Object Detection on a football field. Source: Roboflow

Figure 9 shows an example of an object detection model designed to track movement data on a football field, by inferring the model on each frame. The training data for a model such as this one will make use of human annotation, achieved by manually drawing bounding boxes on training images. Detection models can incorporate multiple classes, as shown by the varying colours in the images. Once annotations are complete, the model can be trained according to specific parameters in order to learn and detect patterns.

```
01 model = our_pretrained_model
02 while camera is active:
03     frame = grab_most_recent_frame()
04     results = object_detection(frame)
05
06 function object_detection(inputFrame):
07     detections = model.predict(inputFrame)
08     inputArray = inputFrame.convertToNDimensionalArray()
09     for detection in detections:
10
11         draw_rectangle(
12             [detection.x1 , detection.y1],
13             [detection.x2 , detection.y2],
```



```

13     )
14     return detections

15
16     function convertToNDimensionalArray(inputImage) :
17         # This function will convert the image to a
18         # format interpreted by e.g. OpenCV
19         # This is usually a pre-defined NumPy function

```

Figure 10 - Object Detection Output Pseudocode

Figure 10 shows partial pseudocode for displaying model inference output on an object-detection based system. This involves ensuring the returned data is converted into a format that can be read by computer vision software.

2.3.3 Instance and Semantic Segmentation

Segmentation is a computer vision process involving assigning semantic labels to pixels in an image, rather than at an object level. Unlike detection, which emphasises points of interest, segmentation attempts to divide the entire image into meaningful regions for a more precise analysis. Segmentation can, therefore, provide a better understanding of an image and its context.



Figure 11 - Semantic Segmentation (left) vs Instance Segmentation (right)
Source: Roboflow

Segmentation branches further into two subcategories: semantic segmentation; and instance segmentation. Figure 11 shows the differences between these techniques; although semantic segmentation provides a region-based output to distinguish different classes of objects, instance

segmentation aims to further enhance the results by providing a classified output for each individual object.

Within the use case presented in this project, instance segmentation has multiple benefits over semantic segmentation, with one of these being a known boost in performance. It also allows for counting and tracking of objects in a way that semantic techniques do not. Instance segmentation does however fall back in its ability to manage overlapping or occluding objects due to individualisation.

3 Approach

This chapter will introduce the choice of methodology undertaken for the implementation phase, with relevant adaptations made to best suit this project. A synopsis of the approach taken for the project development is also provided.

3.1 Project Approach

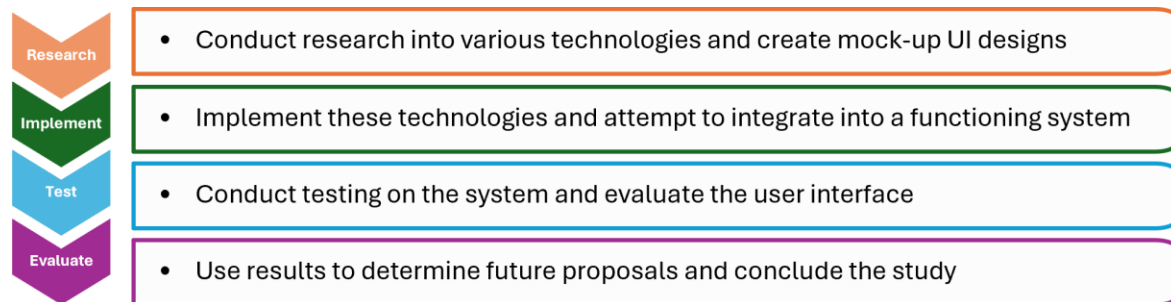


Figure 12 - Project Flow

Figure 12 depicts a generalised view of project stages. The initial ‘research’ stage involved the formation of a system architecture design best structured to adhere to project objectives, with the ‘evaluate’ stage being the creation of a fully functional system that conducts artificial model inference on a smartphone camera input to provide accessible feedback. The ‘implement’ stage will include explanations provided into various technical aspects and their utilisation to optimise the end result. Limitations will also be investigated in this stage to maximise time efficiency.

Building on from this, testing will be conducted into the best ways to incorporate separately designed components into a functional end product. Performance issues and optimisation both with the model and data transfer pipelines will be evaluated alongside research into available software tools to undertake the technical tasks, with reporting done on problems encountered and challenges faced.

Following this, both participant-based testing and functional testing will be carried out in order to provide a full evaluation of results and determine the next steps. Participant testing will involve users connecting to the software page to measure model effectiveness across different cameras and devices. Functional testing will also be done by analysing the training results and re-running model training with different hyperparameter values. The system UI will also be self-evaluated against usability frameworks in this stage.

3.2 Project Methodology

This project will use a linear approach for user requirements and project analysis, as well as formulating design concepts in advance of the system development. The project will then use an iterative approach for implementation and testing for continuous refinement.

3.2.1 Choice of Methodology

Following evaluation into the project criteria, it has been decided to follow a modified version of the **Agile-Waterfall Hybrid Method**, initially described by Bergmann and Hamilton. Diagrams for both agile and waterfall processes are provided in full in the Appendix at Section C.1.

The hybrid method involves combining traditional waterfall-based development and the iterative nature of agile frameworks in a way that is best suited individual projects. For this project, the ordering of various stages of development has been diagrammed in Figure 13 below.

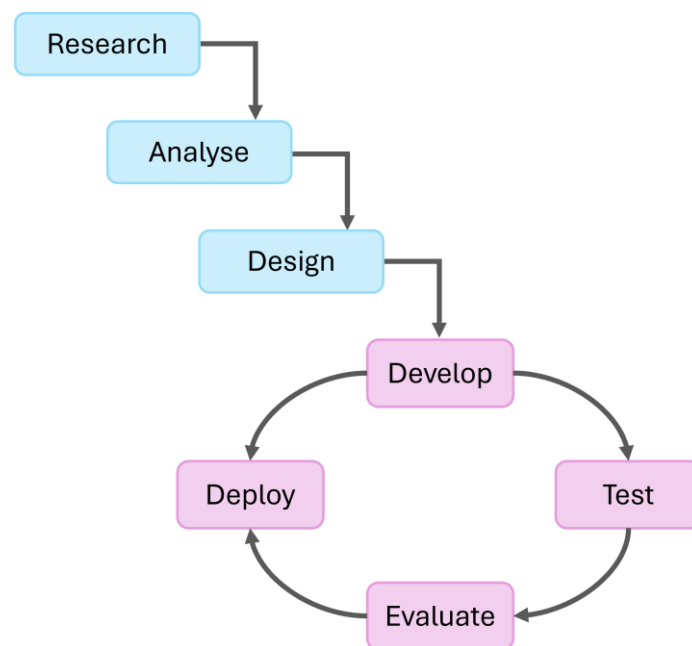


Figure 13 - Modified Agile-Waterfall Hybrid Diagram

3.2.2 Methodology Justification and Analysis

Research conducted into hybrid approaches (Taller, 2021) has suggested that there are multiple criteria that a project can satisfy to make success of the methodology. Taller suggests that should a project 1) consist of equally important hardware and software components, and 2) consist of both backend and frontend technology; the hybrid model will be effective in its development.

Taller states that to “plan with Waterfall, execute with Agile” is to “speed up the whole process”. The process will allow for the shortening of design, analysis, and planning sections (these will not be done iteratively), whilst also allowing for testing on implemented software when observing for shortcomings.

Additional combinations of the hybrid methodology are also possible and could be considered, should the methodology not deem as appropriate as initially intended. Research suggests that hybrid models work best when developers have the freedom to determine the best phase design schematics for a team or individual project (Lucidchart.com, n.d.), with some examples being the use of agile approaches at a “project level”, but waterfall approaches at “organisation level”.

3.2.3 Combining Waterfall and Agile Principles

Waterfall concepts are beneficial when a project has lots of “dependent tasks”, (Lucidchart.com, n.d.). For this project, the objectives require rigid accessibility-based requirements that are not subject to alteration; the most suitable approach is to define these requirements at the start, and keep them. On the other hand, agile concepts are beneficial when a project requires lots of continual refinement to achieve specific goals via iterative testing. An agile approach in this area will need to be undertaken to minimise hardware latency or re-train the detection model for better results. Component design for this project also necessitates an agile approach; one example being the initial use of Flask for the web-based system (see Section 7.2.2 – Problems Encountered) before the realisation that this would not allow for camera transmission, requiring a reimplementations of some of the already-integrated codebases.

In summary, although the use of a hybrid approach may seem unusual – the specialised requirements of this project, in addition to the criteria shown above, have deemed that it is the most suitable development methodology for the construction of this system.

3.3 Project Ethics

This project required ethical approval in order for research to be conducted. The approval protocol states that the project must not make use of any participants with uncorrected visual impairment, and all user-based testing must take place on university premises, which has been risk assessed. All participant testing was taken place within university grounds, whilst functional testing of the system (such as training better models) took place both internally on-campus and externally.

Ethics documentation for the project is attached in Appendix B : Ethics Confirmation.

4 Design

This chapter will provide an in-depth analysis of the first stages of planning the problem solution, with a focus on software choices, user interface blueprints, and accessibility.

4.1 System Architecture

The system and its components follow a 4-step process shown in Figure 14. This involves training an artificial model to detect instances of platform edges for each frame provided and processing the output into a format that is accessible for the target user. Individual components should then be combined into a system that works for the user.

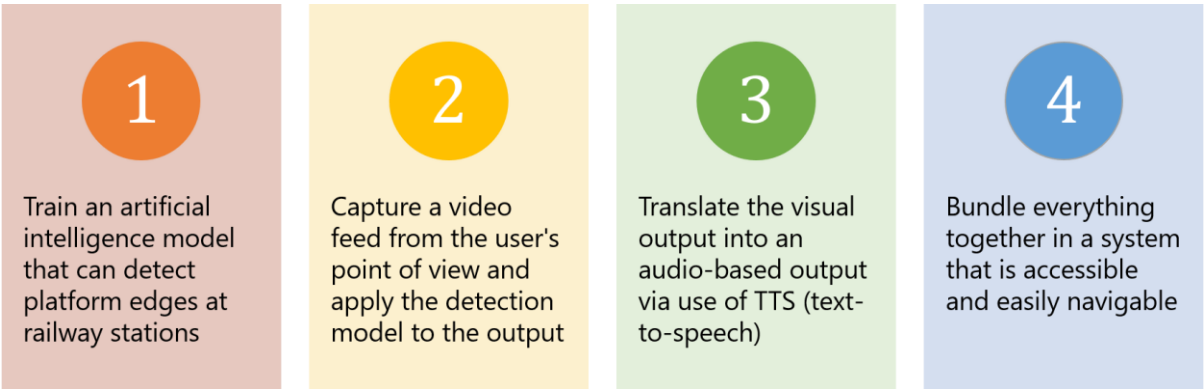


Figure 14 - 4-step process adapted from ‘Task 4 – Presentation’ (Kazak, 2024)

Choosing relevant software for each of the four steps involved detailed research into existing technologies, including frameworks for predictive AI, and image filtering techniques. Research was done into android camera technology, Python web interfacing, and both text-to speech and speech-to-text implementations, and how to combine these technologies. Component integrations have been mapped out, with interactions between integrated systems diagrammed in preparation for the implementation.

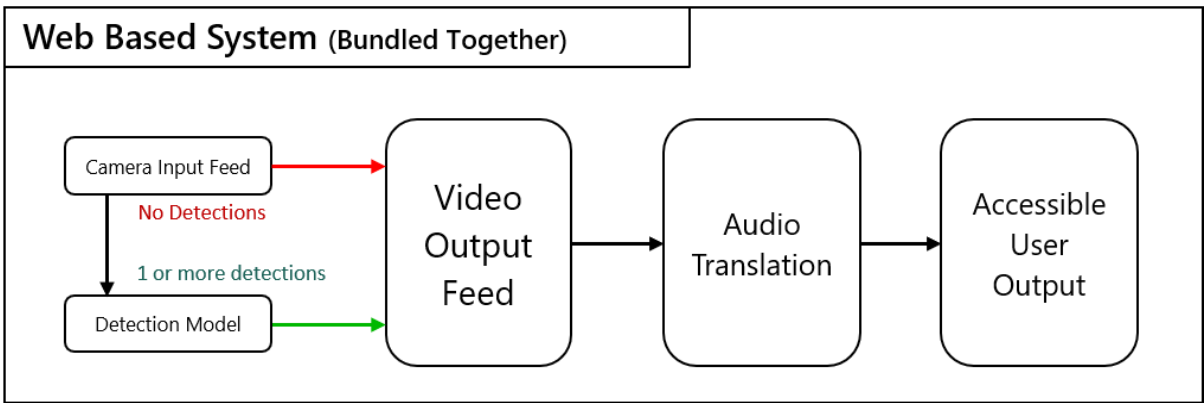


Figure 15 - Component Integrations

4.2 User Interface Design

The user interface designs in this project are uniquely complex and it is important to visualise the system design to ensure ease of use. Figure 16 provides a high-level abstraction of the proposed system design.

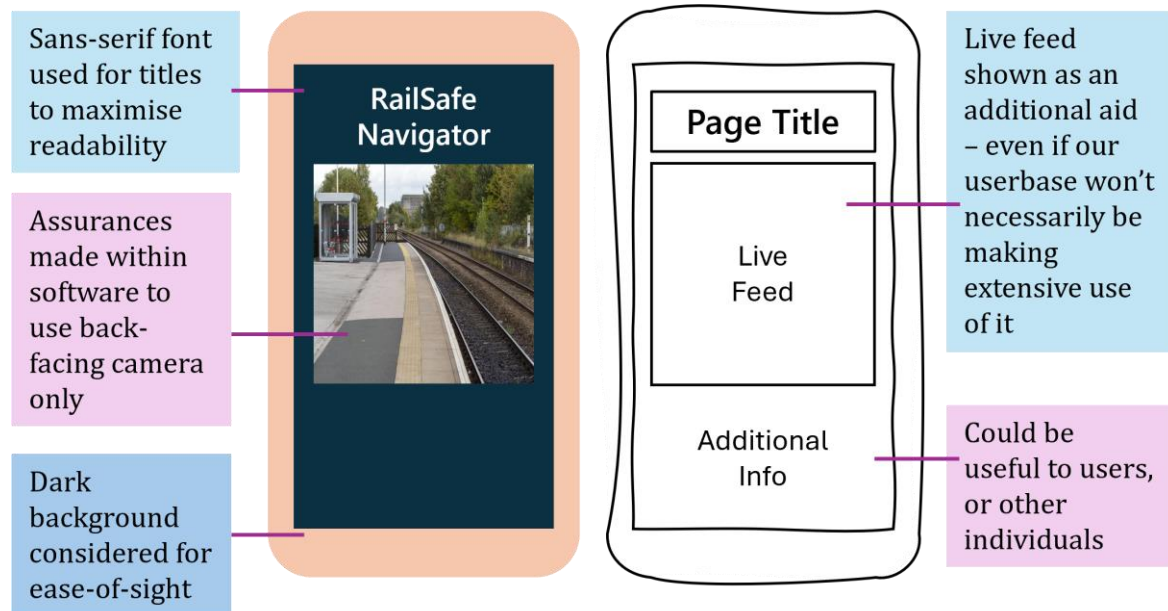


Figure 16 - High-level mock-up of the system design

4.2.1 Designing Visual Aspects

A decision has been made to incorporate the live camera feed and detection outcomes visually within the user interface alongside the audio feedback. despite the project not being designed to use visuals. This decision stems from recognition that sole reliance on the audio-based system may pose difficulties to a new user, particularly with comprehension of how the system functions.

In addition to this, the program will still need to run store, and transmit frame results as an image matrix for model inference to work, regardless of whether the frame is displayed visually or not. Exclusion of the live feed therefore offers no discernible benefit, minus a negligible amount of bandwidth or graphical process savings. In the event of poor performance, deactivating the live feed and falling back to only using audio is a consideration that can be made.

4.2.2 Managing Audio Integration

Much like the visual aspects, which undergo continuous updates as new frames are pulled, the audio system needs to function in a way that benefits the user base without compromising the safety of the system.

One potential method for model outcome feedback audibly to a visually impaired userbase could be the utilisation of stereo audio channels in order to provide outcomes that match the side of the image with the most significant detection.

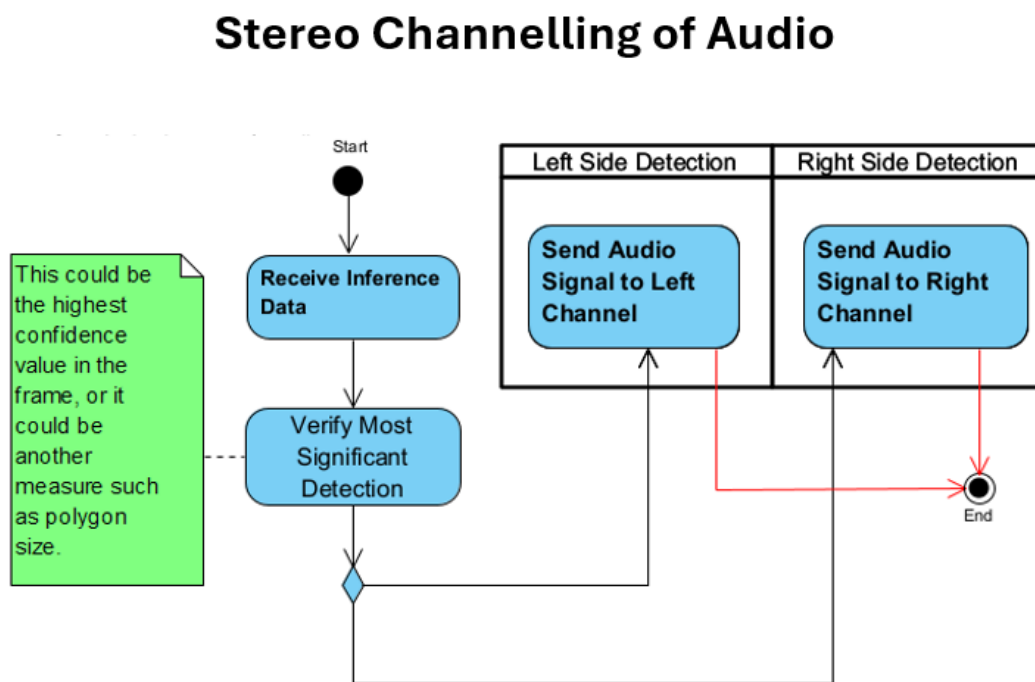


Figure 17 - Per-frame activity diagram for audio system

Figure shows a simplified per-frame activity diagram, illustrating how left and right channels can be used to relay detection information in this way. Although this is quite simple in theory, in practice it could pose some safety risks which will need to be considered. Side accuracy rates may be low if an individual is not standing near a platform edge; there is also the possibility that an individual is standing quite close, causing a right-facing edge to show up on the left side of their camera feed. This could have a significant impact on the potential for designing a safe system. Should this not be implemented rigorously enough during initial development, improvement in this area is a strong candidate for future work.

4.2.3 Decision-Making and Optimisation

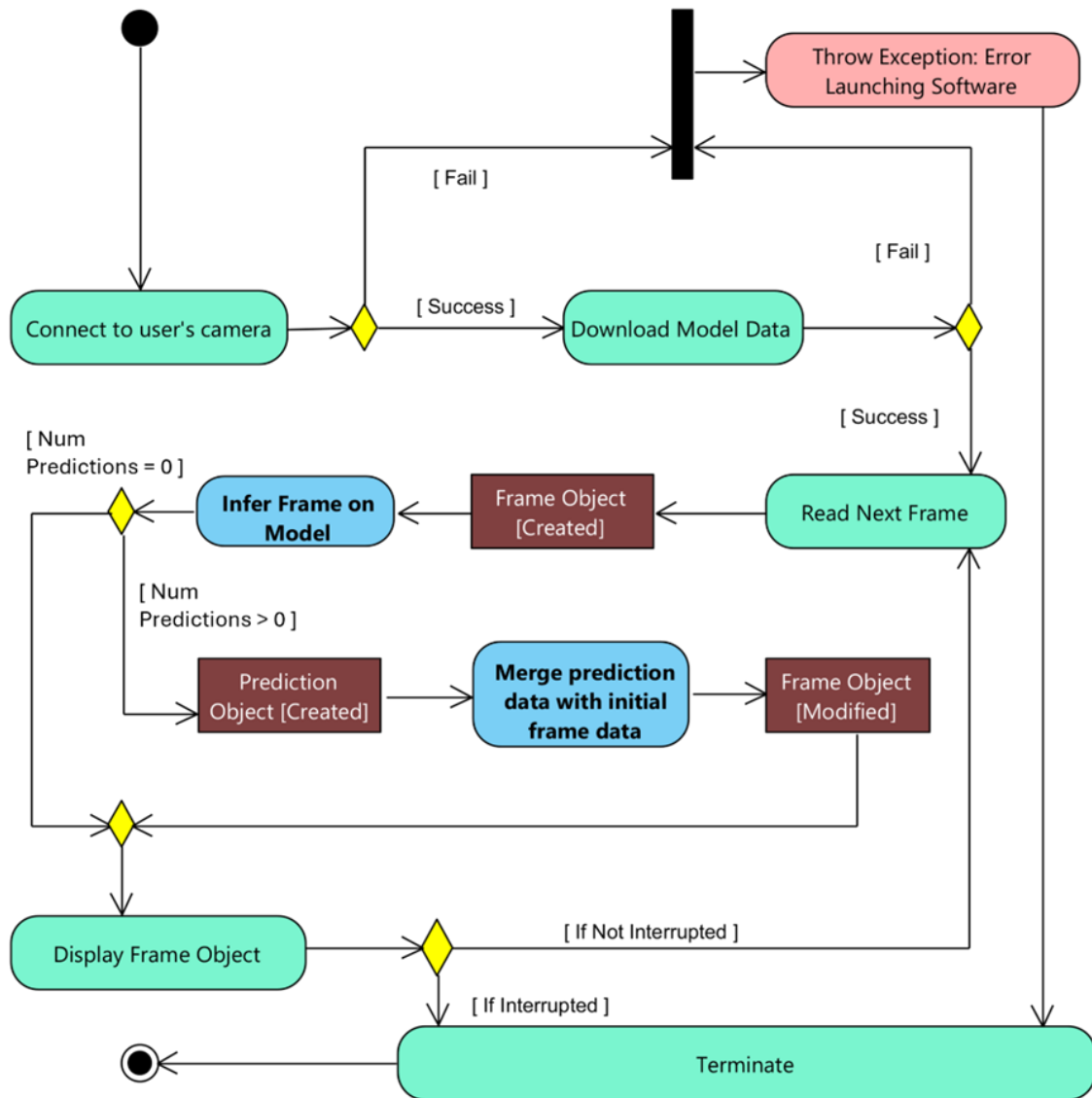


Figure 18 - Activity diagram for frame inference

Figure 18 shows the activity diagram for the frame inference setup. Extensive research was done into the most effective camera feed retrieval methods, emphasising external devices and successful inference outputs. The diagram illustrates the schematic layout for the situational logic the software must adhere to for decision-making. The logical flow will take different paths depending on the interruption state and number of predictions.

4.3 Building Datasets

Data must first be collected in order to construct a model. For this project, building the training data involved scraping hundreds of images of railway platforms. Although possible to do manually, it is far easier to automate the process through the use of pre-built scripts that exist for this purpose. Appendix C.5 discusses the image scraping technique used throughout development.

4.3.1 Roboflow: Training Data

Roboflow is an online platform developed to streamline model training, deployment and inference for computer vision based tasks. It offers broad functionality to optimise the image training process, including built-in tools for annotation visualisation and export, and dataset organisation. It has proven invaluable for saving time, organising various aspects of the project, and providing simplicity in deployment of the model.

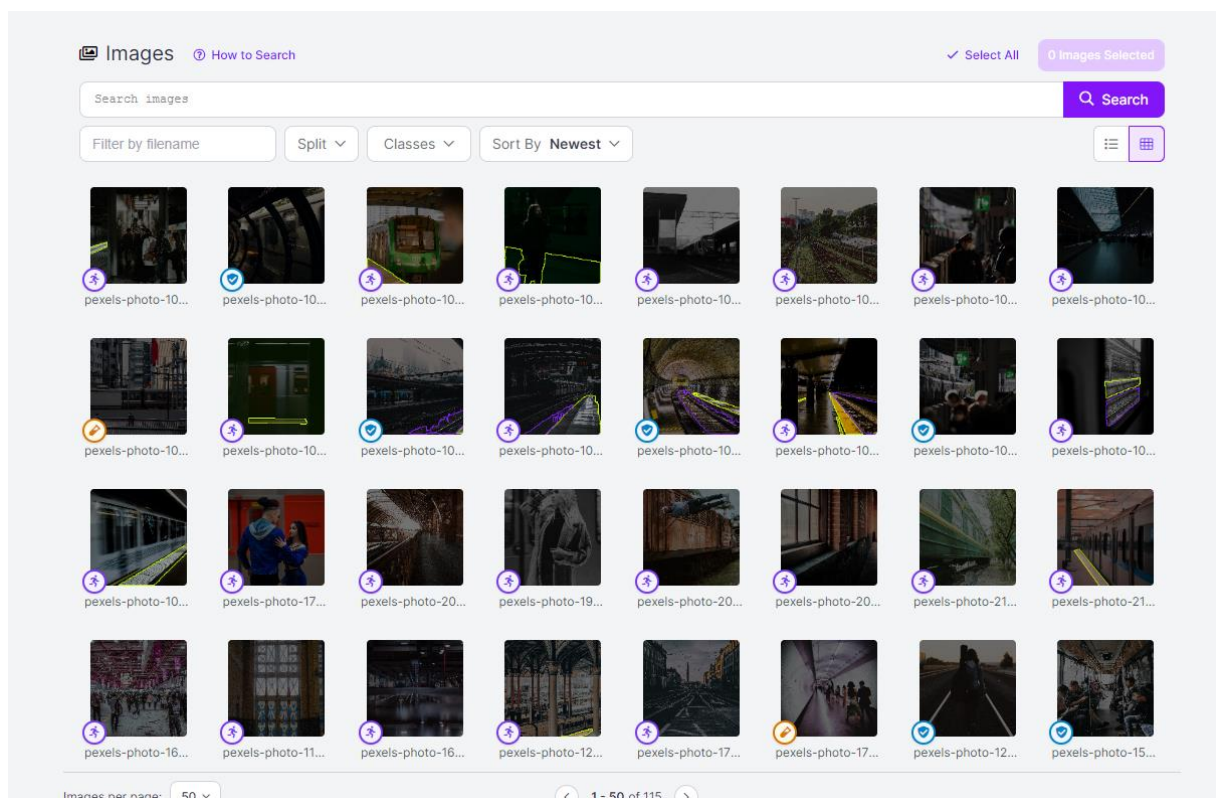


Figure 19 - Project dataset within Roboflow, split into train/test/validate categories
Source: Roboflow

Roboflow can export annotation data and utilise its API to download datasets to a training environment, and information such as model specification (object detection or instance segmentation). Dataset customisation is also available, allowing for specific 'versions' of a dataset to be trained with their own model. Image augmentation options are also available (e.g.

the option to apply a brightness filter before training), and capabilities exist to split the train/test/validate ratio and create variants. Roboflow supports multiple classes when annotating, which has proven to be useful in later iterations.

In addition to this, previously trained models provide information about the success rates, which is useful for comparison. Example web apps, browser-run inference and extensive documentation are also provided.

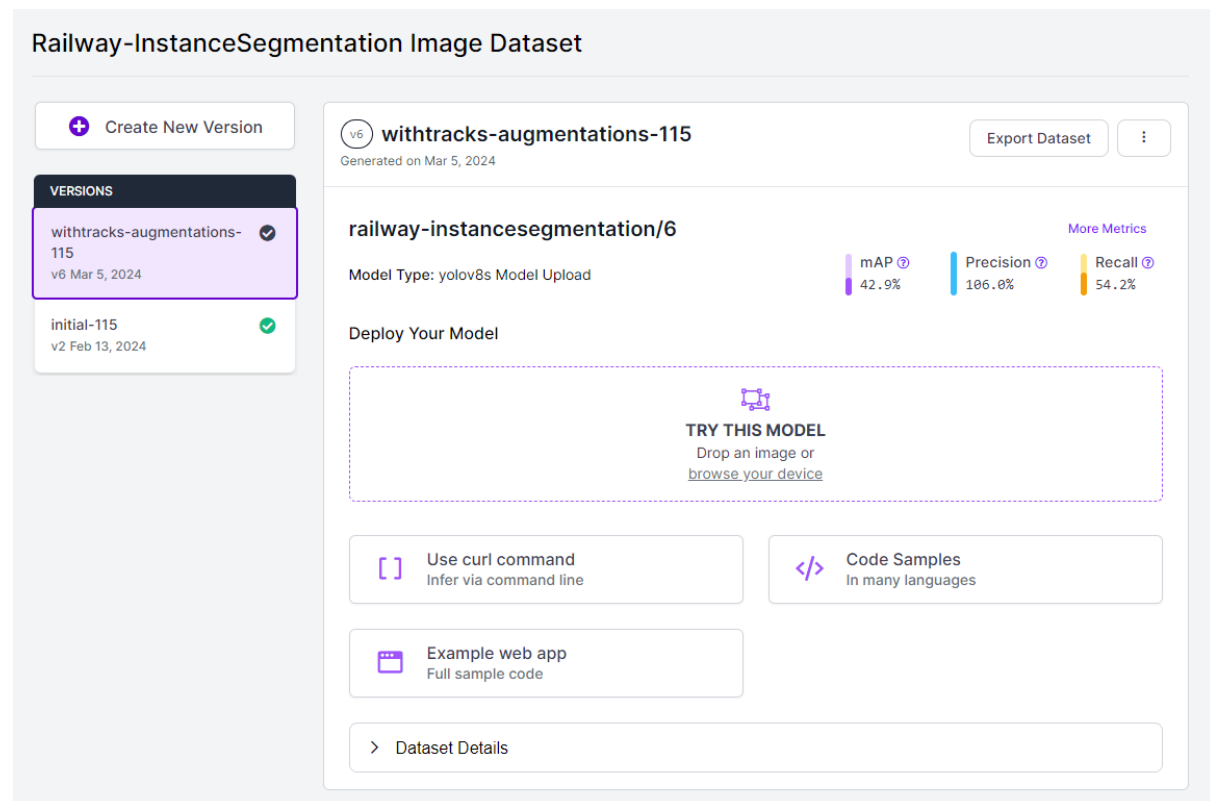


Figure 20 - Roboflow Deployment Page; shows iterative measures. Source: Roboflow

The web-based interface is complemented by the Python 'inference' package, which is used to cache the trained model to a local device and infer an image.

The advanced capabilities of the Roboflow system have proved incredibly beneficial for the iterative nature of this project's methodology, allowing reviews to be conducted at each stage, with results compared and successes/failures easily tracked.

4.3.2 Roboflow: Annotations

Annotating, in simple terms, is when bounding boxes are manually drawn on images. This provides an input set for the model to 'learn' from, so it can adapt to recognise similar patterns when an unseen image is provided, such as from a smartphone camera.

Roboflow provides support for export to a range of models and formats. Support for the YOLOv8 models is natively included, as well as export to COCO format. This has allowed for both object detection and instance segmentation tests to be carried out without the time-consuming process of building different datasets.

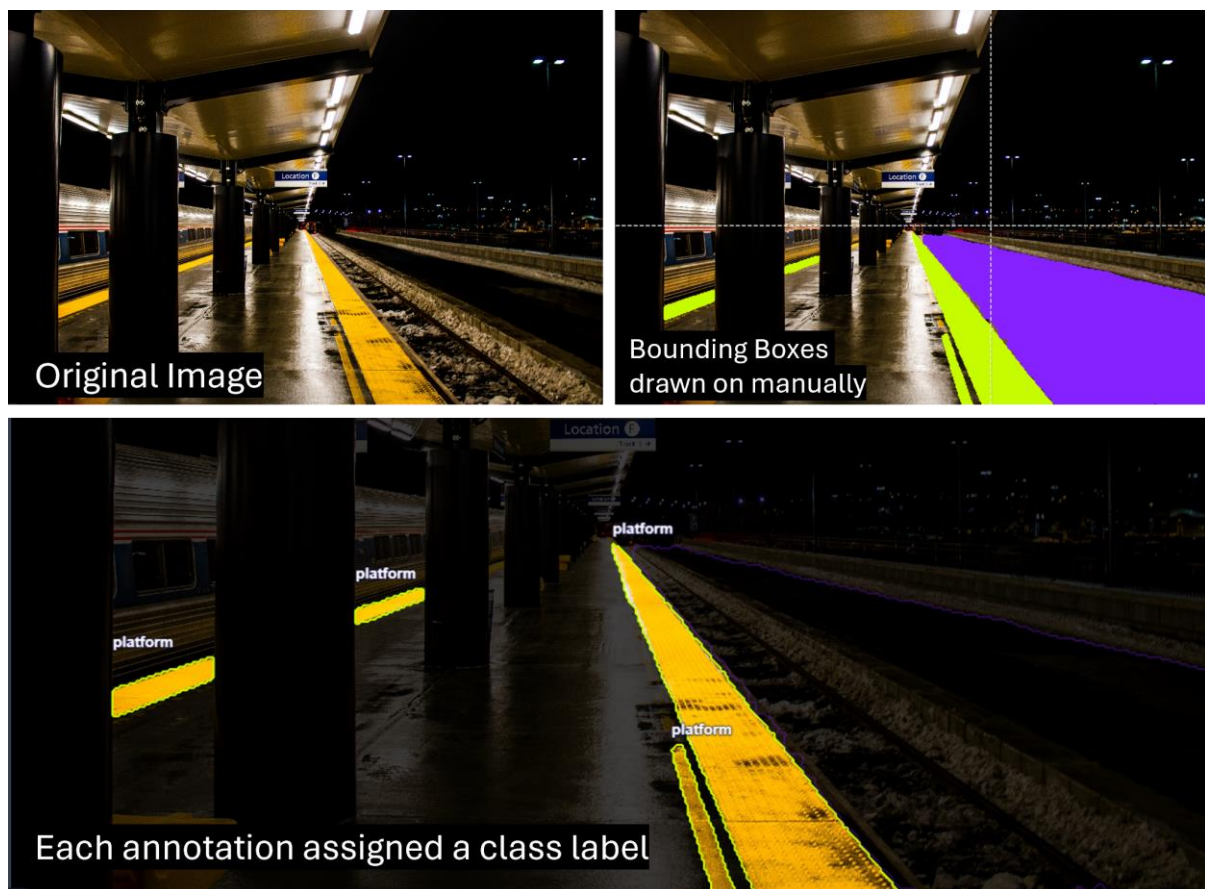


Figure 21 - Annotating within Roboflow

As shown in Figure 21, the annotation process involved drawing boxes on points of interest and then assigning these to a class label, allowing for the model to classify specific regions. This was repeated for all the images in the dataset.

The annotation features of Roboflow allow the user to mark an image as 'null' if there is no relevant data present in an image. This is also useful; a small number of irrelevant images can provide an additional level of depth to a model to slightly mitigate the risk of false positives. Various images were annotated in different lighting conditions to ensure the model is able to account for a range of lighting scenarios when inference is conducted.

5 Implementation

5.1 Chapter Overview

This chapter builds on the principles established in Chapter 4 to document the transition made towards the development of a fully functional system. This includes a discussion of tool and software choices with justifications provided, and explanations of code given, as well as evidence that the project methodology is being followed.

Problems encountered in this chapter range from the more difficult challenges related to performance, as well as the trivial errors that have been time-significant to solve, and the lessons learned from these mistakes. This will provide a foundation for the full analysis of results discussed in Chapter 7 – Evaluation.

The term ‘inference’ is used heavily in this chapter and following sections. This is defined properly in Appendix C.7.1.

5.1.1 System Diagram and Discussion

Figure 22 is an expanded system diagram detailing the process described earlier in Figure 14. Each of the 7 components labelled A-G represent a distinct part of the implementation process. Table 4 shows the software considered and implemented for each component (components are not lettered in order).

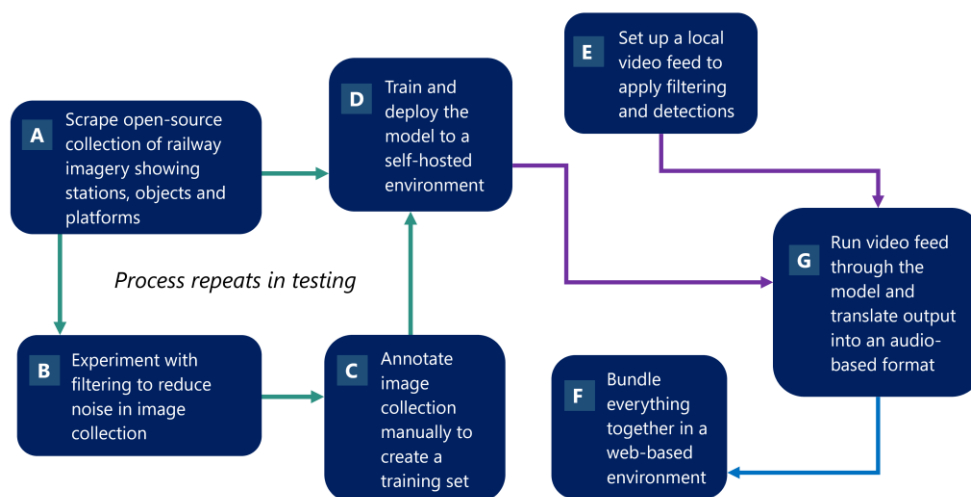


Figure 22 - Expanded System Diagram (adapted from ‘Task 4 – Presentation’)

Component	Software / Tools Considered	Notes
A	Pexels (Website)	A simple batch script was used to scrape images from these sources.
B	OpenCV Python	
C	Roboflow Annotate	
D	Roboflow Deploy; Google Colab	(later done on a local 3070Ti GPU using Jupyter Notebook for code and CUDA support)
E / F	DroidCam, Flask for Python, AIOHTTP + AIORTC for Python,	Initial phases used DroidCam for an IP camera stream. Later on, this was replaced with AIORTC. Flask was tested not used due to compatibility issues (detailed later in this chapter)
G	AIOHTTP; Audacity	Audacity used to edit MP3 files for stereo channelling.

Table 3 - Software choices and explanation

5.2 Model Training

The model was initially trained within a Google Colab environment on an NVIDIA Tesla T4; a specialised single-slot GPU optimised for deep learning. The skeleton code required is provided by Roboflow (Roboflow, n.d.), however this required extensive modification to work as intended.

5.2.1 Training: Google Colab and Jupyter

Upon execution of the script below, the model will begin training, feeding input data through the YOLOv8 segmentation layers (see Appendix C.4) and resizing the scraped input data to match the sizing parameter.

```
!pip install ultralytics==8.0.196
!pip install Roboflow

import ultralytics
from ultralytics import YOLO
from roboflow import Roboflow

rf = Roboflow(api_key="...")
project = rf.workspace("saen").project(
    "railway-instancesegmentation")
dataset = project.version(...).download(
    "yolov5")

!yolo task=segment /
mode=train model=yolov8s-seg.pt data=currentSet/data.yaml /
epochs=20 imgsz=640
```

Install and import necessary Python libraries and scripts.
 # Hook into the Roboflow API to download the model data.
 # This code will download the specified dataset to the /content/datasets folder.
 # For simplicity the dataset folder is manually renamed to currentSet.
 # api_key and version are replaced with the relevant information.
 # (The YOLOv5 data format downloaded is compatible with YOLOv8)

Figure 23 - Colab-based training

Model training is a computationally expensive task and requires a GPU to be completed within an acceptable timeframe. Although technically possible to train locally on a CPU, the parallelistic nature of GPU accelerated training means that it will take exponentially longer.

For reference, an NVIDIA RTX 3070Ti has over 6000 cores and dedicated video memory for intermittent data storage. CPUs, however, typically have around 4 to 8 cores for parallel processing. Later in development, functionality was set up to train the model locally on an NVIDIA RTX 3070 Ti CUDA-enabled GPU. More details on GPU acceleration are provided later in this chapter.

5.2.2 Interpreting Model Results: Precision, Recall and F1-Score

Once training is complete, the model will analyse the data on the test set to produce some output values. Table 4 below provides some of the metrics used to evaluate the performance of a model, with values are complemented by output graphs produced by the training software.

Metric	Explanation
Confidence	Confidence score is analysed per-detection to deem how likely the current detection is to be accurate. Each detection is given a confidence value between 0 and 1.
Precision	Precision is the ratio of positive detections to all detections. It can be seen as a metric of how many detections were actual correct instances. $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$ A precision value of 1 means there were no false positives detected; whereas a value of 0 would mean all detections were false positives. (Google Developers, 2019)
Recall	Recall is the ratio of positive detections to all instances of the class, whether or not these were actually detected. It can be seen as a metric of how many actual correct instances were detected. $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$ A recall value of 1 means there were no false negatives detected; whereas a value of 0 would mean none of the correct instances were identified. (Google Developers, 2019)
F1-Score	F1-Score is defined as the harmonic mean of precision and recall (Kundu, 2022). Harmonic mean is used to encourage similarity in the values. The formula for F1 is as follows: $\frac{2(\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$ F1-Score works well for our model due to the fact it makes use of only two classes. In models with a greater number of classes, F1 may prove to be a less accurate metric of success.

Table 4 - Model performance evaluation metrics

A ‘positive’ detection is where the system has identified a platform or track, whether or not this detection is correct.

A ‘negative’ detection is where the system has determined that no platform or track exists within a region, whether or not that is correct.

5.2.3 Performance Evaluation Sample

For the purposes of the figures below, training was conducted on a dataset of 272 images (115 + augmentations). The image input size was 128x128, the batch size was 32 and the number of epochs was 10.

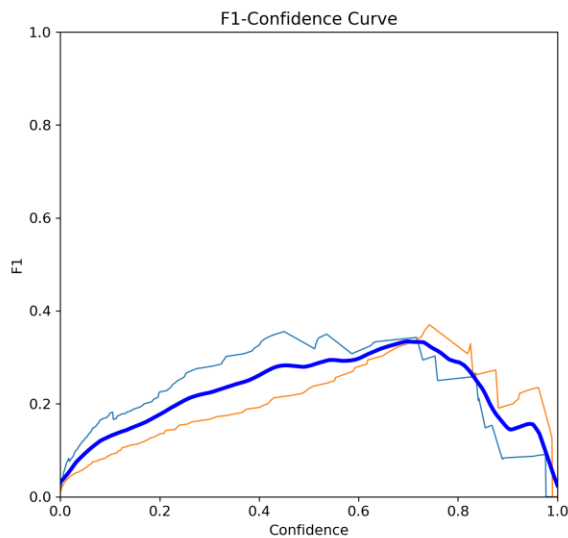


Figure 24 - Sample F1-Confidence Curve

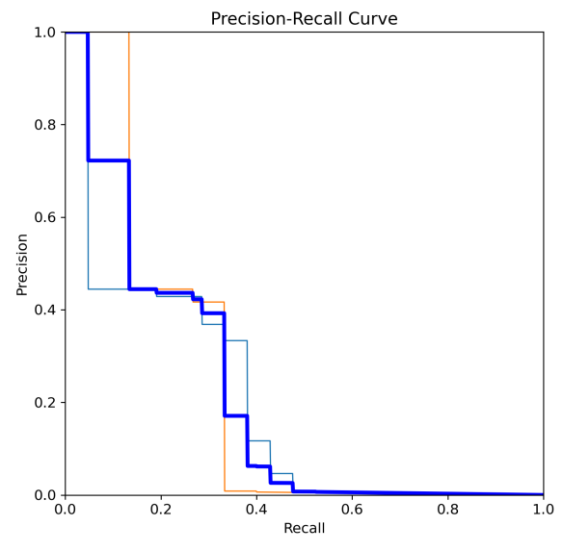


Figure 25 - Sample Precision-Recall Curve

Data shown in the figures above suggests the model is lacking in its overall performance for this run. A steep decline is noted in the F1 value as the confidence threshold approaches. This suggests the model is potentially detecting platforms and tracks as 'less confident' than it should do and that setting a high confidence threshold will lead to lots of true positives being missed. The model precision also drops significantly across higher levels of recall, suggesting that attempting to detect of a high number of positives brings about the potential for lots of false detections to appear.

These results are provided for explanatory purposes in this section. Full results are detailed in the next chapter.

5.2.4 Iterative Training

The poor model performance shown above is undeniable evidence for utilising an iterative approach (explained in Section 3.2 – Project Methodology.) This allows for continual refinement of the model with analysis done at each stage.

5.3 Camera Streaming

DroidCam is an android application that allows for the streaming of a camera feed over the local network. It was used to test the model on a camera feed prior to the introduction of a web based interface. For the purposes of this project, DroidCamX was used, which permits greater customisation of framerates and streaming options.

5.3.1 Web Cameras: DroidCam

The feed provided by a phone camera via DroidCam can be hooked into an OpenCV **VideoCapture** object. This allows us to stream a video feed as input and modify the frame data before sending back the result as an output.

```
import cv2 as opencv
feed =
opencv.VideoCapture("http://192.168.0.18:4747/video")

while True:
    ret, frame = feed.read()
    # OpenCV's filtering options can go here

    if (ret):
        opencv.imshow("Output Frame", frame)
```

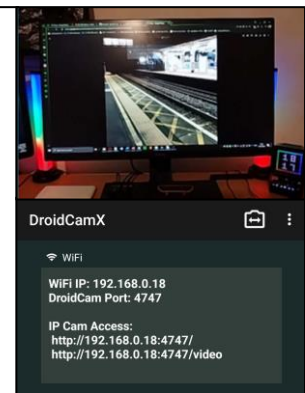


Figure 26 - Python code for hooking into a DroidCam feed

DroidCam was used in early iterations before a fully functional frontend was implemented. While useful to test the model in a localised setting, it is not scalable to multiple devices without changing the code (users would have to install the app and the code would need their device IP).

5.3.2 OpenCV: Visual Displays

OpenCV is an extensive library of open-source tools and programming functions intended for computer vision tasks. In addition to allowing display of a video stream object, it also provides functionality for image filtering and modification. OpenCV was used to provide real-time filtering on the camera feed. OpenCV can be used to modify an input feed for filtered output. (e.g. if we train our entire dataset on entirely augmented adaptive median modified images, OpenCV can be used to apply filters on the input feed).

In order to push a frame through the model, we first have to import the model data using the `inference` package in Python.

```
from inference.models.utils import get_roboflow_model
import numpy as np # Coming up later...

# The version we want to import...
model_name = "railway-instancesegmentation"
model_version = "7"

model = get_roboflow_model(
    model_id="{}/{}".format(model_name, model_version),
    api_key="["...]"
)
# code continues...
```

Figure 27 - Instantiating the model in Python

This will cache a download of the model data which will be automatically detected in the future. A limitation with Roboflow is an inability to truly achieve an ‘offline’ inference model. Although this has significant implications in a railway setting, future implementations could attempt to translate the Roboflow data into a different format that allows for this.

```
def grab_instances(img):
    result = model.infer(image=img)
    response = result[0]
.
```

Figure 28 - Model inference on a single frame

Model inference will return an “InstanceSegmentationInferenceResponse” object encapsulating detections within a list, including bounding coordinates.

From here, image and mask manipulation can be performed on the resultant object in order to construct a colour mask. These masks will also take the form of NumPy arrays, which is incredibly useful as the format similarity allows numerical overlay of the detection structure on the original frame, with different colours used for each class.

At this stage of the project, the model was only classifying platform edges (tracks were done in the next cycle). As a result, only platform detections are inferred in the following code.

```
# Create the mask array, it only has two channels
mask = np.zeros((image_height, image_width), dtype=np.uint8)

# Response object encapsulates a 'predictions' object list
for prediction in response.predictions:

    # Prediction contains X and Y coordinates
    points = prediction.points

    # Convert points to array
    pts = np.array([[point.x, point.y] for point in points], np.int32)
    pts = pts.reshape((-1, 1, 2))

    # OpenCV used to draw prediction polygon on the mask array
    cv2.fillPoly(mask, [pts], 255)
    x = pts[0][0][0]
    y = pts[0][0][1]-20
```

Figure 29 - Image & Mask Coordinate Manipulation with OpenCV

A sample result of running the above code on a single frame is shown in Figure below.



Figure 30 - Sample platform inference on a single frame. Source image: DC-Kits



The first development iteration contained the implementation techniques discussed up to this point. **Section 7.1** provides an analysis into the results found in this cycle, challenges encountered, and next steps considered.

5.4 Camera Integration

Backend functionality was a heavy focus point during the first implementation cycle due to the dependent nature of model training in developing the corresponding user interface. The second cycle placed a greater focus on frontend implementation to transition away from a conceptual system with DroidCam input and towards a client/server based scalable setup allowing for distinct devices.

5.4.1 WebRTC and AIORTC

Research was conducted into the WebRTC (Real Time Connection) protocol and associated dependents in order to achieve this goal. WebRTC is an open-source framework developed in part by Google in order to enhance the capabilities of an HTML5 based environment by providing the means to hook into local input media such as cameras and microphones (Sredojev, et al., 2015).

One of the key aspects of the WebRTC connection protocol is the utilisation of peer-to-peer connections between web browsers and an application server. This is beneficial for a system which is heavily dependent on minimising latency.

In order to achieve this, WebRTC makes use of the ICE (Interactive Connection Establishment) framework in order to maintain connections in a way that bypasses NAT and firewall constraints.

The capabilities of WebRTC make it an excellent choice for the task at hand. However, since our system up to this point is Python-based, whereas WebRTC is JavaScript-based, research was done to find a ‘wrapper’ that could link the two systems together. The AIORTC python library does exactly this; combining WebRTC with Python’s AsyncIO and AIOHTTP frameworks in order to hook into an HTML/JS frontend and pass **MediaStream** objects into Python (allowing us to effectively replace the DroidCam feed). (AIORTC, 2021)

The code used for both the Python and JavaScript components was adapted from the original sample code provided in AIORTC’s example library and combined with the original code to produce the full system.

5.4.2 Web UI Development: Streamlining Sample Code

The default interface provided in the sample code was not designed as a production-level system and required extensive adaptation to be functionally used. This included removing audio capture from the target device and removing console outputs that were initially displayed,

which proved to be a challenging task as various methods within the JavaScript code had to be adapted to function without these features.

The full annotated code is provided in the Appendix in Section 0.

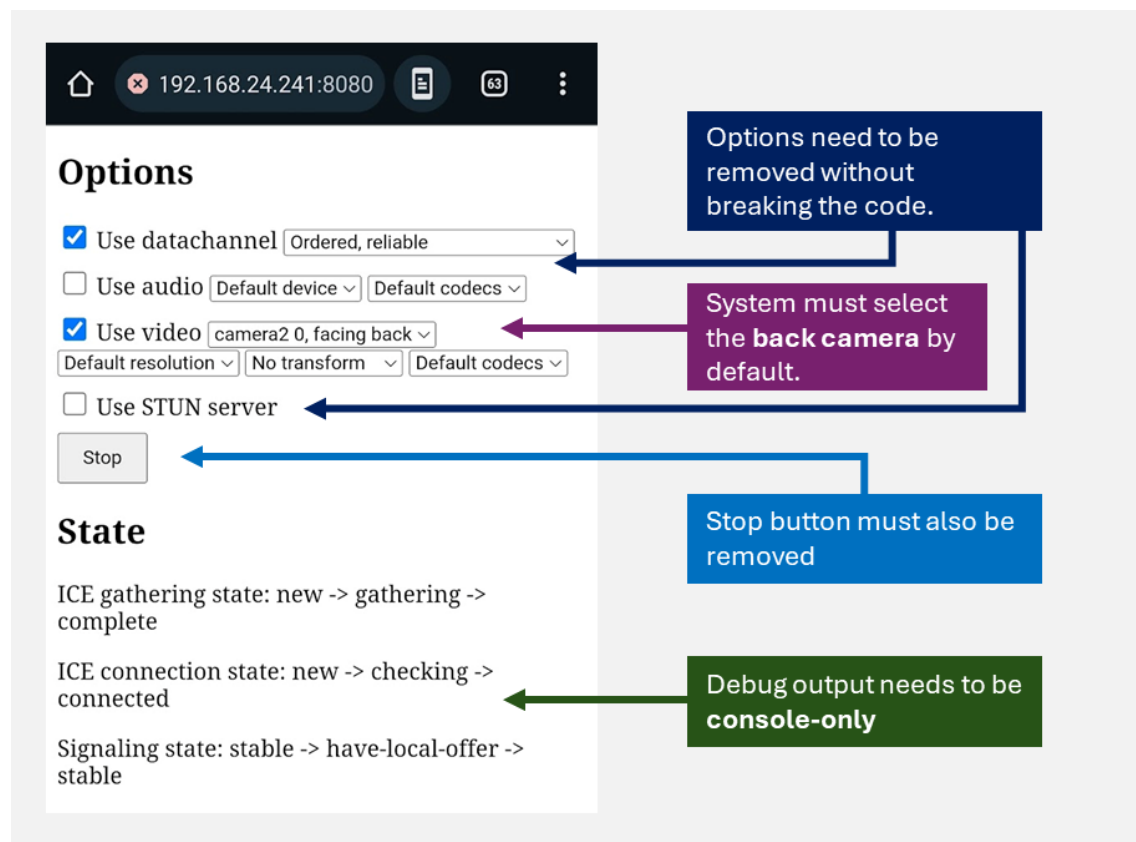


Figure 31 - Default UI & Suggested Changes. Source: AIORTC

The analysis shown in Figure 31 allowed for the integration of a user interface accurately matching the design specification (see Figure 16). This is documented in the second prototype case study.

5.4.3 Integrating WebRTC and Python

AIORTC usage was successful in integrating components together during the second development cycle. The full JavaScript client code and the Python inference backend are provided in the Appendix at Section 0, although this section will highlight and discuss code snippets of particular interest.

```
class VideoTransformTrack(MediaStreamTrack):
    kind = "video"
    def __init__(self, track, transform):
        super().__init__()
        self.track = track
        self.transform = transform
```

```
async def recv(self):
    frame = await self.track.recv()

    # ( ... Inference and Frame Manipulation ... )

    return new_frame
```

Figure 32 - VideoTransformTrack construction code

The WebRTC API utilises the MediaStreamTrack class through JS and Python in order to hook into target devices and interpret a response. Figure 36 shows how these objects are constructed in the codebase. Asynchronous execution is also applied via the `recv()` function, which allows for the code to await each frame before feeding through the rest of the code.

The class definition shown above is then instantiated in the `offer()` function, which will also instantiate the **RTCPeerConnection** class in order to handshake with the client. This is shown in Figure 33. The peer connection object also allows for annotations to signify the workings of specific methods within the class, which is highly useful for event logging and the transmission of information across data channels.

```
async def offer(request):
    params = await request.json()
    offer =
    RTCSessionDescription(sdp=params["sdp"], type=params["type"])

    pc = RTCPeerConnection()
    pc_id = "PeerConnection(%s)" % uuid.uuid4()

    # Event Logging Example
    @pc.on("track")
    def on_track(track):
        log.info("Track %s received", track.kind)

        if track.kind == "video":
            pc.addTrack(
                VideoTransformTrack(
                    relay.subscribe(track),
                    transform=params["video_transform"]
                )
            )
        if args.record_to:
            recorder.addTrack(relay.subscribe(track))
```



```
# Return our peer connection
return web.Response(
    content_type="application/json",
    text=json.dumps(
        {
            "sdp": pc.localDescription.sdp,
            "type": pc.localDescription.type
        }
    ),
)
```

Figure 33 - VideoTransformTrack Instantiation Code

Handshaking involves establishing a direct peer connection between the server and each client. This is explained below with a comparison to the previous setup in Figure 34.

Phase 1 DroidCam setup



Phase 2 Peer Setup

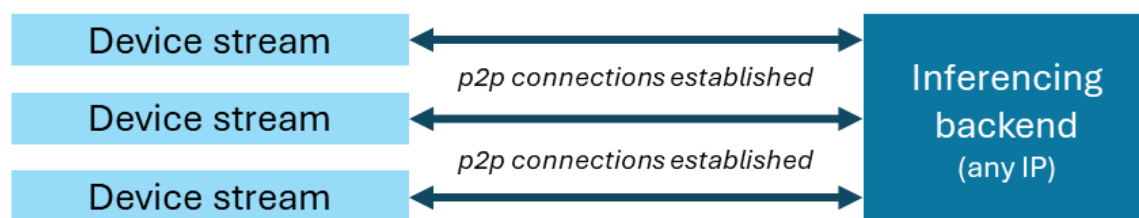


Figure 34 - DroidCam p2p Setup

Event logging methods proved to be advantageous during development. Various network adapters on the server PC were set up from previous projects in ways that interfered with the current setup, including disruption from a VirtualBox adapter used on the same machine previously for a different project. Although seemingly trivial, this is a useful find; should a system such as this one be deployed in a production environment, it is now known that disabling various irrelevant network adapters will speed up the handshaking procedure. *(The system was attempting to connect to random 169.x.x.x. addresses from previous projects).*


```
===== Running on https://192.168.0.28:8080 =====  
(Press CTRL+C to quit)  
INFO:pc:PeerConnection(04036aab-1242-4500-96d2-c00a463df6ab) Created for  
192.168.0.18  
INFO:pc:PeerConnection(04036aab-1242-4500-96d2-c00a463df6ab) Track video  
received  
INFO:pc:PeerConnection(04036aab-1242-4500-96d2-c00a463df6ab) Connection  
state is connecting  
INFO:aioice.ice:Connection(0) Check CandidatePair(('192.168.0.28', 55578) ->  
( '192.168.0.18', 51077)) State.WAITING -> State.IN_PROGRESS  
INFO:aioice.ice:Connection(0) Check CandidatePair(('192.168.0.28', 55578) ->  
( '192.168.0.18', 51077)) State.IN_PROGRESS -> State.SUCCEEDED  
INFO:aioice.ice:Connection(0) ICE completed  
INFO:pc:PeerConnection(04036aab-1242-4500-96d2-c00a463df6ab) Connection  
state is connected
```

Figure 35 - Console output on ICE peer connections

5.4.4 HTTPS: Protocol Certificate Errors

Issues were uncovered during development with WebRTC's de-facto security standards. Since WebRTC follows the safety protocols adhered to by Google, software requirements included the use of HTTPS to transmit camera information. This is not typically a problem for hosted web servers (Sellers, n.d.). For this system, the development environment was self-hosted, and this required the use of a self-signed certificate. Failure to execute the backend script with appropriate certificates would disallow the core functionality, displaying a message that the "site cannot provide a secure connection."

OpenSSL was used to self-sign certificates, generating a certificate file and a key file. The file paths for "**cert.pem**" and "**key.pem**" were then provided as arguments when executing the script (a simple Windows batch script can perform this task).

5.5 GPU Inferencing

Until this point, inferring the model on a frame was a CPU-based operation (and model training was conducted on a cloud-based GPU environment). Since model inference is now a fully functional part of the application, research was conducted into the ways to utilise a local GPU in order to infer frames more efficiently. Framerate testing was also performed on both CPU and GPU architecture in order to verify the prediction that inference performance would improve. In order to inference on the GPU, various tools and programs had to be downloaded and utilised.

5.5.1 CUDA and CuDNN: GPU Setup

Local system runs were conducted on both a home PC and a laptop. The components requiring setup included both a standard NVIDIA RTX 3070Ti GPU, and an RTX 3070Ti Laptop GPU. Since these devices are NVIDIA hardware, a number of setup procedures had to be implemented in order to access their full functionality. The setup steps were repeated for both devices.

The figures below provide descriptions of the relevant GPU setup tools (Figure 36), colour-coded to highlight the dependency pipeline for the software architecture (Figure 37).

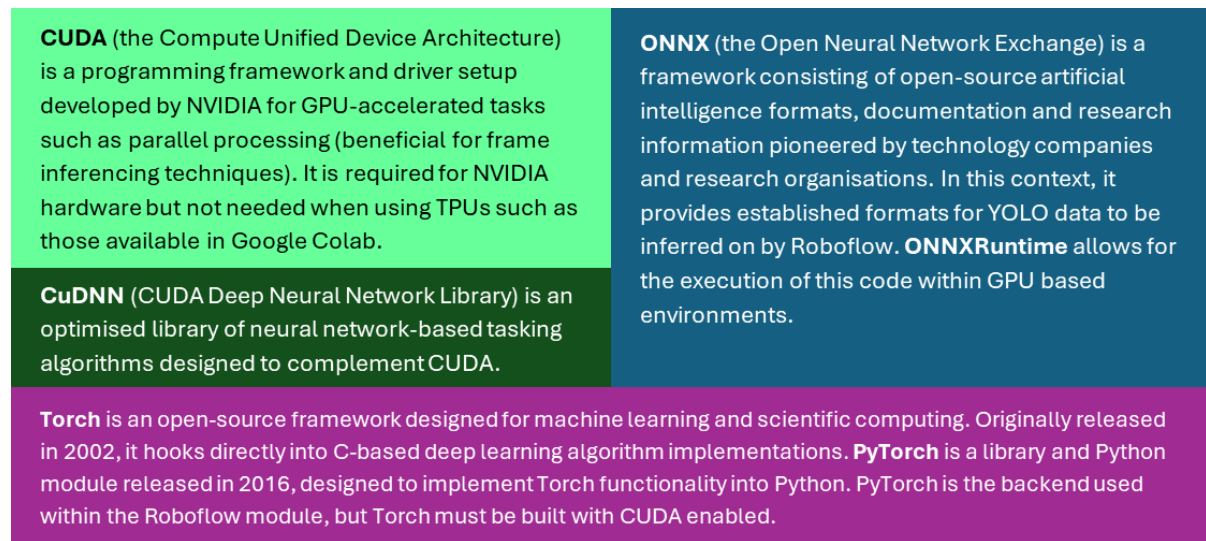


Figure 36 - GPU tool descriptions

Roboflow inference makes use of a PyTorch backend, converting frames into an ONNX-defined format for YOLO interpretation. Both PyTorch and ONNXRuntime will communicate with individual CUDA cores on the hardware during model inferencing via the CUDA and CuDNN drivers.

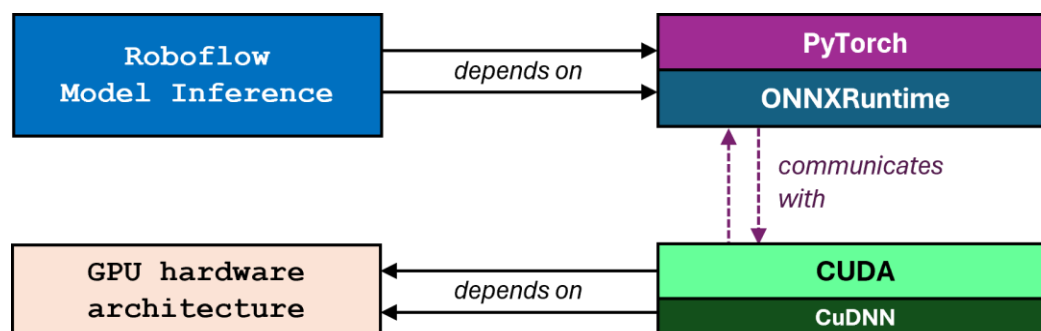


Figure 37 - Dependency graph for GPU inference

5.5.2 Cross-Compatibility: Challenges

CUDA implementation within the project involved challenges into ensuring various software components were compatible. For example, at the time of development, PyTorch did not have

official binaries for source building with CUDA 12.0 (PyTorch Forums, 2024). As a result, CUDA 11.8 was used. The compatibility assurances for CUDA require specific versions of CuDNN and the Roboflow inference package in order to ensure intended functionality is deployed across the project; the full list of Python module versions and software tool compatibilities is documented in Appendix at Section C.7.6.

5.5.3 Second Prototype: Demonstration

Some demonstrative screenshots are provided below to show the implemented UI at this stage, including the inference in action.

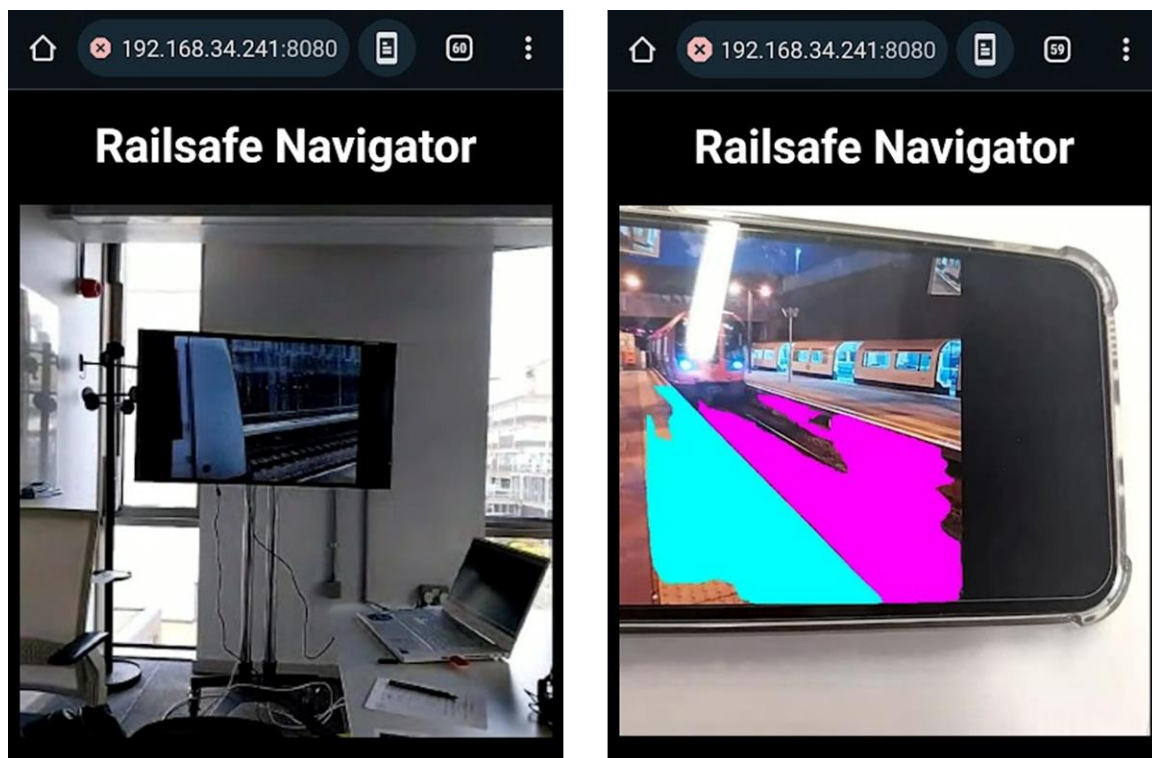


Figure 38 - Second Prototype Demonstration

The system is shown to be successful in detecting platforms and tracks when the camera is pointed at relevant images on a second device.



The second development iteration contained the implementation techniques discussed up to this point. **Section 7.2** provides an analysis into the results found in this cycle, challenges encountered, and next steps considered.

5.6 Audio Implementation

Undertaking audio integration is the final fundamental step in ensuring the system meets the accessibility requirements documented in the project objectives. At this stage of the project, participant testing has already concluded. Due to time constraints it was unfortunately not possible to test the audio based functionalities with external users. This is documented more extensively in Section 6.1.2.

5.6.1 Initial Research: Audio Frameworks

Original plans were to research frameworks such as PyAudio in order to play sounds directly from the inference function. However, this was not deemed to work as the project will require audio to play on client devices, meaning audio integration has to be carried out on the JS end of the system. However, since the client code does not receive any data about detections – only the colour coded image feed, a way of passing information between the two systems had to be added.

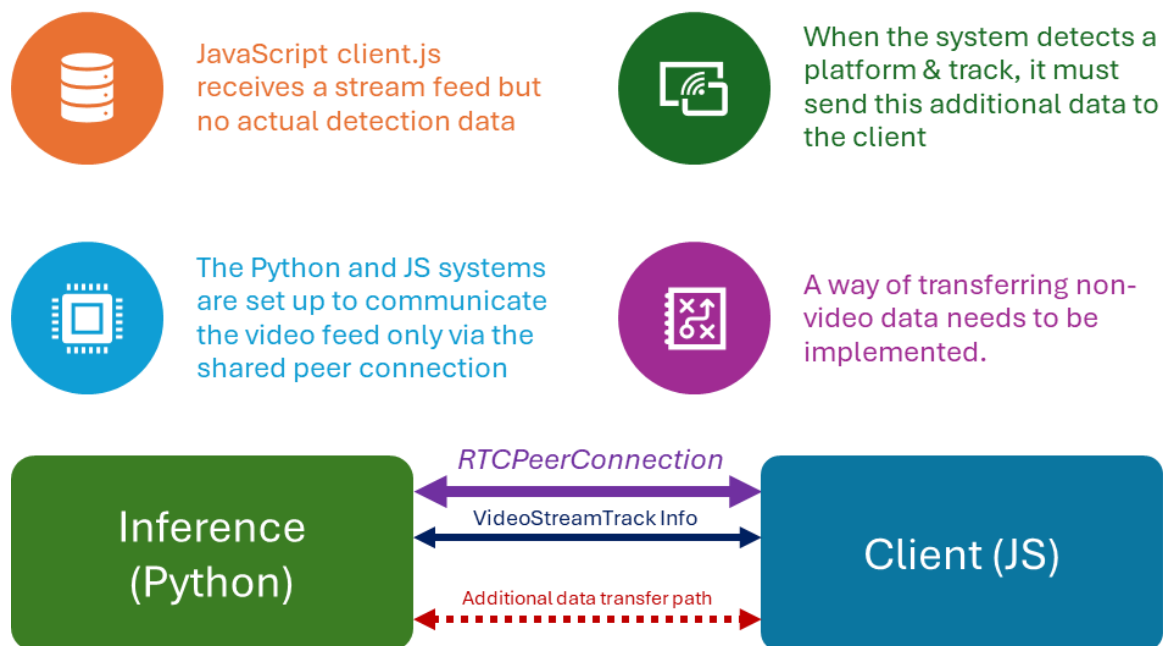


Figure 39 - Information pipelines

5.6.2 Working with Data Channels

Research into the RTC API was conducted in order to attempt to resolve the problem revealed in Figure 39. The **RTCPeerConnection** class contains a subclass: **RTCDataChannel**. Data channels allow for communication of text-based strings between communicating nodes. The class contains methods for sending messages on a channel, and event listeners for receiving messages.

```
# (Python Code - Sending testMsg)
# When the data channel is detected...
# we'll send a message to the JavaScript console
# (pc = Peer Connection object)

@pc.on("datachannel")
def on_datachannel(channel):

    dc = channel
    channel.send("testMsg")

// (JavaScript Code - Receiving testMsg)
// Event listeners created for opening...
// ...channel and receiving a message

dc = pc.createDataChannel('InferenceResult');
dc.addEventListener('open', () => {
    console.log("Channel opened")
});

dc.addEventListener('message', (evt) => {
    if (evt.data === 'testMsg') {
        console.log("test message!");
    }
})
```

Figure 40 - Sending and receipt of data channel messages

This setup was used to send a text string from Python to JavaScript on successful platform and track detection. Should the message be picked up by the event listener, the client can play a specific audio file stored on the server.

5.6.3 Detections: Left and Right

Once the data channelling was set up, two audio files were created and assigned to hidden HTML audio objects that can be played using a simple `getElementById().play()` method. These MP3 files were named LeftDetection.mp3 and RightDetection.mp3. The software Audacity was also used – this allowed for exporting the detection sound, albeit muted in one channel. (Right channel muted for LeftDetection and vice versa).

The method used to determine whether a platform was to the left or right of the user was not as robust as originally intended and could have been better in hindsight. The inference code was modified to account for the following constraints:

1. Only the most significant (highest confidence) platform and track detections will now be overlaid onto the image masks.
2. The system will only return a positive result when both a platform and track are
3. detected simultaneously.

It was decided that this was acceptable for this system since the model precision was already deemed to be high enough for this to be successful. To mitigate risk, the additional measure of setting confidence thresholds for detection much lower was also applied.

Following this, two data channel messages were set up on both scripts: 'detectLeft' and 'detectRight'. The following logic determined which message would be sent by the backend:

If the platform is detected to the **right** of the track, we will assume that the track is to the **left** of the user's camera.



If the platform is detected to the **left** of the track, we will assume that the track is to the **right** of the user's camera.

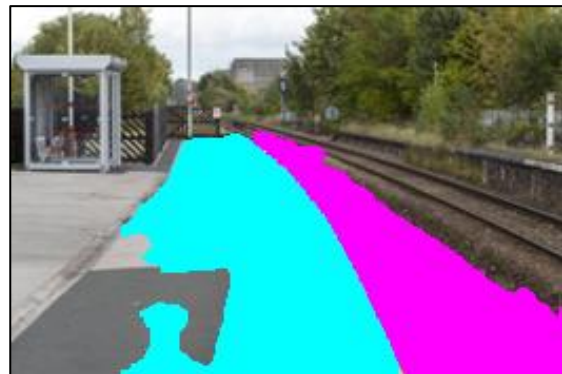


Figure 41 - L/R Logic.

Calculating the position of detections was done by analysing the polygon data returned by OpenCV.



The third development iteration contained the implementation techniques discussed up to this point, in addition to extensive feedback gained from testing, which is documented in the next chapter.

6 Testing

In addition to testing individual prototype case studies in Chapter 5, testing into the system's usability and effectiveness was also conducted with the help of recruited participants. This chapter will provide an insight into this testing and self-testing measures in the late stages of project development, with Section 6.1 detailing the participant testing phase and Section 7.4 providing an overall insight into usability testing against a design heuristic.

6.1 Participant Testing

Participant testing for this project was undertaken on Thursday 14 March and Friday 15 March. The objective was to recruit a minimum of ten people to verify how effective the web application ran on their devices, with reports of system functionality detailed against the different hardware each tester had with them.

6.1.1 Testing Formats

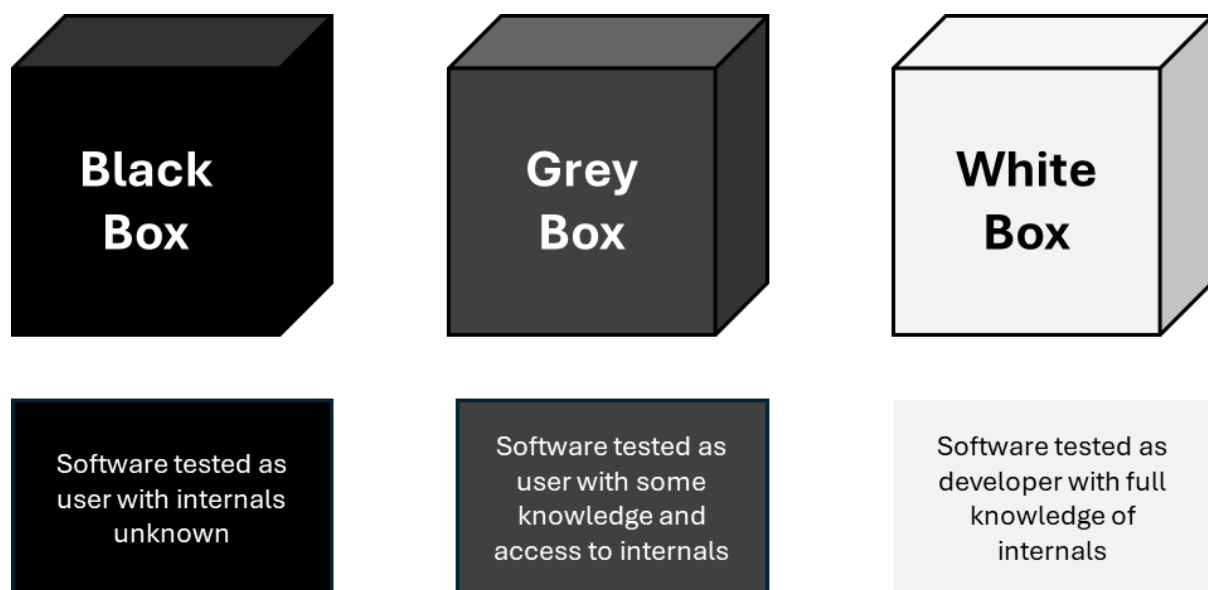


Figure 42 - Box Testing Methods. (Adapted from www.ranorex.com)

For the purposes of this project, **Black Box Testing** was used for participant tests. This refers to where users do not have any prior knowledge about the internal workings of the software or the code behind it, unlike white box testing, where users will have a solid understanding and test a system's functionality at a lower-level setting.

Black box testing has key benefits when used in this setting. For instance, it allows us to simulate the real interactions of users and scenarios and uncover issues related to usability from the lens of an end user – issues that may not have been picked up otherwise.

Although useful in participant testing, it could also be argued that black box testing limits the scope of possibilities as it may be more difficult to uncover more deep-rooted bugs in the code. Despite this, it was still decided on as the best route for this type of analysis.

Table 5 below provides explanations as to each step undertaken during the testing process.

Step	Description
1)	The application is tested initially against a control device of which it has been developed to work perfectly for prior to testing (researcher's own device).
2)	Server backend is initialised on a PC connected to a local network.
3)	Users are given a consent form and participant information sheet to sign.
4)	A user connects to the local network with their device and opens the local URL for the application page on their device. At this point, it is recorded whether the page works successfully on their device, as well as any significant relevant differences between the control device functionality and the user's device.
5)	a) Should the device work successfully, the user is asked to point their camera at printed A4 copies of two distinct railway platform images to verify that model inference is working successfully on the device. Results are recorded as successful or non-successful. Users are then asked to point their device at a large screen containing railway video footage to verify the workings of model inference on motion picture (Friday 15 March only).
	b) Should the device not work successfully, the participant is handed the control device and uses this instead (to allow the collection of some usability feedback in the case of high failure rates). User undertakes the same steps as in a).
6)	In some instances, the researcher adjusted the framerates or other settings of the application as an additional measure, which is recorded, to see if it makes a difference to functionality.
7)	Users are thanked for participating and sent a questionnaire to fill out as a cross-reference; which makes use of modified SUS (system usability scale) survey questions. This also allows for the collection of additional feedback via comment boxes.

Table 5 - Participant testing procedure

6.1.2 Setup and Limitations

The original plan was to test on a very large lecture room screen to attempt to mimic a real-life setting. Due to scheduling constraints this was not possible; however it was fortunate that the room used for testing was spacious with lots of windows as this allowed for increased visibility of the testing images.

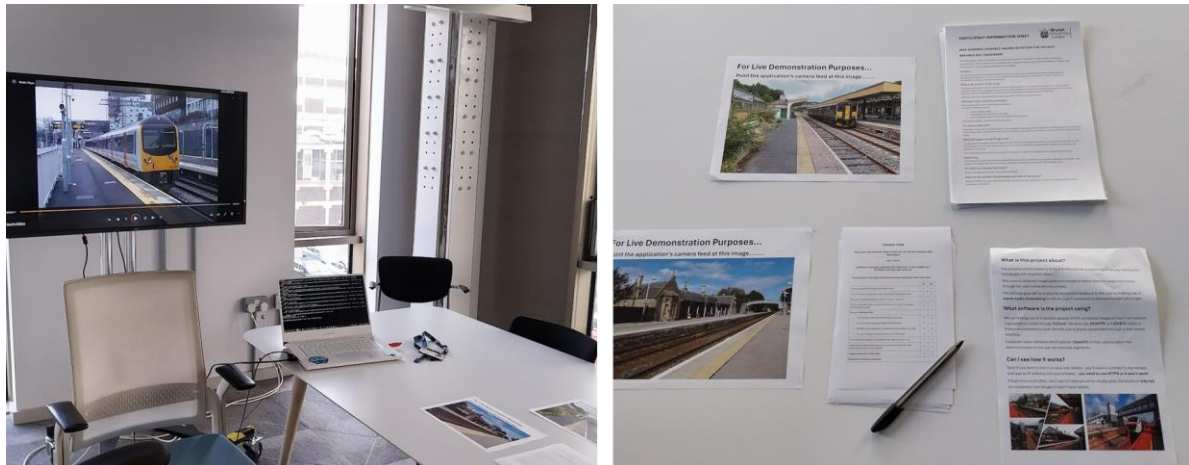


Figure 43 - Testing Setup

Testing was extremely successful in identifying previously unseen problems and participant feedback was extensive and thorough. A total of 13 participants ended up testing the system; 8 tests were conducted on Thursday (see Appendix D: Made in Brunel) and 5 tests were conducted on Friday.

Although the testing process was widely a success, it is important to note the limitations of this analysis. Before system implementation had commenced, the original plan was to test with obstacle navigation of an indoor environment. As a result of this, a risk assessment had to be completed and clearances had to be made with the ethics committee to allow this.

In addition, testing had to be undertaken on university premises and could not involve the use of any participants with ‘uncorrected visual impairment.’ In the end, it was not deemed necessary to test the system via use of obstacles as the testing focus was on determining how well the system worked across different devices.

By not using visually impaired participants, the usability feedback of the application is limited in its scope. However, insight into device compatibility is still widely useful.

6.1.3 Participant Testing Results

This section provides a subsection of interesting results found upon testing the system with 13 overall participants. The control settings were default settings known to work with the application at all times.

Control Settings (These are the default settings used during testing the application) Researcher's phone model is a Samsung Galaxy A23 5G	
Web Framerate was 2 FPS. Web Stream Resolution was 640x640. Confidence Threshold was 0.65. Default orientation was Portrait mode.	
Participant 6 Thursday 14 March Samsung Note 9	
<i>Did the system work on their device?</i>	No
<i>Which camera was detected?</i>	N/A – Did not work
<i>Were errors reported by the application?</i>	Yes - OverconstrainedError
<i>Were any additional adjustments made to the application parameters after the first failure?</i>	Yes – confidence percentage was lowered from 0.65 to 0.25
<i>Did the application work following any changes made?</i>	No
<i>In the event of failure, did the participant use the control device?</i>	Yes – Notes provided

Notes: Suggestions were made that increasing the confidence threshold may help to overcome the errors given, which seem to be due to performance issues. When tested on the control device, this participant noted that they had to tilt the phone quite a bit to get it to work.

Participant 10 Friday 15 March iPhone 10S Max	
<i>Did the system work on their device?</i>	Yes
<i>Which camera was detected?</i>	Back
<i>Were errors reported by the application?</i>	No
<i>Were any additional adjustments made to the application parameters after the first failure?</i>	Yes – Web retriever FPS increased from 2 FPS (Control) to 15 FPS.
<i>Did the application work following any changes made?</i>	Yes
<i>In the event of failure, did the participant use the control device?</i>	N/A

In this testing instance, perfect functionality on an Apple device was noted for the first time. The application did not work in **landscape** on this device the first time this was tested. The application did not lag in frame retrieval on 2 FPS – but did lag a bit more when changed to 15 FPS.

Participant 13 Friday 15 March iPhone 14 Pro Max	
<i>Did the system work on their device?</i>	Yes (see notes)
<i>Which camera was detected?</i>	Front
<i>Were errors reported by the application?</i>	No
<i>Were any additional adjustments made to the application parameters after the first failure?</i>	Yes – (i) : setting was changed to force the application to use the back camera. (ii) : the resolution of the web stream was trialled at 1024x1024, with a previous control setting of 640x640.
<i>Did the application work following any changes made?</i>	Yes (i); Yes (ii)
<i>In the event of failure, did the participant use the control device?</i>	N/A
The system did not work initially in this instance – it was found that we had to turn off the user's VPN in order to get it to work.	

It was noted that this result was quite possibly the best seen. This could be down to the fact it used a new phone model.

Inference on printed images was deemed very good; motion inference was deemed to be functional but not as good.

(i): Forcing back camera use on a software level was avoided previously due to research done into compatibility but it was decided to trial it here.

(ii): The system worked at 1024x1024 resolution on this phone but was noticeably more latent.

Table 6 - Subsection of participant test results

7 Evaluation

7.1 First Prototype: Case Study

At this stage, the model was able to infer platform edges on the DroidCam feed input in real-time. This section discusses some of the problems encountered during this stage of development as well as attempts to resolve problems, and an insight into the steps that were planned for the next development cycle.

7.1.1 Model Results

For the first cycle, the model only has the 'platform' class. The model was trained on 13/02/2024 with an 81-24-10 test/train/valid split, for a total of 25 epochs and an image size of 640.

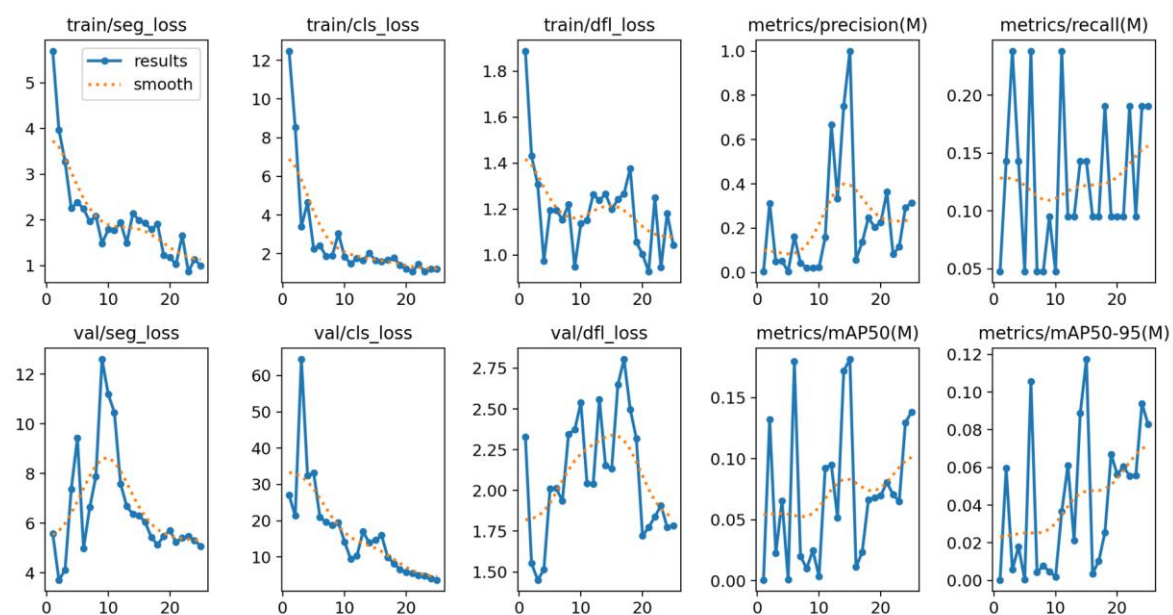


Figure 44 - First Prototype Model Results

The graphs shown above shows the epoch number on the X-axis and a selection of dependent variables on the Y-axes. Table 7 provides descriptions for each metric.

Measurement		Description
Segmentation Loss	seg_loss	Signifies the instance segmentation error rate on a pixel-wise level.
Classification Loss	cls_loss	Represents the error rate in classifying detections as platforms. In YOLOv8, this makes use of binary cross-entropy. (Ultralytics, 2023)

Measurement		Description
Distribution Focal Loss	df_l_loss	Represents the error rate in predicting objects with respect to typical object distributions across images.
mAP50 and mAP50-95		Represent the Mean-Average Precision, an evaluation metric used throughout object detection to contrast varying levels of precision and recall across different IoU (Intersection-over-Union) thresholds. In simple terms, this links to the level of overlapping between nearby objects. mAP50 measures precision and recall for IoU thresholds below 0.5, while mAP50-95 measures for thresholds between 0.5 and 0.95.
Precision		<i>See section 5.2.2</i>
Recall		<i>See section 5.2.2</i>

Table 7 - Measurements & Descriptions

Result analysis was performed with the assistance of ChatGPT's Data Analyst within a GPT-4 environment and cross-referenced with prior knowledge gathered from the sources above.

- This model has shown an ability to learn the detection patterns, with a general trend of decreasing loss shown as training progresses.
- However, the validation loss range is shown to be broader than the training range
- The segmentation loss graph for validation does not reach as low as the training equivalent, suggesting the model is overfitting.
- High variance is detected in precision and recall rates, suggesting a sensitivity to the size of objects within training images.
- Inconsistency was found across mAP rates.

7.1.2 Problems Encountered

Two main problems were encountered in the first implementation cycle. The table below discusses these problems and provides suggestions for ways they can be mitigated and/or resolved in the next cycle.

Problem	Suggestions for improvement
Poor Model Performance The model is providing poor results – specific details discussed in Section 7.1.1.	• Increasing the number of training images by scraping and annotating more data. • Attempting to train with a filter such as adaptive median applied to input and output, in order to compare results.

	• Attempting to train the model at higher settings e.g. image resolution of 1024, smaller batch size, etc.
Poor Performance on Inference The application was not providing sufficient frame inference speeds to meet the project objectives.	• Experimenting with multithreading and parallelism process frames synchronously within Python • Using GPU resources for inference to attempt to reduce latency.

Table 8 - First Prototype: Problems Encountered

7.1.3 Next Steps

Analysis of results and issues detected led to the formation of an objectives list for the next development cycle. The list was as follows:

- 1) The model needs to be upscaled to at least 200 images and tested to improve accuracy rates.
- 2) The system needs to be moved away from a fixed stream towards a dynamic, client based architecture via camera integration.
- 2) The UI needs to be implemented in a basic form.
- 3) GPU utilisation needs to be attempted to improve performance.

Figure 45 - First Prototype: Next Steps

7.2 Second Prototype: Case Study

The second cycle of development successfully moved the project towards web-based interfacing and implemented GPU utilisation for inferencing. At this stage, the training images had been updated to include learning annotations for both platforms and tracks.

This section discusses some of the problems encountered as well as extensive comparison to the first prototype, along with the enhancements made to the model and the steps ready to be undertaken for the next prototype.

7.2.1 Comparative Model Testing

Following poor model performance previously, suggestions were made to upscale the model to at least 200 images and attempt to train at higher resolutions. For this cycle, two models were trained with the same images and parameters; the second model using Otsu's thresholding to preprocess the images (see section 2.2.2). This was useful in determining if filtering can potentially improve the results.

Additionally, previous results depicted in Figure 44 imply best values for precision between epochs 10-20. Based on this information, a decision was made to reduce the number of epochs from 25 to 10.

Attempts were made to train on the local GPU unsuccessfully, with memory issues cited as a failure reason. The models were trained on 338 images with a 237/69/32 train/test/valid split. An image size of 1024 and batch size of 16 were used for a total of 10 epochs, making use of a Tesla T4 cloud GPU. The unfiltered set took 11 minutes and 3 seconds to train; the filtered set took 11 minutes and 54 seconds.

Figure 46 shows how precision varied across confidence for both classes in each model.

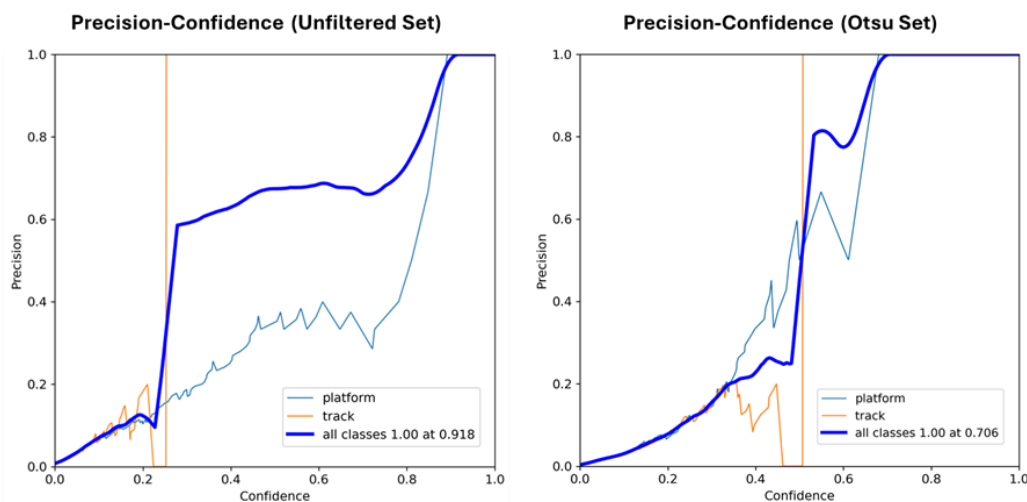


Figure 46 - Precision-Confidence comparison between sets

The unfiltered set shows fluctuations in precision at low confidence values, before a jump at around the 0.2 mark; where the track class jumps to maximum. The filtered set shows a lower precision at equivalent values; although the jump to maximum also occurs at lower values. This suggests that the Otsu-trained model is better at ignoring false positives on average for lower confidence detections, but less effective for higher confidences. This is a positive result for the project due to the safety of setting lower confidence thresholds on inference.

Although precision was very good for this model, recall values were not so good. Figure 47 shows how recall varied across confidence for each class.

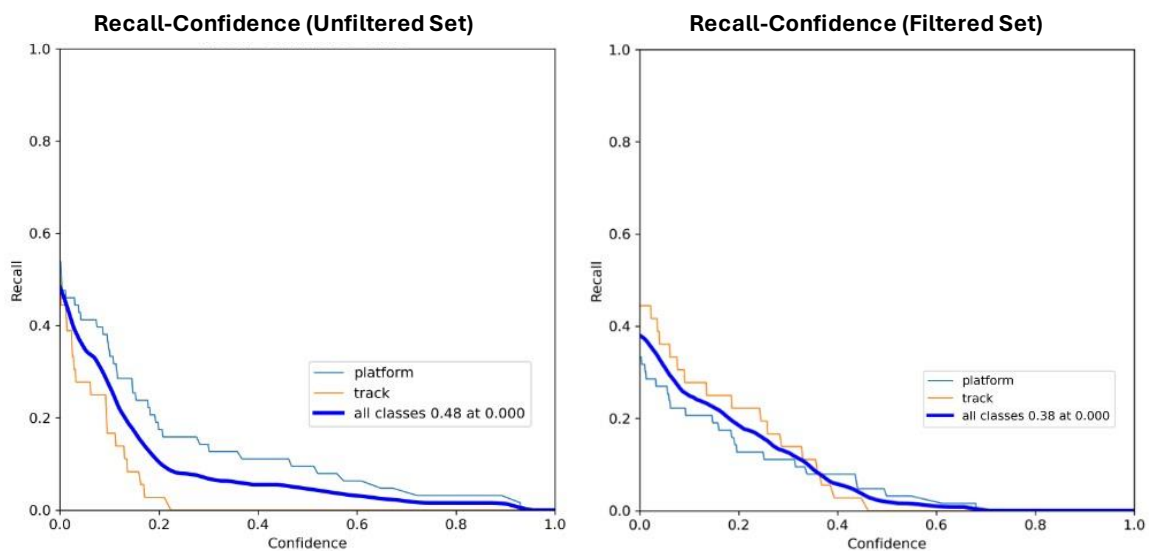


Figure 47 - Recall-Confidence comparison between sets

As shown, recall values across both classes never reached a maximum value and dropped to 0. Analysis conducted into model results states the following:

For a given class:

- A high precision and a high recall → The class has been **well managed** by the model.
- High precision and low recall → The **class is not well detected** but when it is, the model is very reliable.
- Low precision and high recall → The **class is well detected**, but also includes observations of other classes.
- Low precision and low recall → the **class has not been handled well** at all.

(Data Science Courses | DataScientest, 2023)

Based on this information, it is clear that the model is not handling detections effectively, but the detections that are picked up are almost universally not false positives, especially at high confidence values.

7.2.2 Problems Encountered

Cycle 2 Problems	Suggestions for improvement
Lack of External Evaluation Inference has only been tested on a single device up to this point, without knowledge of alternative hardware.	Undertaking participant testing and recording the phone model of participants, to verify system functionality.
Low Recall Values Although the model is performing better than previously, its ability to find all correct detections is insufficiently low for the project use case. Recall scores for the model are still low, although filtering with Otsu seems to have produced slightly better results.	- Expanding the dataset to utilise Roboflow augmentation techniques on images. - Expanding the dataset further to utilise filtered images as additional augmentations, testing other filters and comparing results.
No Qualitative Data Although this project will not use a typical design evaluation (see Table 2), it has been suggested that some qualitative feedback would be of benefit to the final results.	The inclusion of some modified SUS survey questions within participant testing.
No Audio Implementation The system is functional in its goal of retrieving images and determining detection locations but does not currently provide this feedback to the user in an accessible format.	Attempting to implement stereo channelling in the next cycle.

Table 9 - Second Prototype: Problems Encountered

7.2.3 Next Steps

Analysis of results and issues detected led to the formation of an objectives list for the next development cycle. The list was as follows:

- 1) The model needs to be tested externally by participants – this was in the process of being arranged during this cycle.
- 2) The model needs to be upscaled further using augmentation techniques.
- 3) The UI needs to be evaluated against a usability framework.
- 4) The audio system needs to be implemented.

Figure 48 - Second Prototype: Next Steps

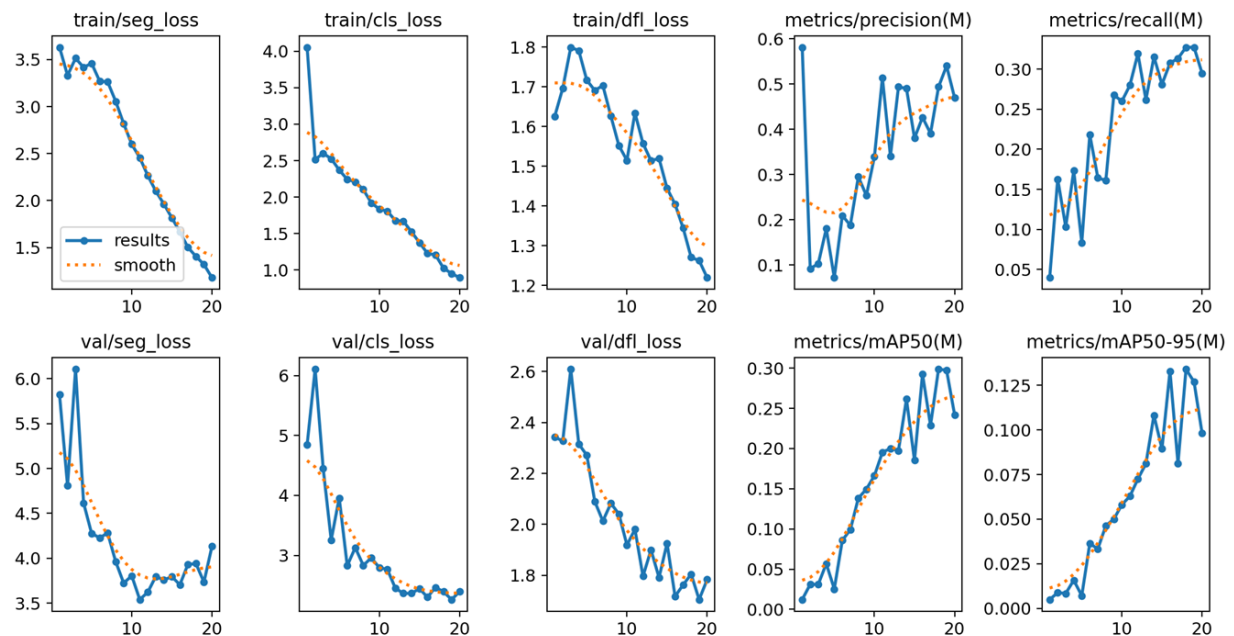
7.3 Final Prototype: Notes

The third prototype was the final development cycle for this project. Implementation at this point was near full functionality as intended, with the model upscaled a second time, participant testing providing usability feedback, and the audio implementation working as intended. As with all development cycles in a project such as this one, problems were still encountered. This is the version of the project submitted alongside this dissertation.

7.3.1 Final Model Results

The final model iteration was upscaled via the use of augmentations, from 338 images to a total of 812 images. No other filtering techniques were used; attempts were made to train a 1624 image model using 812 standard + 812 Otsu images but this was unsuccessful with memory constraints cited.

An image size of 1024 and batch size of 16 were used for a total of 20 epochs, making use of an NVIDIA RTX 3070Ti Local GPU.



These results will not be explained in full, as this has already been done extensively in previous cycles. However, it is apparent that this model performed much better with augmentations added, and for the first time, an upward trend in recall is seen across confidence. This model was far better than previous attempts, with most metrics optimising around epochs 18-19. Further optimisation will focus less on upscaling the model and improving efficiency, and more on exploration into alternative model architectures that could provide a better output (see next section).

7.3.2 Future Considerations

Future work for the project as a whole is documented in Chapter 8. The table below summarises specific suggestions from the final prototype results.

Cycle 3 – Potential Enhancements	Future Considerations
Audio Interface Testing The audio-based feedback was never tested by participants due to time constraints.	A more sophisticated participant re-evaluation including re-submission of ethics to account for participants with visual impairments, to provide a full usability evaluation.
Testing Alternative Models Only the YOLOv8 segmentation model was tested during the course of the project. YOLOv9 was looked at but does not support segmentation at time of writing.	Training on different model architectures and collating test results to evaluate potential performance improvements.
Inference Deployment Platforms One of the key constraints in deployment of the project is the need for a network connection in order to infer the model (whether this is internet-based or a local network URL).	Deploying the model to mobile hardware via an offline application containing the trained model data. This could also allow for integration with existing accessible technology on an individual's device.
Expansion of functionality The system could be expanded into other areas of rail accessibility by hooking into existing APIs and systems to retrieve additional railway-related data.	Examples of this could include: • The use of microphones to detect platform announcements and relay to the user's headphones • The use of TTS (text-to-speech) systems to provide information about step-free access, braille locations or other localised information • The use of dedicated hardware within the infrastructure, such as agreements with railway operators to host the model at stations, or the use of dedicated cameras to perform the inference tasking. • Dedicated camera usage to provide a more discreet way to use the system, which could enhance user safety.

7.4 Heuristic Evaluation

Following usability testing, a modified SUS survey questionnaire was emailed to participants in order to collect some qualitative data regarding the system's usability.

This section documents the questionnaire results and evaluates the findings in order to provide some user-centred data.

7.4.1 System Usability Scale

The System Usability scale was pioneered by John Brooke in 1986 and was designed to be adaptable to a range of products and services (Usability.gov, 2019). It is a user-centric approach with advantages over other methods of data collection, such as the ability to standardise testing results for comparison between systems and measure a range of different factors such as satisfaction and learnability of a system.

Although the project was unable to evaluate on visually impaired participants, the use of generalised questions such as these can still provide feedback, as long as the limitations of the feedback are understood by those interpreting it. For this project, nine questions were used and participants were asked to provide additional comments, should they wish.

The questions were as follows:

SUS Survey Questions	
<i>Rated on a scale from 1 (Strongly Disagree) to 5 (Strongly Agree):</i>	
1)	I thought the software was easy to use.
2)	I felt very confident using the software.
3)	I would need the support of a technical person to use the software.
4)	I found the software unnecessarily complex.
5)	I thought there was too much inconsistency in the software.
6)	I found the software very cumbersome to use.
7)	I found the software very rigid and inflexible to use.
8)	The software responded too slowly to my inputs.
9)	I felt very satisfied with the software.

Table 10 - SUS Survey Questionnaire

7.4.2 Evaluation of Results

Results for all questions are shown in Appendix C.6.1. This section will evaluate some of the most interesting findings.

Out of the 13 participants emailed, 6 participants provided responses. 3 of the participants attended testing on Thursday at Made in Brunel, and 3 attended testing on Friday in WLFB 209. Of these 6 participants, 4 out of 6 reported success on their device, with 2 out of 6 handed the control device to use.

(Graphs here are rated from 1 – Strongly Disagree to 5 – Strongly Agree)

I thought the software was easy to use.

6 responses

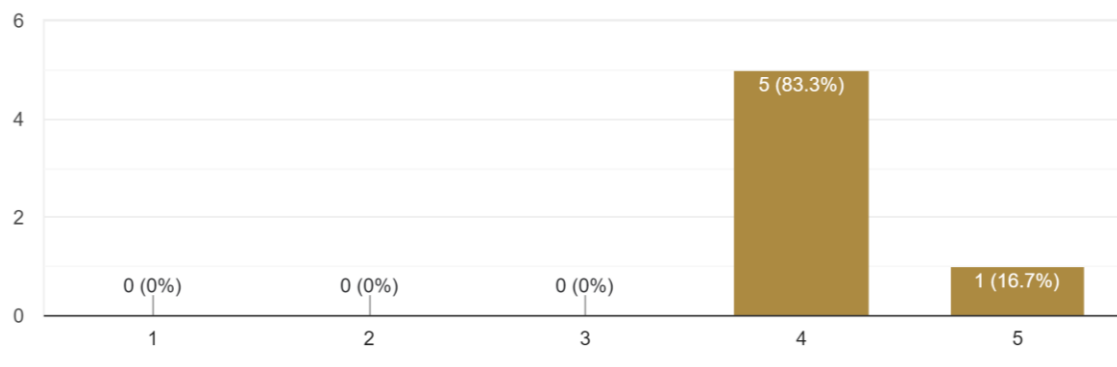


Figure 49 - SUS Q1

Regarding ease of use (Q1), responses were consistent that the system was easy to use, suggesting a high learnability rate. This is a positive outcome but should be taken with a grain of salt; participants with visual impairments are the intended userbase and may not find the system as easy to navigate.

I would need the support of a technical person to use the software.

6 responses

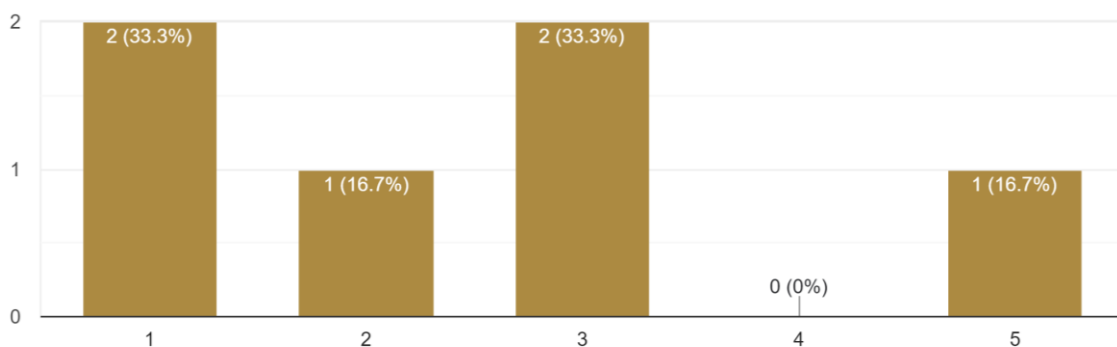


Figure 50 - SUS Q3

Results for Q3 were much more inconsistent and provide an insight into the technical scope of the testing base. The mean of response values was 2.5, which suggests that although the system is usable, it may require enhancements to be fully adaptable to a wider range of users.

The software responded too slowly to my inputs.

6 responses

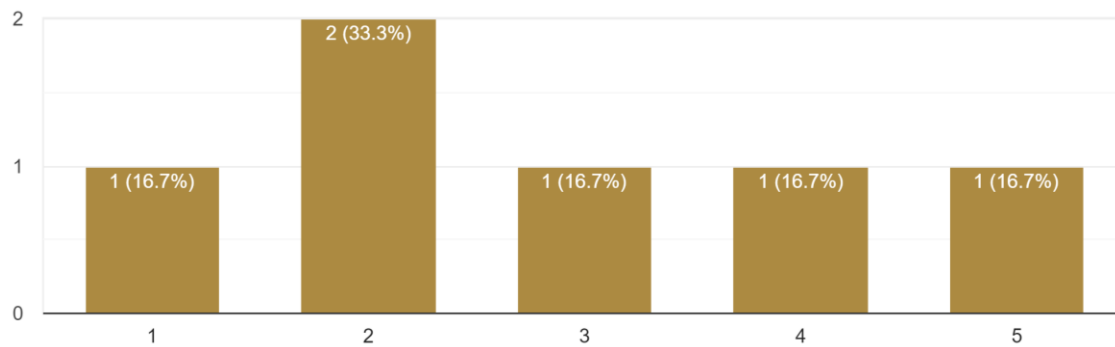


Figure 51 - SUS Q8

Results found in Q8 were also inconsistent across participants, suggesting performance is hardware dependent. Interestingly, two of the six participants reported using an iPhone 14 Pro Max, with one participant reporting a 2 and the other reporting a 1 on this question. Two participants, using Samsung devices on the contrary (with code adjustments made and documented since software originally did not work on their device) reported a 4 and a 5. These results suggest that improvement is needed to ensure the system does not suffer performance failures should it be used in a real-world setting.

7.4.3 Comment Feedback

Four of the six participants gave additional feedback in the provided comment box. This is documented in Appendix C.6.2.

8 Future Work

This section expands on the considerations documented in Section 7.3.2 in order to provide more detail about future development considerations of potential improvement, following evaluation. The system has provided an adequate reading into proving the use of artificial intelligence within an accessibility context, although future considerations are possible to further enhance the system success.

8.1 Customisation of Models

8.1.1 Tweaking Architecture and Re-Training

The YOLO architecture is comprised of model schematics with predefined layers. However, functionality exists within machine learning frameworks to create custom model schematics that use modified neural networks and a developer-defined layer structure. This allows much greater control and flexibility since these models can be tweaked and tuned in order to optimise their use for a specified problem. Custom model schematics can also be used to maximise performance for a wider range of device hardware, since inference optimisation is in the hands of developers.

On the other hand, this could be time consuming and may end up increasing the resource insensitivity of the model or lead to undesired results. A sound knowledge of machine learning processes would be required in order to determine the best layer positioning or type. Training custom models may also require much larger datasets in order for an accurate trend assessment to be carried out.

8.1.2 Colour-Based Convolution

Railway architecture across the world varies significantly, and some of the patterns detected well on UK railway stations may not be effective in other countries e.g. Japan, where yellow lines are laid out differently. Alternative station layouts may make it harder for models to adapt to a wider range of surroundings.

The use of custom convolution layers for individual colour bands could make the models far more robust and able to adapt to different surroundings. This could also open up the potential to use this technology in other scenarios such as road line detection. Evidence shows that dataset upscaling generally leads to better results, and a wider range of input from different surroundings could further enhance the results.

8.2 Customisation of Filtering

Filtering techniques used in the second cycle present the use of Otsu’s thresholding via binarization of image data. However, this was shown to only be marginally more successful than the unfiltered set. Railway imagery is heavily dependent on colours and shading, which may have contributed to the lack of effectiveness in this type of filtering.

One suggestion is to customise the filtering techniques used within training images (and subsequently inference) to see if filter banding as opposed to binarization can allow for better results. Examples are shown in the figure below. Whilst Otsu uses binary imaging to determine a threshold point, simultaneous use of Otsu on threshold banding is also possible. Should the project have not been constrained by time, this, along with other filtering techniques, would have been explored.

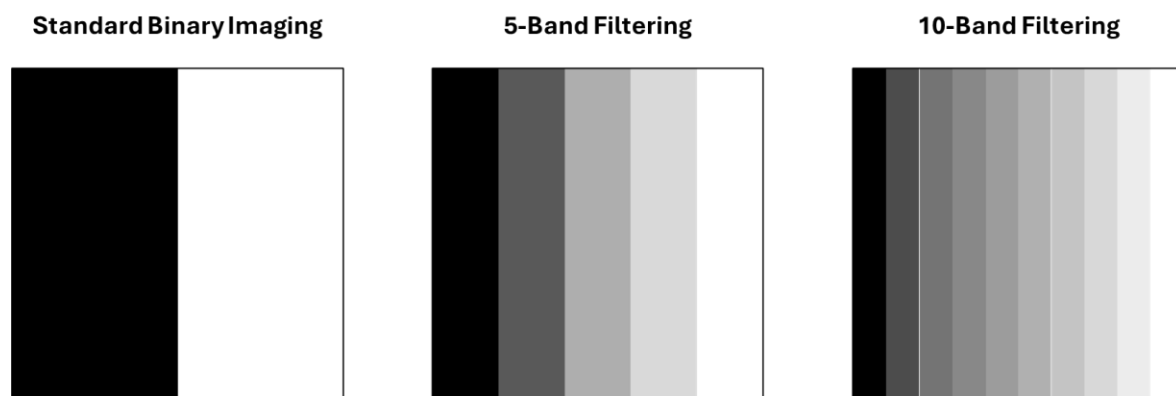


Figure 52 - Band-based Otsu filtering

8.3 Other Enhancements

Provided below are additional unique examples of enhancements that do not fit into the subsection categories above.

8.3.1 Using Multiple Data Streams

The system up to this point has zoned in on per-user accessibility enhancement. There is potential to utilise railway based hardware such as fixed cameras in order to provide the same information.

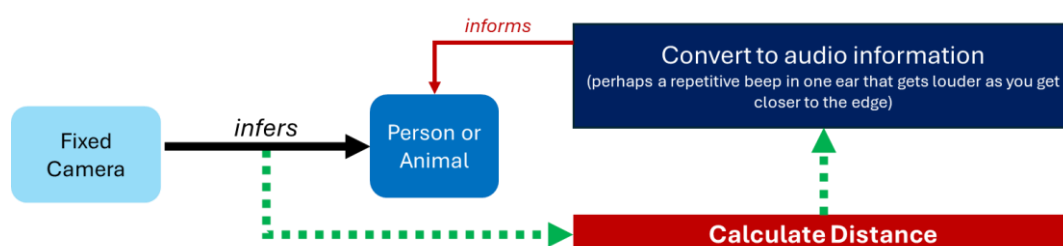


Figure 53 - Multiple Streams Proposal

This could involve upgrading the dataset to add a 'person' class for segmentation and then algorithmically calculate the distance between an individual and a platform edge. Users could then connect to a simpler frontend, perhaps on a smartphone, and audio information can be relayed in a similar manner.

8.3.2 Depth Perception for Enhancement of Results

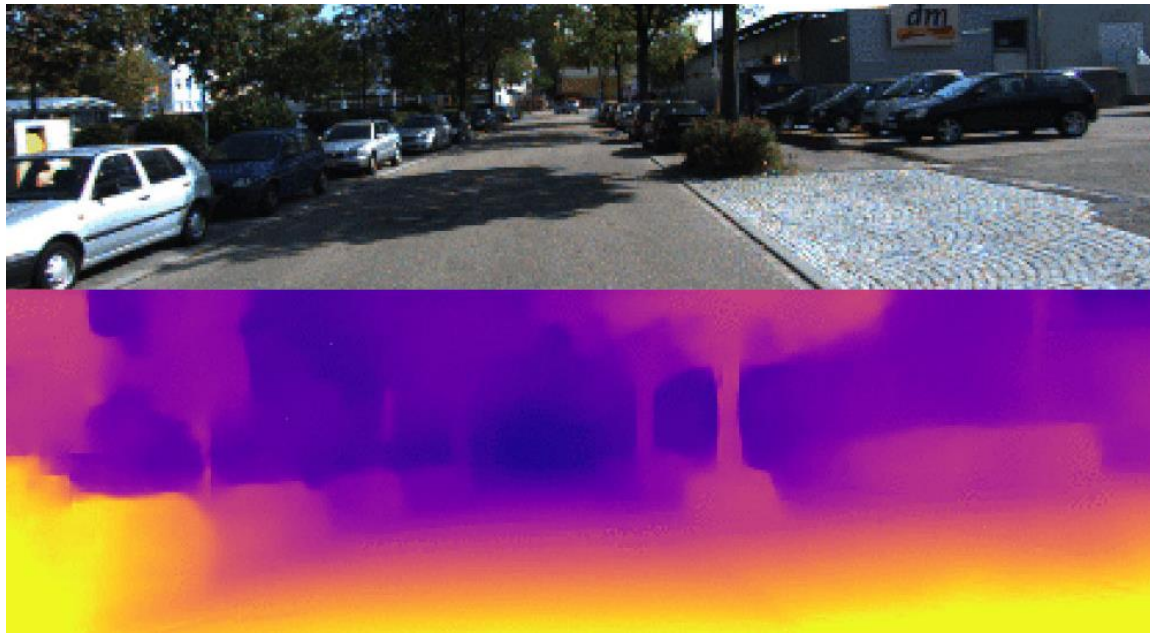


Figure 54 - Depth Perception in AI. Source image: (K-Tech, 2021)

Implementing a depth perception algorithm was considered at one point but dropped due to complexities in assigning a perception algorithm to such a wide range of camera architectures. Depth perception could allow for audio info to be depth-dependent, such as louder beeps for closer edges.

8.3.3 Focus Reversal: Visual Information for Audially Impaired Passengers

This proposal is of great interest. Whilst audially impaired passengers are unlikely to struggle with navigation of railway stations, safety risks are still posed to this group by difficulties hearing passing trains, or when announcements are made audially by train operators without a visual counterpart (this is particularly common to inform that a train is delayed). This system could use the visual techniques of this project to detect trains at various points along a railway track.

9 Conclusion

This project has endeavoured to critically assess and enhance railway accessibility standards for a visually impaired userbase. To summarise: all project objectives were met, and extensive research has been undertaken to provide knowledge into the field.

The table below lists the project objectives and justifies the reasoning behind the achievement of each.

Objective 1 To design, develop and implement a system that assists visually impaired users with navigation of railway stations.	This objective has been met. The final results of the project include a fully functional system that achieves the accessibility goals set out.
Objective 2 To expand upon shortfalls and successes in background research, analysing both design aspects and similar existing technologies.	This objective has been met. The project has extensively analysed previous research and technologies and provided insight into the successes and failures found.
Objective 3 To follow a software development methodology throughout the lifecycle of project.	This objective has been met. The Hybrid Methodology has been utilised throughout the development process.
Objective 4 To create a scalable proof-of-concept implementation that can be expanded upon through an extensive insight into future work and evaluation.	This objective has been met. Extensive future work documentation is provided as well as the system's limitations and justifications.
Objective 5 To evaluate feedback from testing stages and adjust the requirements, as necessary in order to refine the system to ensure elevated levels of precision and accuracy are complied with as far as practicably possible.	This objective has been met. Testing stages were conducted with extensive feedback provided, with requirements adjusted as necessary.
Objective 6 To design with a focus on continual improvement in attempts to support as broad a range of specialised requirements as possible, with a focus on users with partial sight.	This objective has been met. A feasible user scope has been provided in order to justify the need for limitations.

Table 11 - Project Objective Conclusions.

10 References

AIORTC, 2021. *AIORTC on GitHub*. [Online]

Available at: <https://github.com/aiortc/aiortc>

Amazon, 2024. *Amazon Machine Learning: Developer Guide*. [Online]

Available at: <https://docs.aws.amazon.com/pdfs/machine-learning/latest/dg/machinelearning-dg.pdf#model-fit-underfitting-vs-overfitting>

[Accessed 16 March 2024].

Data Science Courses | DataScientest, 2023. *Managing Unbalanced Classification Problems - Part 1*. [Online]

[Online]

Available at: <https://datascientest.com/en/management-of-unbalanced-classification-problems>

[Accessed 20 March 2024].

Dixon, S., Smith, C. & Touchet, A., 2018. *The Disability Perception Gap: Policy Report*. [Online]

Available at: <https://www.scope.org.uk/scope/media/files/campaigns/disability-perception-gap-report.pdf>

[Accessed 8 March 2024].

DPTAC, 2019. *Working towards a fully accessible railway: a frame of reference for the Williams Review*. [Online]

[Online]

Available at: <https://abcommuters.files.wordpress.com/2022/05/190508-working-towards-a-fully-accessible-railway-a-frame-of-reference-for-the-williams-review-amended-final-docx.pdf>

[Accessed 14 March 2024].

Garaj, V., Hunaiti, Z. & Balachandran, W., 2007. The effects of video image frame rate on the environmental hazards recognition performance in using remote vision to navigate visually impaired pedestrians.

Google Developers, 2019. *Classification: Precision and Recall | Machine Learning Crash Course*. [Online]

Available at: <https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall>

[Accessed 16 March 2024].

Heller, M., 2022. *What is CUDA? Parallel programming for GPUs*. [Online]

Available at: <https://www.infoworld.com/article/3299703/what-is-cuda-parallel-programming-for-gpus.html>

[Accessed 20 March 2024].

Jain, V., 2019. *Everything you need to know about 'Activation Functions' in Deep learning models*. [Online]

Available at: <https://towardsdatascience.com/everything-you-need-to-know-about-activation-functions-in-deep-learning-models-84ba9f82c253>

[Accessed 16 March 2023].

Keates, S., 2007. *Designing for accessibility : A Business Guide to Countering Design Exclusion*. Mahwah, N.J: Lawrence Erlbaum Associates.

K-Tech, 2021. *Understanding Depth Perception in Computer Vision*. [Online]

Available at: <https://coe-dsai.nasscom.in/understanding-depth-perception-in-computer-vision/>

[Accessed 22 March 2024].

Kundu, R., 2022. *F1 Score in Machine Learning: Intro & Calculation*. [Online]
Available at: <https://www.v7labs.com/blog/f1-score-guide>
[Accessed 16 March 2024].

Lucidchart.com, n.d. *Is Agile-Waterfall Hybrid Right for Your Team?*. [Online]
Available at: <https://www.lucidchart.com/blog/is-agile-waterfall-hybrid-right-for-your-team>
[Accessed 14 March 2024].

Moin, M., 2023. *Object Detection: A journey from R-CNN to Mask R-CNN and YOLO*. [Online]
Available at: <https://medium.com/augmented-startups/object-detection-a-journey-from-r-cnn-to-mask-r-cnn-and-yolo-698ba097d490>
[Accessed 8 March 2024].

Musstafa, 2022. *Optimizers in Deep Learning*. [Online]
Available at: <https://musstafa0804.medium.com/optimizers-in-deep-learning-7bf81fed78a0>
[Accessed March 16 2024].

Nielsen, J., 1994. *10 Heuristics for User Interface Design*. [Online]
Available at: <https://www.nngroup.com/articles/ten-usability-heuristics>
[Accessed 8 March 2024].

Nyuytiybiy, K., 2021. *Parameters and Hyperparameters in Machine Learning and Deep Learning*. [Online]
Available at: <https://towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac>
[Accessed March 16 2024].

Office of Rail & Road, 2019. *Passenger rail usage*. [Online]
Available at: <https://dataportal.orr.gov.uk/statistics/usage/passenger-rail-usage/>

Office of Rail & Road, 2023. *Experiences of Passenger Assist Report 2022-2023*. [Online]
Available at: <https://www.orr.gov.uk/sites/default/files/2023-07/2022-2023-passenger-assist-report-by-mel.pdf>
[Accessed 10 March 2024].

Olegkhomenko, G., 2020. *pexels-downloader on GitHub*. [Online]
Available at: <https://github.com/olegkhomenko/pexels-downloader>
[Accessed 10 February 2024].

Park, K., Goh, T. & So, H.-J., 2015. Toward accessible mobile application design: Developing Mobile Application Accessibility Guidelines for People With Visual Impairment. *Proceedings of HCI KOREA*.

Pexels, n.d. *Pexels.com - The best free stock photos, royalty free images & videos shared by creators*, s.l.: s.n.

PyTorch Forums, 2024. *PyTorch for Cuda 12*. [Online]
Available at: <https://discuss.pytorch.org/t/pytorch-for-cuda-12/169447>
[Accessed 20 March 2024].

Redmon, J. D. S. G. R. a. F. A., 2016. You Only Look Once: Unified, Real-Time Object Detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

RMT, 2021. *Rebuilding Passenger Confidence*, United Kingdom: RMT Media & Publications.

Roboflow, n.d. *How to Train YOLOv8 Object Detection on a Custom Dataset*. [Online]
Available at: <https://github.com/roboflow/notebooks/train-yolov8-object-detection-on-custom-dataset.ipynb>

SabrePC, 2023. *Epochs, Batch Size, Iterations - How they are Important..* [Online]
Available at: <https://www.sabrepc.com/blog/Deep-Learning-and-AI/Epochs-Batch-Size-Iterations>
[Accessed 16 March 2024].

Sellers, A., n.d. *How to Get an SSL Certificate*. [Online]
Available at: <https://blog.hubspot.com/website/best-free-ssl-certificate-source>
[Accessed 21 March 2024].

Sredojev, B., Samardzika, D. & Posarac, D., 2015. *WebRTC technology overview and signaling solution design and implementation*. [Online]
Available at: <https://doi.org/10.1109/mipro.2015.7160422>
[Accessed 18 March 2024].

Taller, H., 2021. *When, Why, and How to Use the Agile-waterfall Hybrid Model*. [Online]
Available at: <https://www.ptc.com/en/blogs/alm/when-why-how-to-use-the-agile-waterfall-hybrid-model>
[Accessed 14 March 2024].

Transport for London, 2024. *Transport Accessibility*. [Online]
Available at: <https://tfl.gov.uk/transport-accessibility/>
[Accessed 8 March 2024].

Ultralytics, 2023. *Loss Functions*. [Online]
Available at: <https://docs.ultralytics.com/reference/utils/loss/>
[Accessed 17 March 2024].

Usability.gov, 2019. *System Usability Scale (SUS)*. [Online]
Available at: <https://www.usability.gov/how-to-and-tools/methods/system-usability-scale.html>
[Accessed 22 March 2024].

Washington County, OR, n.d. *Americans with Disabilities Act and Pedestrian Accommodations*. [Online]
Available at: <https://www.washingtoncountyor.gov/traffic-road-engineering/ada-and-pedestrian-accommodations>
[Accessed 8 March 2024].

Appendix A : Reflection

A.1 Project Reflection

This project has been very successful in both producing a functional output and a wide range of further avenues for continued development. It is very clear to me in retrospect that the right choice of methodology was made and adhered to, which was incredibly beneficial in allowing for the documentation of results as development progressed, something which was of incredible importance.

A.1.1 What was learned through project development?

The development process has been an incredible learning experience. I have thoroughly enjoyed every stage of research, from the relevant readings and background analysis all the way to the testing and showcasing of the system in action. I am very satisfied with the way the project has evolved from the unclear synopsis of what I wanted to achieve in the beginning towards a full trajectory of what needs to be done.

I now understand the importance of methodology use and its benefits in a practical context, something which I had previously cast doubt on as I typically undertake hobby projects by jumping right into coding without a clear vision as to what I wanted to achieve. In addition to this, the research aspects of the project have been very enjoyable and have given me transferrable skills in literature analysis.

A.2 Personal Reflection

I have been enthusiastic about railways for many years and this project allowed me to strike a fine balance between stepping out of my comfort zone – delving into new technologies I had never used before – and applying these newfound concepts and tests to a familiar setting. I feel as if the project has targeted my interests in this way and this has allowed me to apply a true enthusiasm and passion that I may not have benefitted from otherwise.

A.2.1 What could be improved on?

Although the entire project and related fields is of interest to me, I do feel as if I have spent too long focusing on the parts of the project that were most intriguing to me such as the neural network development – and not enough time emphasising other areas such as the web based functionality, the system access focus and usability requirements. Time management was also an issue; I procrastinated the audio implementation and third prototype right until the day

before the project deadline! I also submitted the BREO application far too late which delayed my testing research.

I most certainly should have started my dissertation earlier and definitely conducted the background research much earlier. I could have organised my data more efficiently; there were times when I did not save trained model results and had to redo these again to collect graphs. I would have also liked to go further and experiment with voice activation, TTS, and AI-based voice training, although I never managed this in part due to inefficient time management.

A.2.2 How would the project be done again differently?

If I were to do the project again, I would make the following changes:

- 1) I would formalise what I wanted to achieve and start earlier.**
- 2) I would refine my testing process and attempt more rigorous testing for a broader range of results.**
- 3) I would provide a more detailed evaluation.**
- 4) I would experiment with a wider range of computer vision and artificial intelligence techniques.**

Appendix B : Ethics Confirmation

B.1 Ethics Confirmation

Ethical approval was granted on 31st January 2024 for this study to be carried out.

The ethics approval letter is shown on the next page.

31 January 2024

LETTER OF APPROVAL

APPROVAL HAS BEEN GRANTED FOR THIS STUDY TO BE CARRIED OUT BETWEEN 12/02/2024 AND 22/03/2024

Applicant (s): Mr Saen Kazak

Project Title: Utilising artificial intelligence to provide accessibility enhancements at railway stations for passengers with impaired vision

Reference: 46644-LR-Jan/2024- 49547-2

Dear Mr Saen Kazak

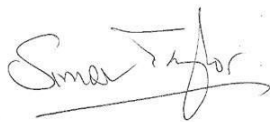
The Research Ethics Committee has considered the above application recently submitted by you.

The Chair, acting under delegated authority has agreed that there is no objection on ethical grounds to the proposed study. Approval is given on the understanding that the conditions of approval set out below are followed:

- **The agreed protocol must be followed. Any changes to the protocol will require prior approval from the Committee by way of an application for an amendment.**

Please note that:

- Research Participant Information Sheets and (where relevant) flyers, posters, and consent forms should include a clear statement that research ethics approval has been obtained from the relevant Research Ethics Committee.
- The Research Participant Information Sheets should include a clear statement that queries should be directed, in the first instance, to the Supervisor (where relevant), or the researcher. Complaints, on the other hand, should be directed, in the first instance, to the Chair of the relevant Research Ethics Committee.
- Approval to proceed with the study is granted subject to any conditions that may appear above.
- The Research Ethics Committee reserves the right to sample and review documentation, including raw data, relevant to the study.
- If your project has been approved to run for a duration longer than 12 months, you will be required to submit an annual progress report to the Research Ethics Committee. You will be contacted about submission of this report before it becomes due.
- You may not undertake any research activity if you are not a registered student of Brunel University or if you cease to become registered, including abeyance or temporary withdrawal. As a deregistered student you would not be insured to undertake research activity. Research activity includes the recruitment of participants, undertaking consent procedures and collection of data. Breach of this requirement constitutes research misconduct and is a disciplinary offence.



Professor Simon Taylor

Chair of the College of Engineering, Design and Physical Sciences Research Ethics Committee

Brunel University London

Appendix C Other Material

C.1 Nielsen's 10 Usability Heuristics

Sourced and quoted directly from (Nielsen, 1994)

1) **Visibility of System Status**

The design should always keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.

2) **Match Between the System and the Real World**

The design should speak the users' language. Use words, phrases, and concepts familiar to the user, rather than internal jargon. Follow real-world conventions, making information appear in a natural and logical order.

3) **User Control and Freedom**

Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action without having to go through an extended process.

4) **Consistency and Standards**

Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform and industry conventions.

5) **Error Prevention**

Good error messages are important, but the best designs carefully prevent problems from occurring in the first place. Either eliminate error-prone conditions, or check for them and present users with a confirmation option before they commit to the action.

6) **Recognition Rather than Recall**

Minimize the user's memory load by making elements, actions, and options visible. The user should not have to remember information from one part of the interface to another. Information required to use the design (e.g. field labels or menu items) should be visible or easily retrievable when needed.

7) **Flexibility and Efficiency of Use**

Shortcuts — hidden from novice users — may speed up the interaction for the expert user so that the design can cater to both inexperienced and experienced users. Allow

users to tailor frequent actions.

8) Aesthetic and Minimalist Design

Interfaces should not contain information that is irrelevant or rarely needed. Every extra unit of information in an interface competes with the relevant units of information and diminishes their relative visibility.

9) Help Users Recognize, Diagnose, and Recover from Errors

Error messages should be expressed in plain language (no error codes), precisely indicate the problem, and constructively suggest a solution.

10) Help and Documentation

It's best if the system doesn't need any additional explanation. However, it may be necessary to provide documentation to help users understand how to complete their tasks.

C.2 Design Heuristics from Study Referenced in Section 2.1.3

1) **Substitutive text: Provide a substitutive text to all User Interface (UI) components**

There are many UI components in a mobile application. VoiceOver can read text without additional programming settings in a simple text case. In case of buttons, however, VoiceOver cannot read them unless designers provide a substitutive text.

2) **Object with focus: Every object can get a focus**

The focus is the most important concept to people with visual impairment. The focus acts as a computer mouse. It is a very natural concept to point everything using a mouse.

3) **Logical focus flow: Provide a logical focus flow**

People with visual impairment receive information in a focused order. The focus sequence should be designed with consideration of information completeness.

4) **OS accessibility support: Use accessibility functions and properties provided by OS**

It is desirable that developers and designers use native UI components provided by OS because it is easy to provide compatibility with other assistive devices.

5) **Press action support: All controls should be handled with press action**

In VoiceOver, the main selection action is to press (one-click, tap). Designer should provide control with a press action.

6) **Simple structure: Provide a simple structure and under 30 items at one page**

If there are too many items in one page, people with visual impairment have trouble with browsing. They have to listen to substitutive texts of all items to search for information. Simple structure also helps focus configuration.

7) **Substitutive action: Provide a substitutive button on overlapped action**

In VoiceOver, some gestures are preset as default. Designers should design UI considering these default actions. For example, the gesture of flick right on VoiceOver is overlapped to the function of opening sub-menu on KakaoTalk. They have to prohibit duplicated UI with VoiceOver gestures.

8) **Notice function: Provide various notice methods as much as possible**

Every notice information in screen should have more than two methods, and exploit various sensory functions.

9) **Consistent UI: Provide consistent placement for UI components**

To increase learnability, UI component placement should be consistent.

(Garaj, et al., 2007)

C.3 Waterfall and Agile Diagrams

This section complements Chapter 3 by providing additional diagrams for Waterfall and agile processes.

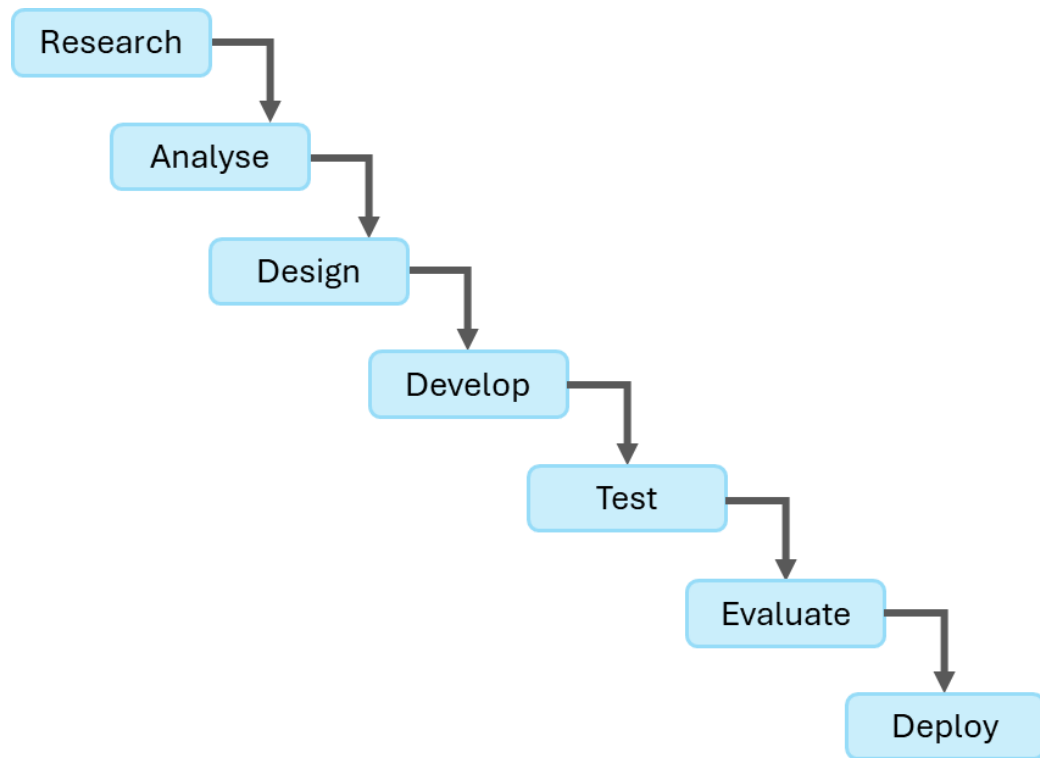


Figure 55 - Waterfall Model

The waterfall model provides a linear approach during development. It is generally used for rigid software development with unchanging requirements and/or a strict regulation in play.

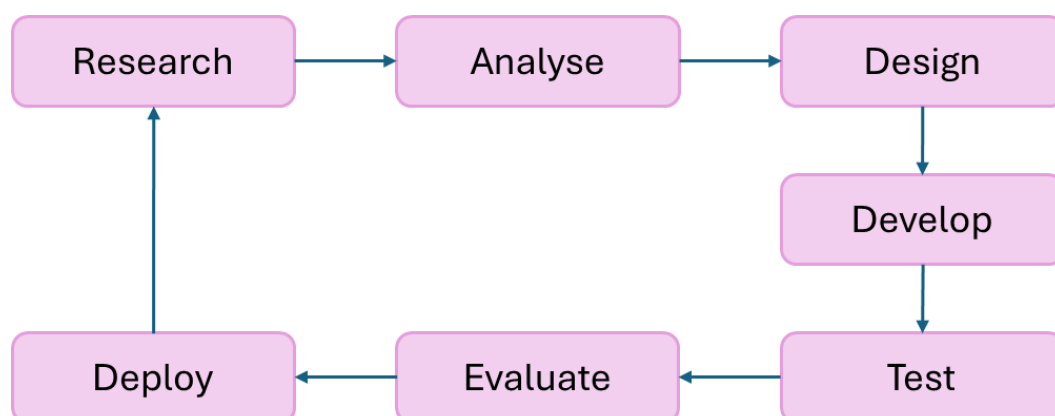


Figure 56 - Agile Model

The agile model is much more widely used and follows iterative approaches for all development cycles, typically in the form of sprints. It allows for continual refinement when user requirements are flexible.

C.4 The YOLO Algorithm

It is vital that a chosen inference framework supports both accuracy and speed since both are crucial to our use case. YOLO (You Only Look Once) is an object detection algorithm (with segmentation support added in later versions) with a heavy emphasis on both accuracy and speed, which has been said to have revolutionized this area of computer vision when it was first introduced (Redmon, 2016). Unlike previous algorithms, which made use of a two-stage R-CNN model, YOLO treats the object detection problem as a single linear regression problem requiring only a single CNN. Put simply, YOLO looks at the entire image as a single problem instead of regionalizing (Moin, 2023).

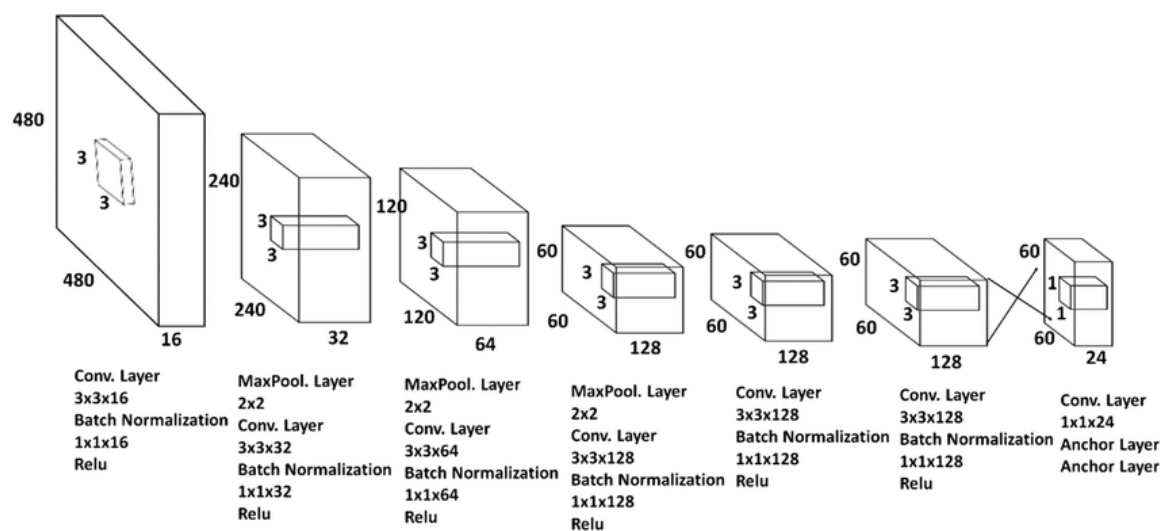


Figure 57 - YOLO Model Architecture. Source: YOLO

The YOLO algorithm has a total of 24 convolutional layers and ReLU activation functions, with the final layer being a linear activation function as shown in Figure 57. The instance segmentation capabilities of the model added in YOLOv5 made minor modifications to the architecture via the addition of prototype masking capabilities.

C.5 Pre-Training Preparations: Image Collection and Scraping

Pexels is an open-source image platform providing royalty free stock photos. An analysis of the Pexels attribution licence suggests that it is permitted to use this data within the project (Pexels, n.d.).

For automation, the GitHub repository ‘pexels-downloader’ (GitHub: Olegkhomenko, 2020) was cloned and used. The script allows for a search term input as a parameter and returns scraped results from Pexels, with terms used including ‘train’, ‘railway’, and ‘platform’.

When the script is re-run, it will ignore previous results and continue scraping from the last point it was run to.

The scraping tool was successful overall in its task. However, it was apparent that the relevance of returned images began to decrease after around 200 images were scraped for each search term.

It was not deemed time-efficient to implement a more effective way to scrape the data. In hindsight, this can be attributed to the fact that this was the first step of project implementation and was treated as a precursor to the ‘more interesting’ project aspects such as running model training. As a result, less time was spent on this as opposed to annotation and training.

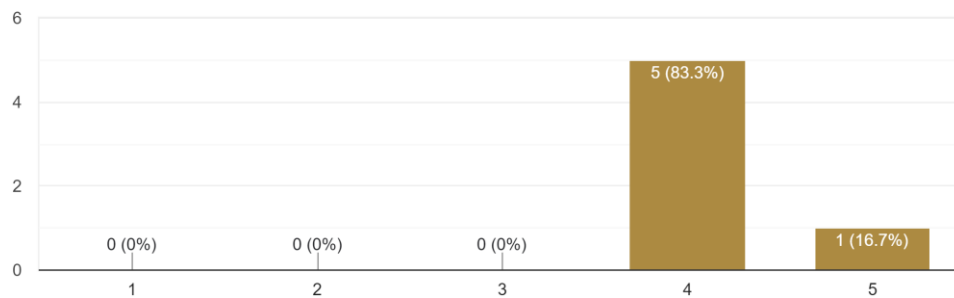
C.6 Participant Testing Data

C.6.1 Usability Survey: All Question Results

The results of all SUS survey questions are provided here as additional info. Questions are ranked on a scale of 1 – Strongly Disagree to 5 – Strongly Agree.

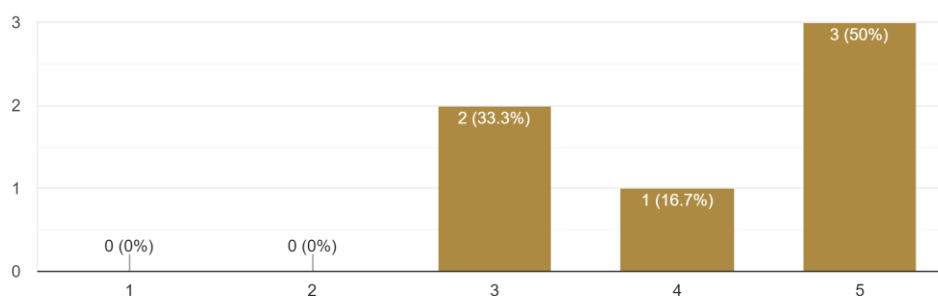
I thought the software was easy to use.

6 responses



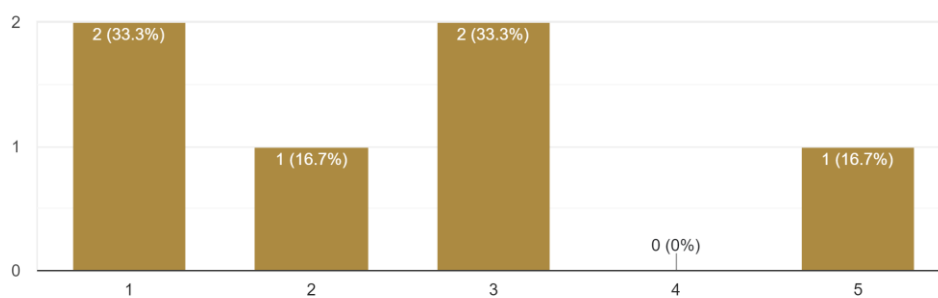
I felt very confident using the software.

6 responses



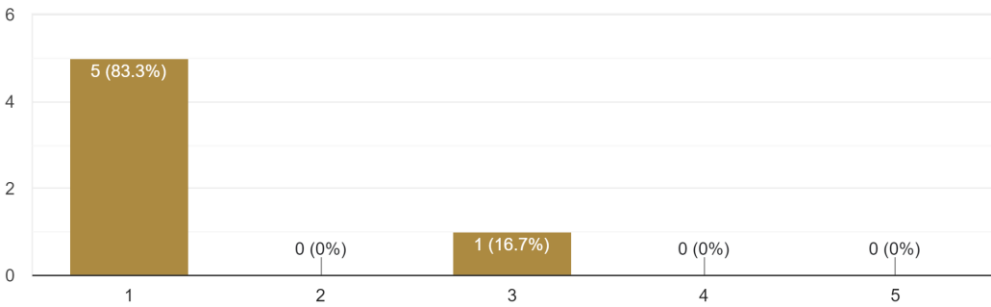
I would need the support of a technical person to use the software.

6 responses



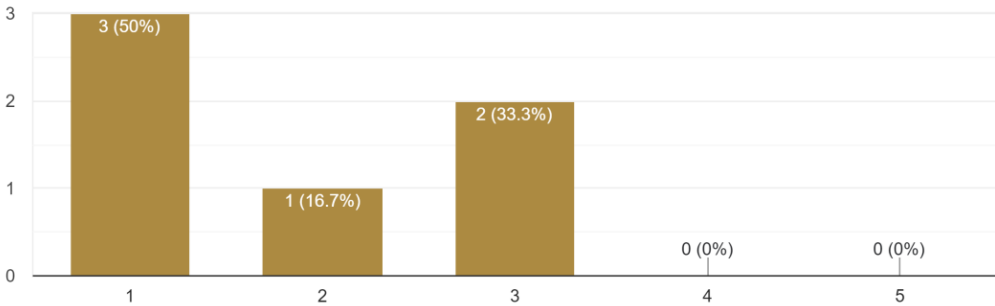
I found the software unnecessarily complex.

6 responses



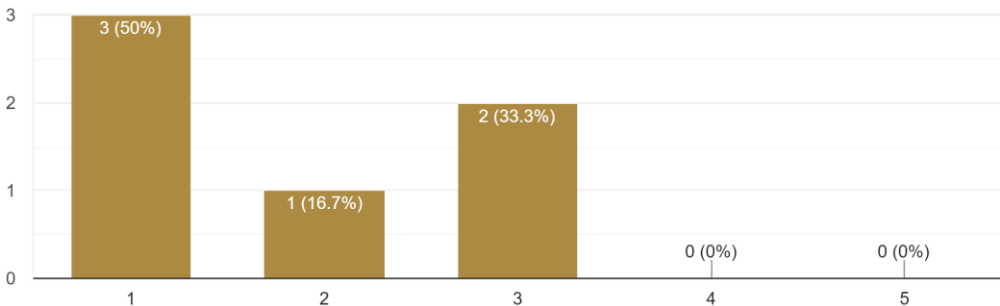
I thought there was too much inconsistency in the software.

6 responses



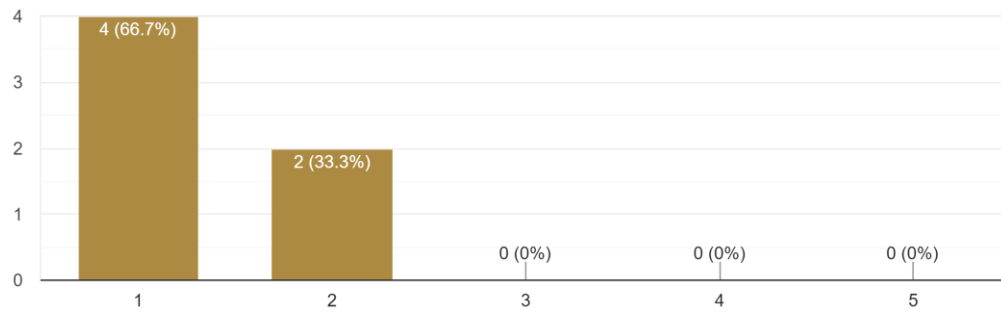
I thought there was too much inconsistency in the software.

6 responses



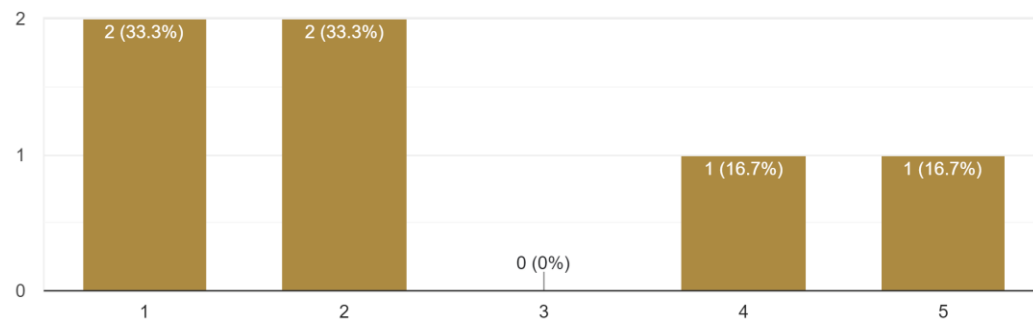
I found the software very cumbersome to use.

6 responses



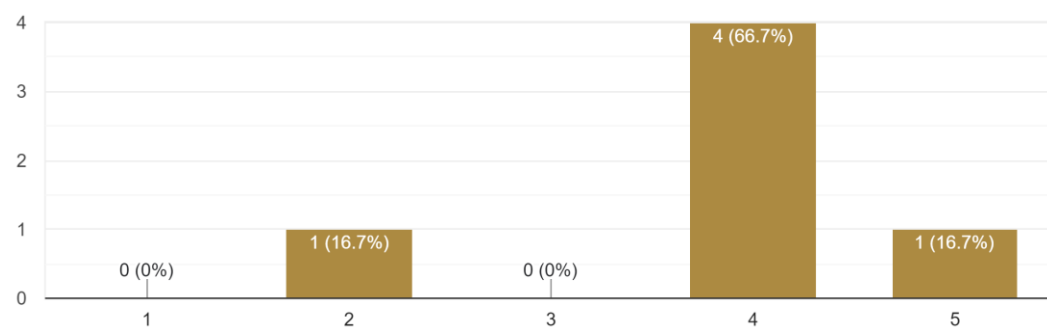
I found the software very rigid and inflexible to use.

6 responses



I felt very satisfied with the software.

6 responses



C.6.2 Comment Feedback

This feedback was provided by testing participants that filled out the questionnaire.

Comment Feedback Data
Q: Is there anything else you would like to tell us about your experience testing this software?
<i>A: Can improve its tracking, as the trackers often highlighted the monitor as part of the tracks. Found that this could be very revolutionary for the future of accessibility and people whom have disability to travel on their own. (Samsung Galaxy S20)</i>
<i>A: I would be concerned on how this would be used in the real world -- e.g. how would some visually impaired people know that the camera is pointing in the right direction/that it's properly capturing the relevant surroundings. (Samsung Galaxy A33)</i>
<i>A: Although there were no errors, it only let me use my front camera rather my back camera. There was no option to change this however, it still worked with the front camera! (iPhone 14 Pro Max)</i>
<i>A: Establish a consistent frame rate and test the software on more smartphone models. (Participant 13)</i>

Table 12 - Comment Feedback

C.7 Model Training – Additional Data

Training databases for machine learning will typically be split into training data, testing data, and validation data (discussed in C.7.4) with defined proportions for each. Roboflow allows for automatic and random splitting of a dataset into these categories; Figure 19 in the main document shows how each annotated image above will be assigned one of these.

C.7.1 Definitions: Inference

Inference simply refers to the model results being tested on a single frame. Within the main document, references to model inference, inferring the model or inferencing are defined as such.

C.7.2 Definitions: Train and Test Sets

The **‘train’** set is a core set of examples used to ‘learn’ the classification of image segments. Put simply, this set is the one that a model will examine to learn the best variable combinations, such as pixel values, to detect any trends or patterns that allow it to become better at ‘fitting’ the dataset.

The **‘test’** set, on the other hand is not used to train the model, but instead provides independent unseen images for evaluation after training. This is necessary to detect **‘overfitting’**, which is when a model stops performing well on new data on account of becoming too accustomed to the training data.

C.7.3 Overfitting

Should a model score well in training, but badly on the test set, it is likely that the model has fit too well and cannot generalise to new data.

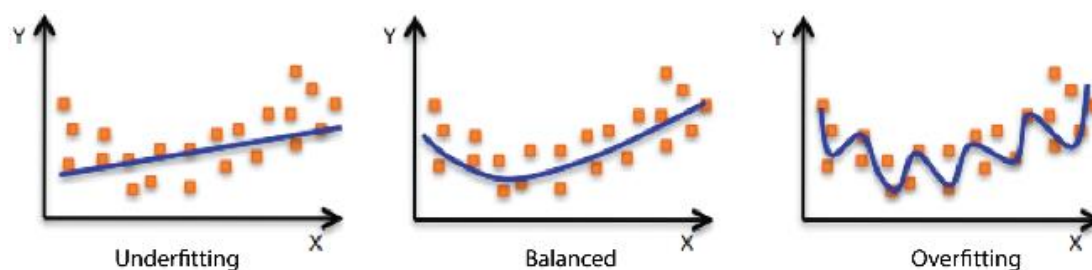


Figure 58 - Underfitting vs Overfitting (Amazon, 2024, p. 33)

Figure 58 shows the capturing of the relationship (blue) between the input values (X) and the output values (Y) in various scenarios. overfitting the model is unable to generalise the relationship to the data. For a balanced model, the relationship will need to follow the course of the data. Since we are following an iterative methodology for implementation, train/test/valid splits were altered in order to find the best settings. For the first collection of 115 images, a split

of 81/24/10 was used. It is important to note that our project makes use of a **supervised learning** approach with known outputs i.e. classification, where final outputs values are predetermined (unlike clustering).

C.7.4 Hyperparameters

Machine learning models train data according to specified parameter values, which can be ‘tuned’ to tweak a broad range of variables. Hyperparameters are known as such due to being “top-level parameters” (Nyuytiymbiy, 2021), set before the model begins training. Table 13 shows common hyperparameters tuned during model training.

Name	Description	Example Values
Train/Test Ratio	The ratio of images that will be used in the train and test sets.	70/30
Optimiser	Machine learning models will use an optimiser algorithm to adjust the learning rates of a model. Optimisers are dependent on the learnable parameters of a model including the weights and biases (Musstafa, 2022).	Gradient descent; Stochastic gradient descent; Adaptive Moment Estimation (Adam)
Activation Function	Activation functions are used to convert the output values of a ‘dense’ layer into appropriate input values for the next layer. This allows us to restrict certain outputs to avoid computational issues. (Jain, 2019).	ReLU; Sigmoid
No. of Epochs	An epoch in machine learning refers to one cycle of all the training data through the neural network. The optimal number of epochs will typically not be too many that the model overfits but include enough cycles so that the model is able to learn the patterns in the data (SabrePC, 2023).	10, 20, 100, etc.
Batch Size	Batch size refers to the number of training inputs that are processed together; known as a single batch. Setting a batch size too high can speed up training but may cause a model to overfit. Batch sizes that are too small are more computationally expensive and use more processing power but may lead to greater accuracy (SabrePC, 2023).	16, 32, 64, 128 etc.
No. of Iterations	Defined as the number of batches processed in each epoch (SabrePC, 2023).	50, 100 etc.

Table 13 - Common Hyperparameters used in Machine Learning

C.7.5 Definitions: Validation Set

The ‘**validation**’ set works similarly to the ‘test’ set, with the exception that the images are evaluated between epochs as opposed to end evaluation. This allows us to determine per-epoch model performance in order to analyse the data and verify whether hyperparameter adjustment is needed and detect instances of overfitting or falling precision/recall rates.

C.7.6 Software Compatibilities

This section provides the software versions deployed on the testing computers. On the testing computers, the following software combination was compatible when an NVIDIA RTX 3070Ti GPU was used.

Python Modules

aiohttp	3.9.3
aioice	0.9.0
aioresponses	0.7.6
aiortc	1.7.0
aiosignal	1.3.1
inference	0.9.14
inference-cli	0.9.14
inference-gpu	0.9.14
inference-sdk	0.9.13
numpy	1.26.4
nvidia-cublas-cu11	11.11.3.6
nvidia-cuda-nvrtc-cu11	11.8.89
onnxruntime-gpu	1.15.1
openai	1.12.0
opencv-python	4.8.0.76
opencv-python-headless	4.8.0.74
requests	2.31.0
requests-toolbelt	1.0.0
roboflow	1.1.19
torch	2.2.1+cu118
torchaudio	2.2.1+cu118
torchvision	0.17.1+cu118
webencodings	0.5.1
websocket-client	1.7.0
websockets	12

Other Software

NVIDIA CUDA	v11.8
NVIDIA CuDNN	v8.6

Table 14 - Software Versions

C.7.7 Adaptive Median Thresholding

Other thresholding methods also exist for images, which have proved to be useful during implementation. Unlike Otsu and Binary techniques, which apply global thresholding to an image, adaptive median thresholding divides an image into smaller segments or 'neighbourhoods.' The median pixel density is then calculated for each neighbourhood for application, with the threshold 'adapting' based on the image surroundings, hence the name. This is beneficial for images that may have unconventional illumination. Within the context of railways, adaptive median is incredibly useful in low light environments.

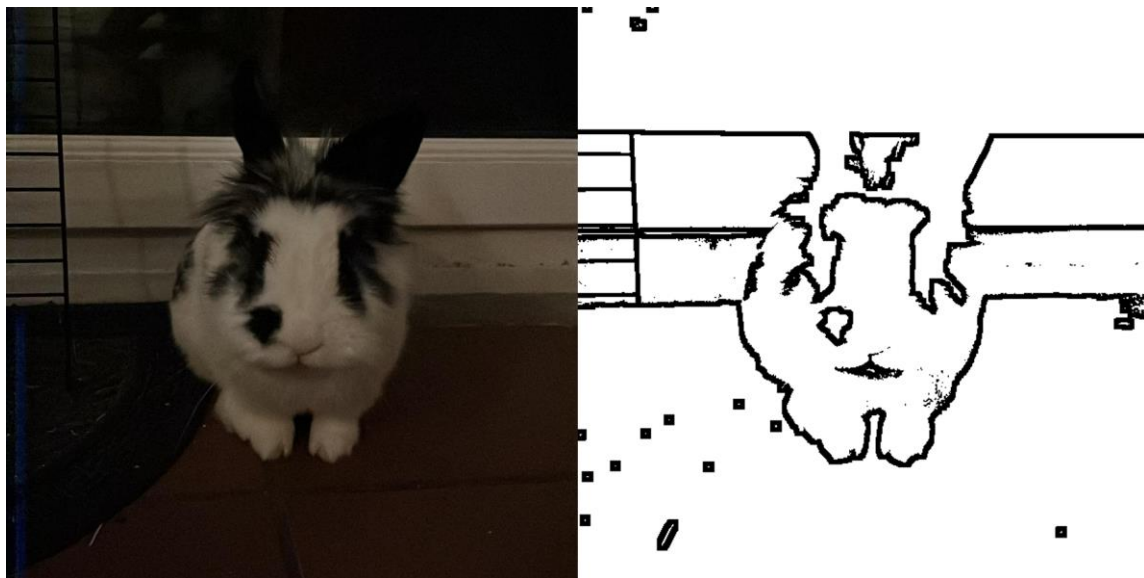


Figure 59 - Adaptive Median works well in low light

C.8 The Project Code

This section provides the raw code for both scripts used, annotated with comments throughout. This consists of a Python inferencing backend and a JavaScript client script. I have also provided the HTML skeleton code for the UI and a directory listing of the server folder (as the scripts reference various files within this folder).

C.8.1 JavaScript Client Code

```
// Initialise our peer connection

var pc = null;

var dc = null,
    dcInterval = null;

function createPeerConnection() {
    var config = {

    };

    pc = new RTCPeerConnection(config);
    // Listen for track events

    pc.addEventListener('track', (evt) => {
        if (evt.track.kind == 'video')
            document.getElementById('video').srcObject =
evt.streams[0];
        else
            document.getElementById('audio').srcObject =
evt.streams[0];
    });

    return pc;
}

// The handshaking procedure done when connecting a new client
function negotiate() {
    return pc.createOffer().then((offer) => {
        return pc.setLocalDescription(offer);
    }).then(() => {

        return new Promise((resolve) => {
            if (pc.iceGatheringState === 'complete') {
                resolve();
            } else {
                function checkState() {
                    if (pc.iceGatheringState === 'complete') {

pc.removeEventListener('icegatheringstatechange', checkState);
```



```

        resolve();
    }
}
pc.addEventListener('icegatheringstatechange',
checkState);
});
}).then(() => {
    var offer = pc.localDescription;
    var codec;

    codec = "H264";

    return fetch('/offer', {
        body: JSON.stringify({
            sdp: offer.sdp,
            type: offer.type,
            video_transform: "1024x1024",

        }),
        headers: {
            'Content-Type': 'application/json'
        },
        method: 'POST'
    });
}).then((response) => {
    return response.json();
}).then((answer) => {
    return pc.setRemoteDescription(answer);
}).catch((e) => {
    alert(e);
});
}
error = "";

function start() {
    pc = createPeerConnection();

    const constraints = {
        audio: false,
        video: false
    };
    // Create the data channel - we access this in the python script

    dc = pc.createDataChannel('InferenceResult');
    dc.addEventListener('close', () => {
        clearInterval(dcInterval);
    });

    dc.addEventListener('open', () => {
        console.log("Channel opened")
    });

    dc.addEventListener('message', (evt) => {

```

```
    if (evt.data === 'setGlobal') {
      console.log("channel is global");
    }

    // when we receive these events from the python code, we
    hook into the HTML audio objects to play the file.
    if (evt.data === 'playLeft') {
      document.getElementById("detectLeft").play()
    }

    if (evt.data === 'playRight') {
      document.getElementById("detectRight").play()
    }

  });

  // This is where we try and find cameras on the client device
  navigator.mediaDevices.enumerateDevices()
    .then((devices) => {

      const videoDevices = devices.filter(device =>
device.kind === 'videoinput');
      if (videoDevices.length === 0) {
        throw new Error('No video input devices found');
      }

      const lastVideoDeviceId =
videoDevices[videoDevices.length - 1].deviceId;

      const videoConstraints = {}
      videoConstraints.facingMode = {
        ideal: "environment" // We want the back facing
'environment' camera
      }
      videoConstraints.deviceId = {
        ideal: lastVideoDeviceId
        // We'll take the last device ID because on most
devices this is the back camera if 'environment' fails.
      }

      const resolution = "1024x1024" // These custom
constraints can be changed and have been during participant testing.
      const dimensions = resolution.split('x');
      videoConstraints.height = {
        ideal: 640
      }
      videoConstraints.frameRate = {
        exact: 10
      }
      videoConstraints.width = {
        ideal: 640
      }

      constraints.video = Object.keys(videoConstraints).length
? videoConstraints : true;

```

```

        document.getElementById('media').style.display =
'block';
        return navigator.mediaDevices.getUserMedia(constraints);
        // Once constraints are set up, we can grab our media
device based on them.
    })
    .then((stream) => {
        stream.getTracks().forEach((track) => {
            pc.addTrack(track, stream);
        });
        return negotiate();
    })
    .catch((err) => {
        alert('Could not acquire media: ' + err);
        error = err
    });
}

function stop() {
    document.getElementById('stop').style.display = 'none';

    if (dc) {
        dc.close();
    }

    if (pc.getTransceivers) {
        pc.getTransceivers().forEach((transceiver) => {
            if (transceiver.stop) {
                transceiver.stop();
            }
        });
    }

    pc.getSenders().forEach((sender) => {
        sender.track.stop();
    });

    setTimeout(() => {
        pc.close();
    }, 500);
}

function sdpFilterCodec(kind, codec, realSdp) {
    var allowed = []
    var rtxRegex = new RegExp('a=fmtp:(\\d+) apt=(\\d+)\\r$');
    var codecRegex = new RegExp('a=rtpmap:([0-9]+) ' +
escapeRegExp(codec))
    var videoRegex = new RegExp('(m=' + kind + ' .*?) ( ([0-
9]+) ) *\\s*$')

    var lines = realSdp.split('\n');

    var isKind = false;
    for (var i = 0; i < lines.length; i++) {

```

```

    if (lines[i].startsWith('m=' + kind + ' ')) {
        isKind = true;
    } else if (lines[i].startsWith('m=')) {
        isKind = false;
    }

    if (isKind) {
        var match = lines[i].match(codecRegex);
        if (match) {
            allowed.push(parseInt(match[1]));
        }

        match = lines[i].match(rtxRegex);
        if (match && allowed.includes(parseInt(match[2]))) {
            allowed.push(parseInt(match[1]));
        }
    }
}

var skipRegex = 'a=(fmt|rtcp-fb|rtptime):([0-9]+)';
var sdp = '';

isKind = false;
for (var i = 0; i < lines.length; i++) {
    if (lines[i].startsWith('m=' + kind + ' ')) {
        isKind = true;
    } else if (lines[i].startsWith('m=')) {
        isKind = false;
    }

    if (isKind) {
        var skipMatch = lines[i].match(skipRegex);
        if (skipMatch &&
!allowed.includes(parseInt(skipMatch[2]))) {
            continue;
        } else if (lines[i].match(videoRegex)) {
            sdp += lines[i].replace(videoRegex, '$1 ' +
allowed.join(' ')) + '\n';
        } else {
            sdp += lines[i] + '\n';
        }
    } else {
        sdp += lines[i] + '\n';
    }
}

return sdp;
}

function escapeRegExp(string) {
    return string.replace(/[\.*+?^$()|[\]\|\|/g, '\\$&');
}

// We use this to run through the device list to try and get the
last device ID
function getDeviceList() {

```

```
result = new Array();
navigator.mediaDevices.enumerateDevices()
  .then((devices) => {
    devices.forEach((device) => {
      if (device.kind == 'videoinput') {
        result.push(device.deviceId)
      }
    })
  })
  .then((devices) => {
    return r[devices.length - 1].deviceId;
  })
  .catch((err) => {});
}

start();
```

(Adapted from AIORTC, 2021)

C.8.2 Python Backend Code

```
# Import the modules we need

print("Importing OpenCV.")
import cv2
print("Loaded OpenCV.")

print("Importing Roboflow Inference...")
from inference.models.utils import get_roboflow_model
print("Loaded Roboflow Inference...")

print("Loading other modules...")
import numpy as np
import argparse
import asyncio
import json
import logging
import os
import ssl
import uuid
from aiohttp import web
from aiortc import MediaStreamTrack, RTCPeerConnection, RTCSessionDescription
from aiortc.contrib.media import MediaBlackhole, MediaPlayer, MediaRecorder, MediaRelay
from av import VideoFrame
print("Loaded other modules.")

ROOT = os.path.dirname(__file__)

# Allows us to log our RTCPeerConnection
logger = logging.getLogger("pc")
pcs = set()
relay = MediaRelay()

# Cache the model from Roboflow Servers

model_name = "railway-instancesegmentation"
model_version = "[version]"
dc = None

model = get_roboflow_model(
    model_id="{}/{}".format(model_name, model_version),
    api_key="[KEY]"
)

# We define a class that will take our frames and infer them on the recv() method
class VideoTransformTrack(MediaStreamTrack):
    kind = "video"
    def __init__(self, track, transform):
        super().__init__()
        self.track = track
        self.transform = transform

    async def recv(self):
        frame = await self.track.recv()
        w = frame.width
        h = frame.height
        img = frame.to_ndarray(format="bgr24") # Convert the raw data to inferrable N-
        Dimensional array

        # Infer the model
        result = model.infer(
            image=img,
            confidence=0.25,
            iou_threshold=0.5
        )

        # Grab response object
        response = result[0]

        image_width, image_height = img.shape[1], img.shape[0]

        # Create individual masks for tracks and platforms in the same format
        maskplatforms = np.zeros((image_height, image_width), dtype=np.uint8)
        masktracks = np.zeros((image_height, image_width), dtype=np.uint8)
```

```

# Initialise for 'Most Significant Platform/Track Detection'
mspd = [0,0]
mstd = [0,0]

# Dig through the response data
for prediction in response.predictions:

    if(prediction.class_name == "platform"):
        # CCP = Current Confidence, Platform.
        ccp = prediction.confidence
        if(ccp > mspd[1]): # If it's a more significant detection than the others in
the response object...
            mspd[0] = prediction
            mspd[1] = ccp
            # ...we'll set it to be the one we display

    #
    if(prediction.class_name == "track"):
        # CCP = Current Confidence, Track
        cct = prediction.confidence
        if(cct > mstd[1]): # If it's a more significant detection than the others in
the response object...
            mstd[0] = prediction
            mstd[1] = cct
            # ...we'll set it to be the one we display

if(mspd != [0,0]): # If we actually detected platforms in this frame...
    points = mspd[0].points # Get the most significant track's polygon points

    # Draw the shape on our platform mask
    pts = np.array([[point.x, point.y] for point in points], np.int32)
    pts = pts.reshape((-1, 1, 2))
    cv2.fillPoly(maskplatforms, [pts], 255)

if(mstd != [0,0]): # If we actually detected platforms in this frame...
    points = mstd[0].points # Get the most significant track's polygon points

    # Draw the shape on our track mask
    pts = np.array([[point.x, point.y] for point in points], np.int32)
    pts = pts.reshape((-1, 1, 2))
    cv2.fillPoly(masktracks, [pts], 255)

# As long as we have BOTH a platform and a track detected
# (we need both for the audio)
if (mspd != [0,0] and mstd != [0,0]):

    # Combine our masks together
    maskplatforms = cv2.threshold(maskplatforms, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)[1]
    masktracks = cv2.threshold(masktracks, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)[1]
    img[maskplatforms == 255] = (255, 255, 0) # Will display as yellow
    img[masktracks == 255] = (255, 0, 255) # Will display as blue

    detected = '' # Right or left of the camera?

    # If the track is to the right of the platform...
    if(mspd[0].points[0].x < mstd[0].points[0].x):
        detected = 'R' # We'll set 'detected' to 'R'

    # If the track is to the left of the platform...
    else:
        detected = 'L' # We'll set 'detected' to 'L'

    if dc != None: # as long as the data channel is set up...
        if detected == 'L':
            dc.send("playLeft") # Send the playLeft message to the datachannel

        if detected == 'R':
            dc.send("playRight") # Or send the playRight message to the datachannel

    # Convert our N-d array back to image format with all the masking added and return as
a new frame.
    new_frame = VideoFrame.from_ndarray(img, format="bgr24")
    new_frame.pts = frame.pts

```

```

        new_frame.time_base = frame.time_base
        return new_frame

# These functions allow us to serve elements of the media server as URL paths...

async def index(request):
    content = open(os.path.join(ROOT, "index.html"), "r").read()
    return web.Response(content_type="text/html", text=content)

async def javascript(request):
    content = open(os.path.join(ROOT, "client.js"), "r").read()
    return web.Response(content_type="application/javascript", text=content)

# For MP3s, we need to interpret as binary and send that way:
async def leftFile(request):
    filepath = os.path.join(ROOT, "LeftDetection.mp3")
    with open(filepath, "rb") as file:
        content = file.read()
    return web.Response(body=content, content_type='audio/mpeg')

async def rightFile(request):
    filepath = os.path.join(ROOT, "RightDetection.mp3")
    with open(filepath, "rb") as file:
        content = file.read()
    return web.Response(body=content, content_type='audio/mpeg')

# This is where we set up peer connections between the host and clients
async def offer(request):
    params = await request.json()
    offer = RTCSessionDescription(sdp=params["sdp"], type=params["type"])

    pc = RTCPeerConnection()
    pc_id = "PeerConnection(%s)" % uuid.uuid4()

    pcs.add(pc)

    # Some logging methods
    def log_info(msg, *args):
        logger.info(pc_id + " " + msg, *args)

    log_info("Created for %s", request.remote)

    player = MediaPlayer(os.path.join(ROOT, "demo-instruct.wav"))
    if args.record_to:
        recorder = MediaRecorder(args.record_to)
    else:
        recorder = MediaBlackhole()

    # When the data channel is detected, we'll send a message to the JS console
    @pc.on("datachannel")
    def on_datachannel(channel):
        global dc # We also want to make sure datachannel is global...
        # ...so the VideoStreamTrack class can send audio info

        dc = channel
        channel.send("setGlobal")

    # Event logging methods...

    @pc.on("connectionstatechange")
    async def on_connectionstatechange():
        log_info("Connection state is %s", pc.connectionState)
        if pc.connectionState == "failed":
            await pc.close()
            pcs.discard(pc)

    @pc.on("track")
    def on_track(track):
        log_info("Track %s received", track.kind)

        if track.kind == "audio":
            pc.addTrack(player.audio)
            recorder.addTrack(track)
        elif track.kind == "video":

```



```

        pc.addTrack(
            VideoTransformTrack(
                relay.subscribe(track),
                transform=params["video_transform"]
            )
        )
        if args.record_to:
            recorder.addTrack(relay.subscribe(track))

        @track.on("ended")
        async def on_ended():
            log_info("Track %s ended", track.kind)
            await recorder.stop()

    # Handling the 'offer'
    await pc.setRemoteDescription(offer)

    # Send Answer
    answer = await pc.createAnswer()
    await pc.setLocalDescription(answer)

    # Return our peer connection
    return web.Response(
        content_type="application/json",
        text=json.dumps(
            {"sdp": pc.localDescription.sdp, "type": pc.localDescription.type}
        ),
    )

async def on_shutdown(app):
    # close peer connections
    coros = [pc.close() for pc in pcs]
    await asyncio.gather(*coros)
    pcs.clear()

# Set up arguments for running this script.
# Mainly unchanged from the original example code
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="WebRTC audio / video / data-channels demo"
    )
    parser.add_argument("--cert-file", help="SSL certificate file (for HTTPS)")
    parser.add_argument("--key-file", help="SSL key file (for HTTPS)")
    parser.add_argument(
        "--host", default="0.0.0.0", help="Host for HTTP server (default: 0.0.0.0)"
    )
    parser.add_argument(
        "--port", type=int, default=8080, help="Port for HTTP server (default: 8080)"
    )
    parser.add_argument("--record-to", help="Write received media to a file.")
    parser.add_argument("--verbose", "-v", action="count")
    args = parser.parse_args()

    if args.verbose:
        logging.basicConfig(level=logging.DEBUG)
    else:
        logging.basicConfig(level=logging.INFO)

    if args.cert_file:
        ssl_context = ssl.SSLContext()
        ssl_context.load_cert_chain(args.cert_file, args.key_file)
    else:
        ssl_context = None

    # This links to our earlier serving methods...
    app = web.Application()
    app.on_shutdown.append(on_shutdown)
    app.router.add_get("/", index)
    app.router.add_get("/left", leftFile)
    app.router.add_get("/right", rightFile)
    app.router.add_get("/client.js", javascript)
    app.router.add_post("/offer", offer)

    # This IP address will need to be manually changed on different backend computers.
    web.run_app(

```

```
app, access_log=None, host=" [... IP GOES HERE ...]", port=8080,  
ssl_context=ssl_context  
)
```

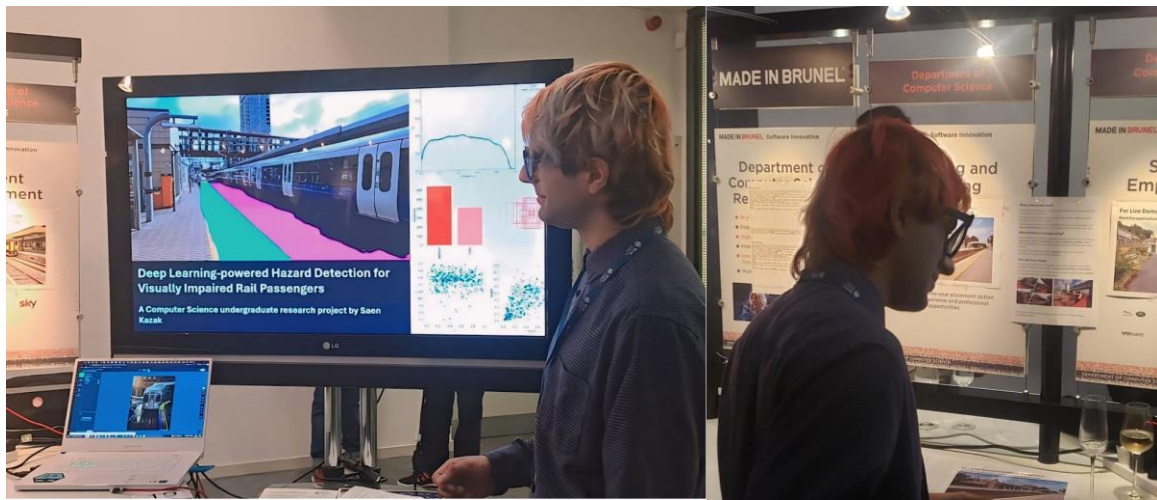
(Adapted from AIORTC, 2021)

C.8.3 A Note About Code

The code shown in both scripts was originally adapted from sample code provided by AIORTC. Although the code has since been extensively modified to fit the use cases of the project and is almost unrecognisable from the sample code, save for some unchanged functions, both original source codes are cited and provided in the references.

Appendix D : Made in Brunel

I was fortunate enough to have been given the opportunity to document and showcase this project at the annual Made in Brunel: Software Innovation exhibition, with a large screen set up to showcase the workings of the system along with some testing results. In addition to this, I was able to recruit 8 of the total 13 participants to partake in its testing. This was a fantastic opportunity to discuss the work with others. I received many challenging questions about the workings of the system; something I thoroughly enjoyed discussing.



Luckily for me, the system worked fully as intended. I would like to give additional thanks in this section to Dr Mahir Arzoky for organising the event, Dr Nadine Aburumman for help in setting up the screen, as well as all other staff, students and friends who stopped by to ask questions.



The project ended up winning Professor Kate Hone's Head of Department Sponsorship Prize at the event.